

Deep Neural Networks

Computer Vision and Artificial Intelligence

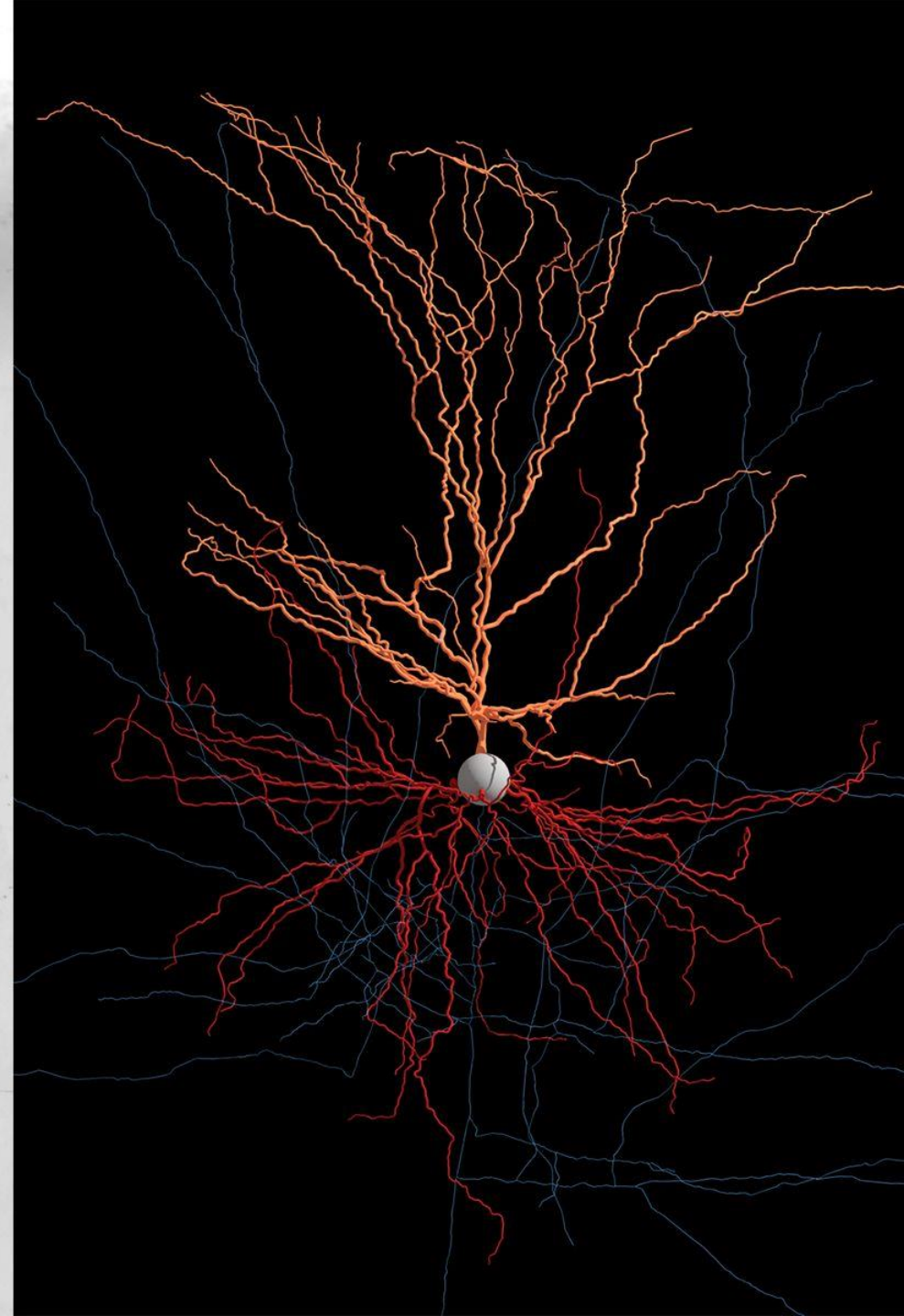
Draft version do not print

Dr Varun Ojha

varun.ojha@ncl.ac.uk

School of Computing

Newcastle University



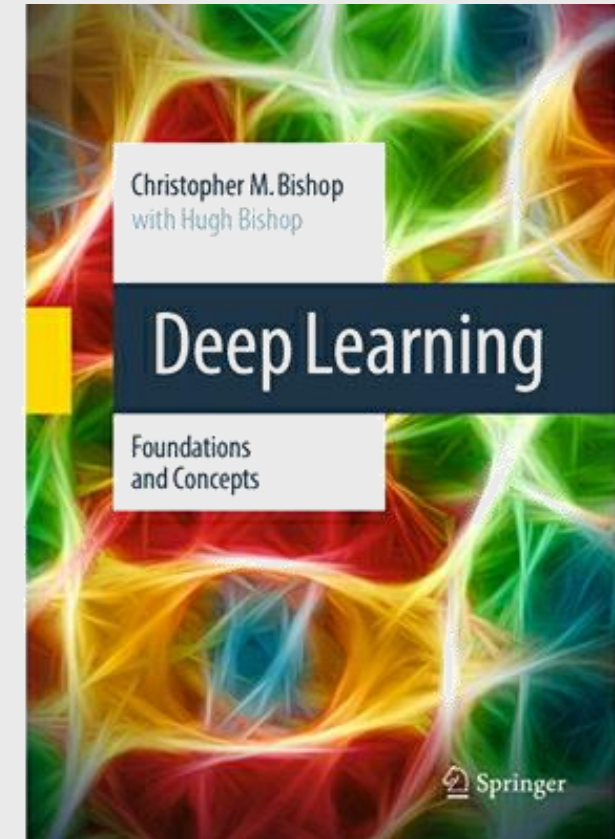
Learning objectives (deep neural networks)

By the end of this week, you will be able to

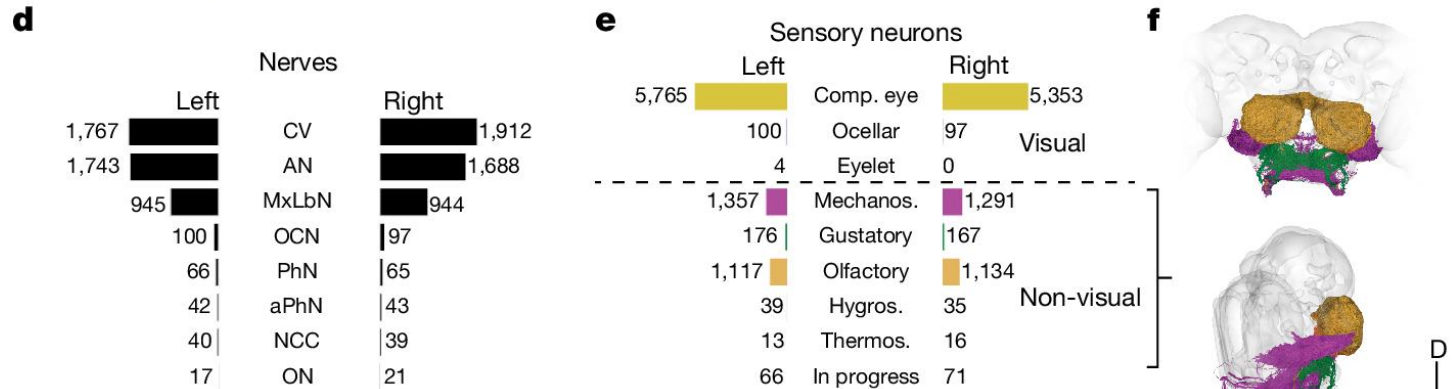
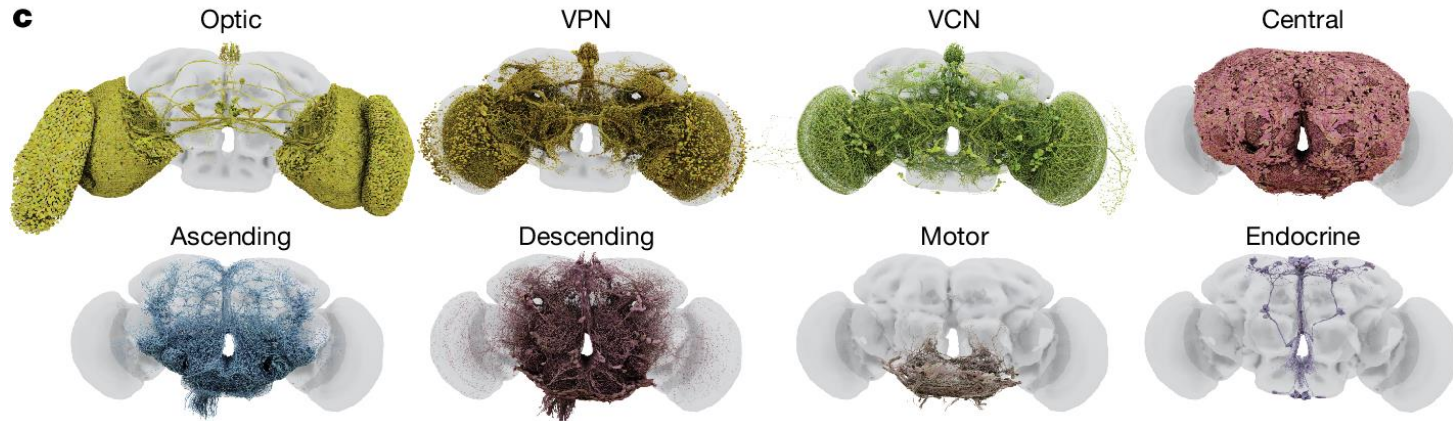
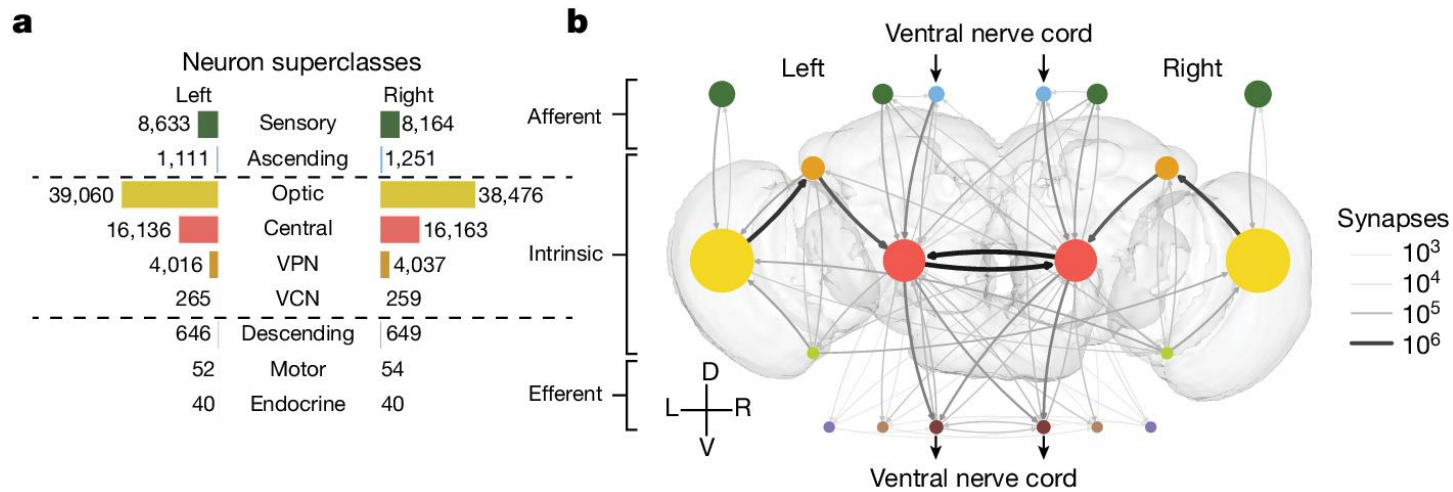
- Learn ‘concepts of learning’ in neural networks
- Understand gradient descent and backpropagation algorithms
- Distinguish shallow and deep neural network architectures
- Identify challenges of learning/training a deep neural network
- Apply and evaluate neural networks for a pattern recognition (solve image classification) problem – *in practical session*

Content (neural networks)

- **Part 1: Introduction**
- **Part 2: Fundamental Theory**
 - Supervised Learning problem
 - Design of learning process
 - Gradient descent optimization
- **Part 3: Neural Network Architectures**
 - Neural Network Components
 - The Backpropagation algorithm
 - Deep Neural Networks
 - Challenges of training a Neural Networks
- **Part 4: Practical Exercise**



I recommend *Deep Learning* by C Bishop
<https://www.bishopbook.com/>



Part 1

Introduction

Brain / Intelligence

Image only for illustration/motivation
not for study

Intrinsic Intelligence?

Inside a baby's mind

Experiment:
Warneken & Tomasello (2006)

10:24
AM



Video Source:
<https://www.youtube.com/watch?v=cUWllxpUfM0>
(Accessed on 21 Feb 2021)

Causal understanding of water displacement by a crow

Experiment: Sarah et al. (2014), Auckland and Cambridge
Video Source: <https://www.youtube.com/watch?v=ZerUbHmuY04>



Learning by example

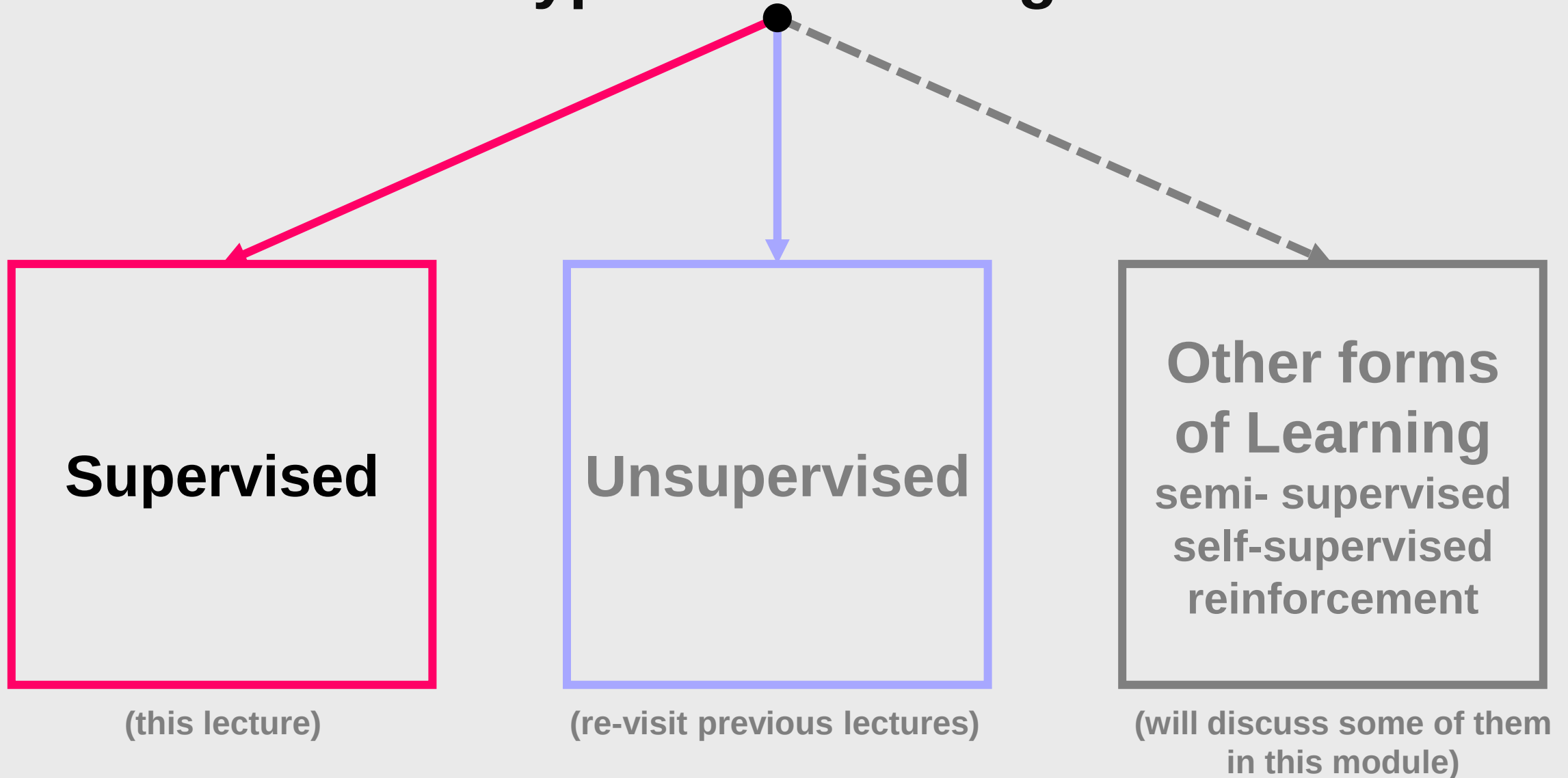
Learning / Training

10:24
AM

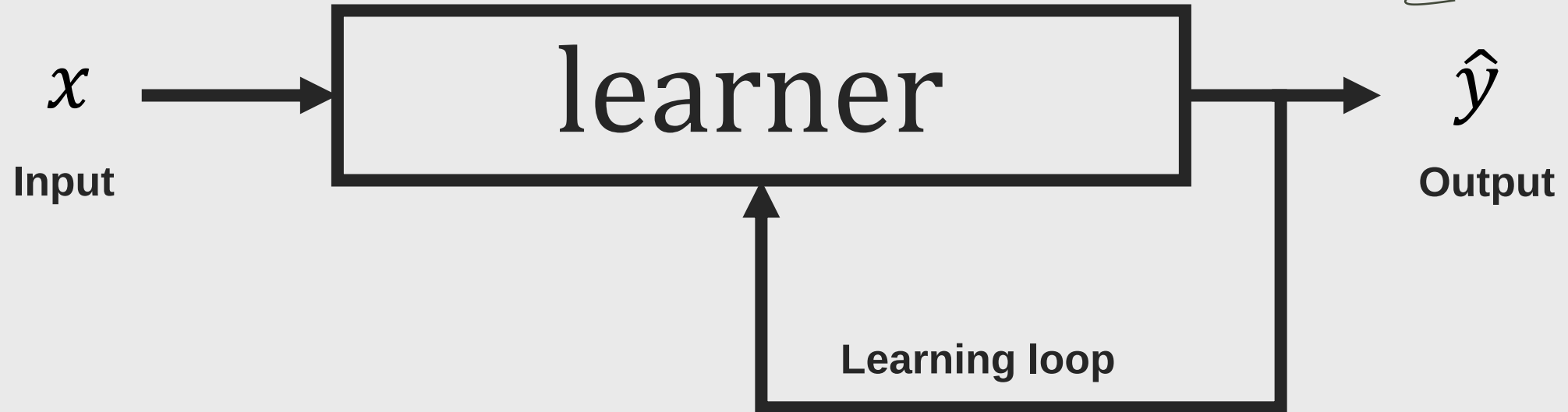
Video Source:
<https://www.youtube.com/watch?v=Ak7bPuR2rDw>
(Accessed on 21 Feb 2021)



Types of Learning



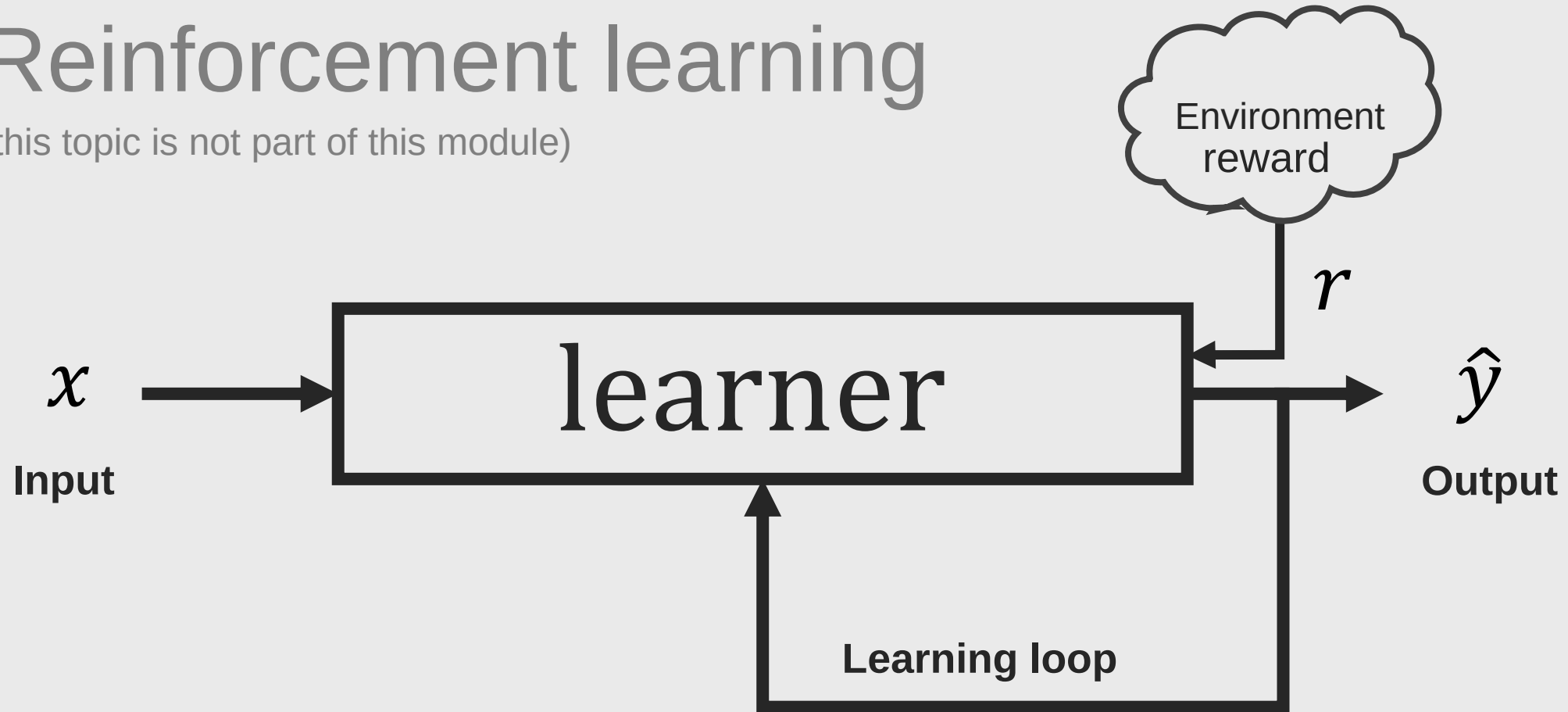
Unsupervised



Re-visit previous lectures: Clustering (K-Means, DB Scan); Dimensionality reduction; Anomaly detection, Variational Autoencoder (VAE; *is not a part of this module*):

Reinforcement learning

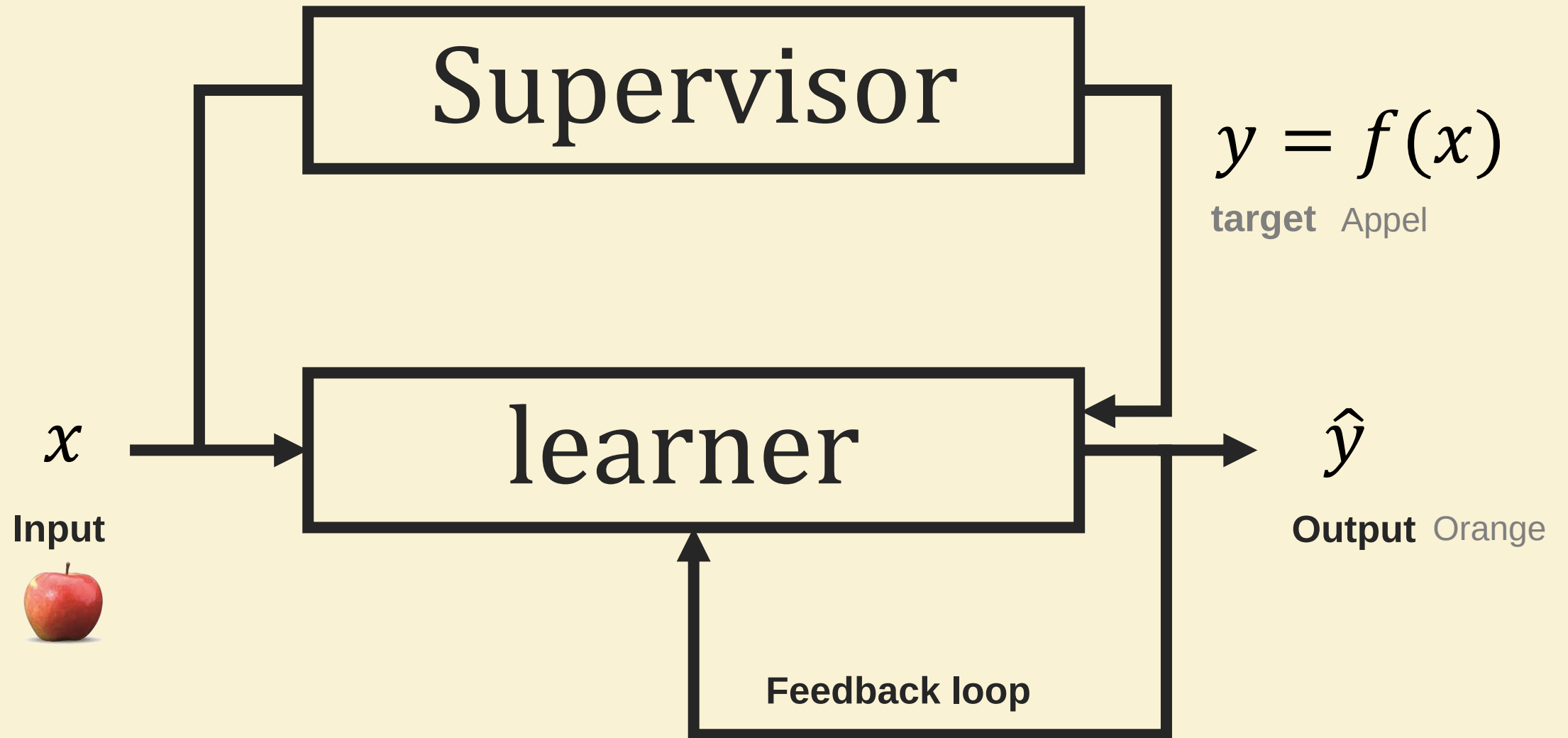
(this topic is not part of this module)



Example are game playing (most popular games are 'Atari' and 'Hide and Seek' where reinforcement learning (RL) is used, or RL is heavily used in robotics for learning control and actions)

Explore (not part of this module): Reinforcement Learning: An Introduction by Richard S Sutton

Supervised



Learning: $g(\mathbf{X}) \sim f(\mathbf{X})$

We need to search a function $g(X)$ that can approximate $f(X)$

Example Training Task: AND Logic Problem

	Colour	Shape	Fruit Name
	x_1	x_2	y
Fruits $\mathbf{x}_1$:	0	0	0
$\mathbf{x}_2$:	0	1	0
$\mathbf{x}_3$:	1	0	0
$\mathbf{x}_4$:	1	1	1

Input $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4)^T$, $\mathbf{x}_i = (x_{i1}, x_{i2})$,
Output $y = \{0,1\}$

Number of Inputs $d = 2$

Each input x takes values either 0 or 1

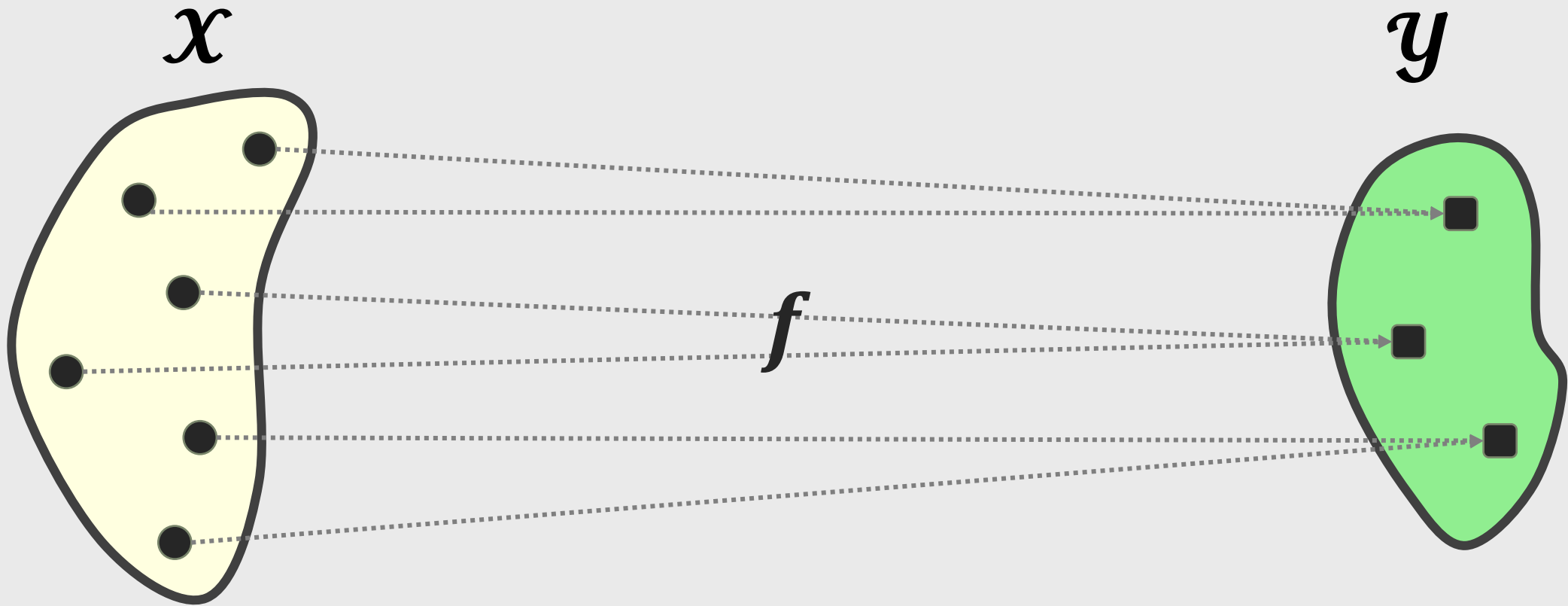
Input-space $\mathcal{X} = 2^d = 2^2 = 4$

Number of outputs 1

Output y takes 2 options from $\{0,1\}$ and 4 examples
the concept-space $\mathcal{C} = \text{options}^{\text{examples}} = 2^4 = 16$

Learning $f : \mathcal{X} \rightarrow \mathcal{Y}$

Supervised learning is a mapping f of inputs $\mathcal{X}$ to outputs $\mathcal{Y}$

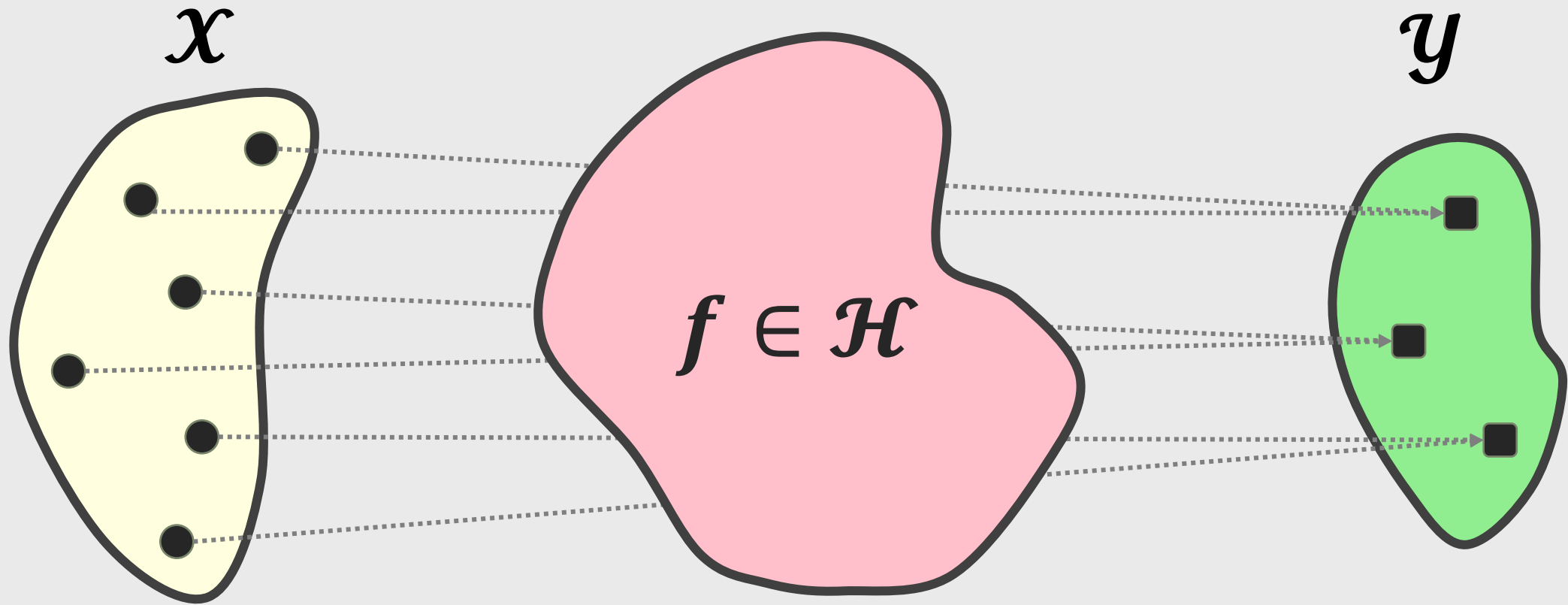


Inputs $\mathbf{X} \in$ Input space $\mathcal{X}$

outputs $\mathbf{y} \in$ output space $\mathcal{Y}$

Learning $f : X \rightarrow y$

We need to find the unknown target function f that maps x to y

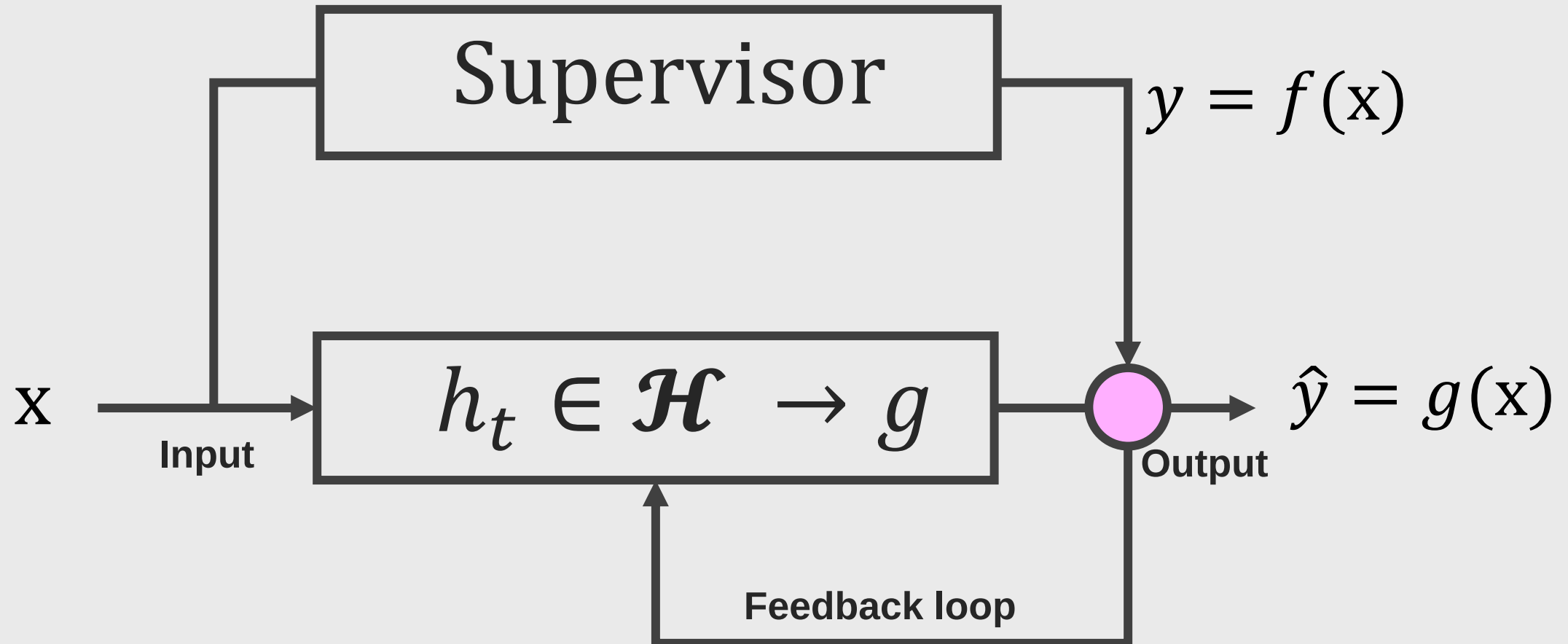


Inputs $X \in$ Input space $\mathcal{X}$

model space $\mathcal{H}$

outputs $y \in$ output space $\mathcal{Y}$

Search a function $g: X \rightarrow y$ that approximates $f(x)$



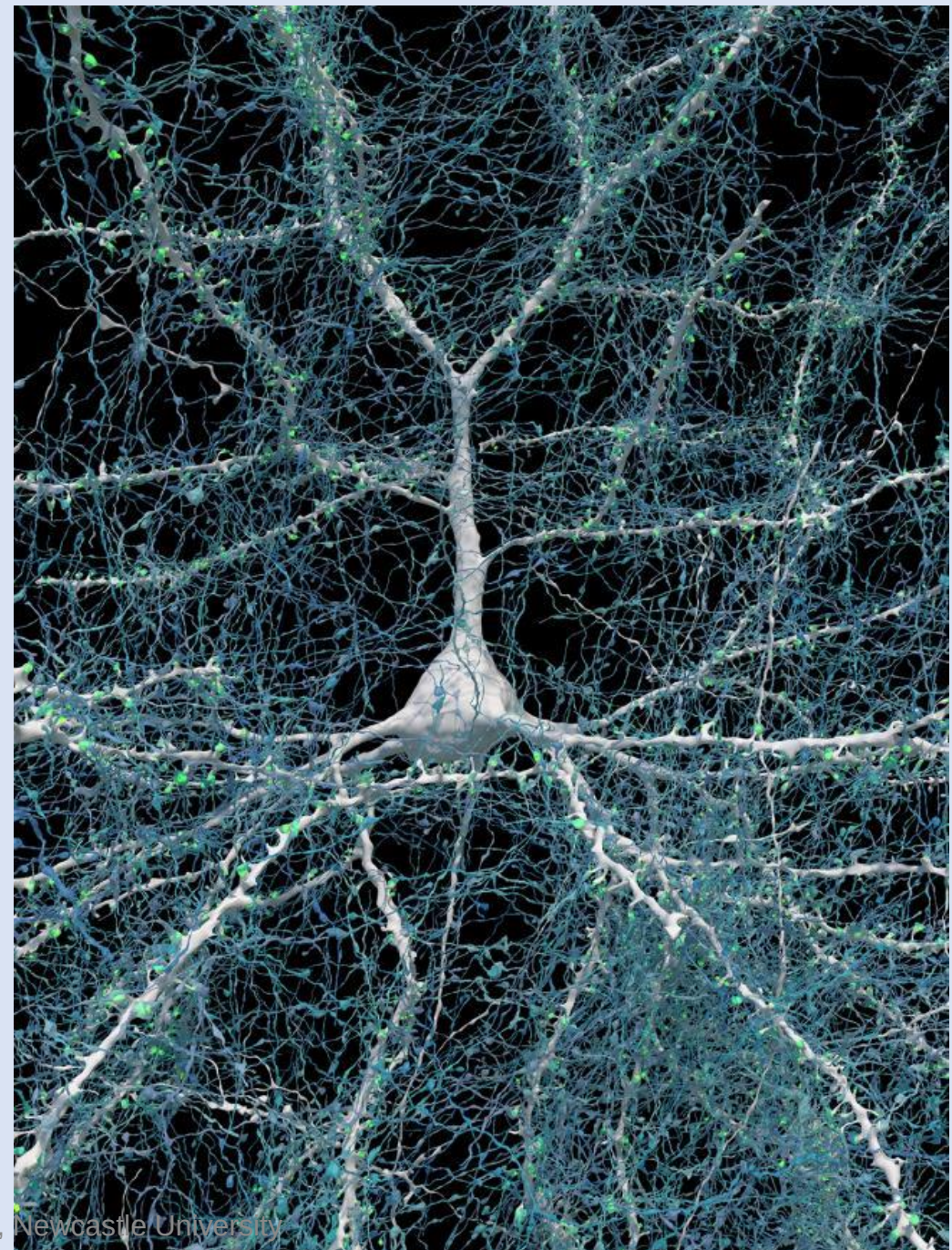
Part 2

Learning

Algorithms

Image only for illustration/motivation
not for study

A single neuron with 5,600 nerve fibres,
Google Research a7 Harvard University
Dr Varun Ojha, Newcastle University



Requirements of Learning

Learning needs to

Represent a model (use a neural network architecture, deep neural networks)

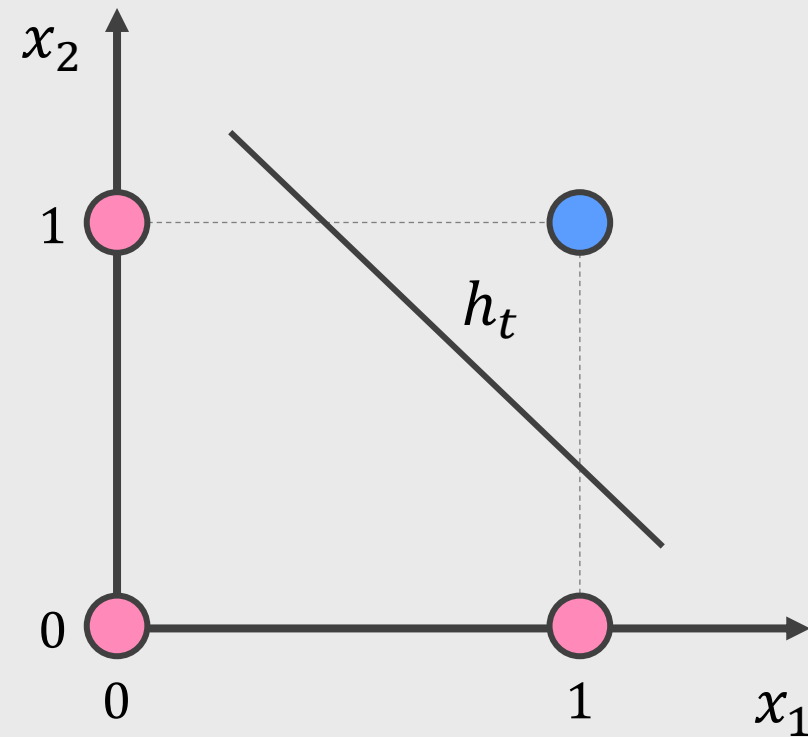
Evaluate the model (use a loss/cost function, e.g., Cross Entropy or MSE)

Optimize the model (use an optimizer, e.g., backpropagation – Adam or SGD)

Represent a model

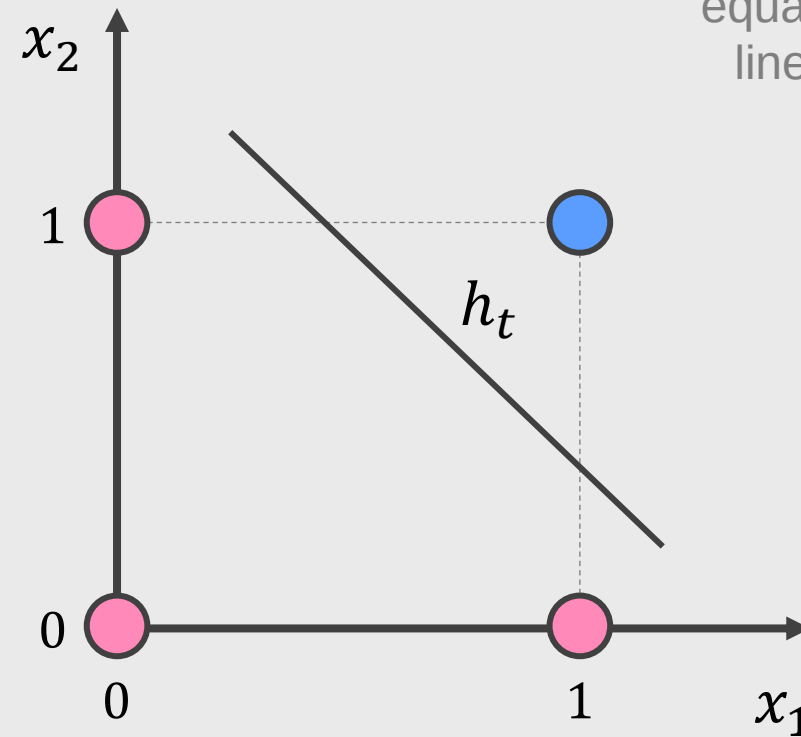
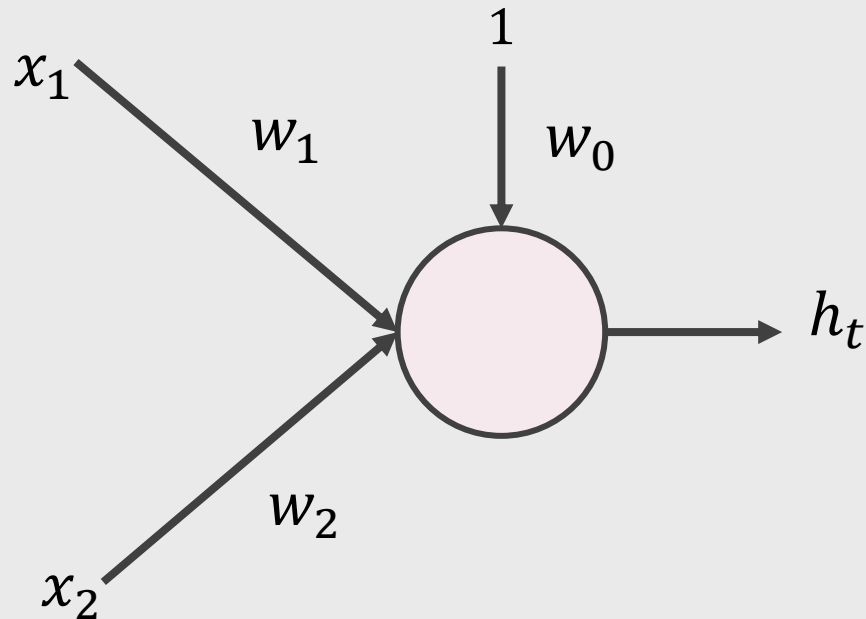
A line separating data can be considered as a model

$\mathcal{D}$		x_1	x_2	y
	1:	0	0	0
	2:	0	1	0
	3:	1	0	0
	4:	1	1	1



Represent a model, $h_t = w_1x_1 + w_2x_2 + w_0$

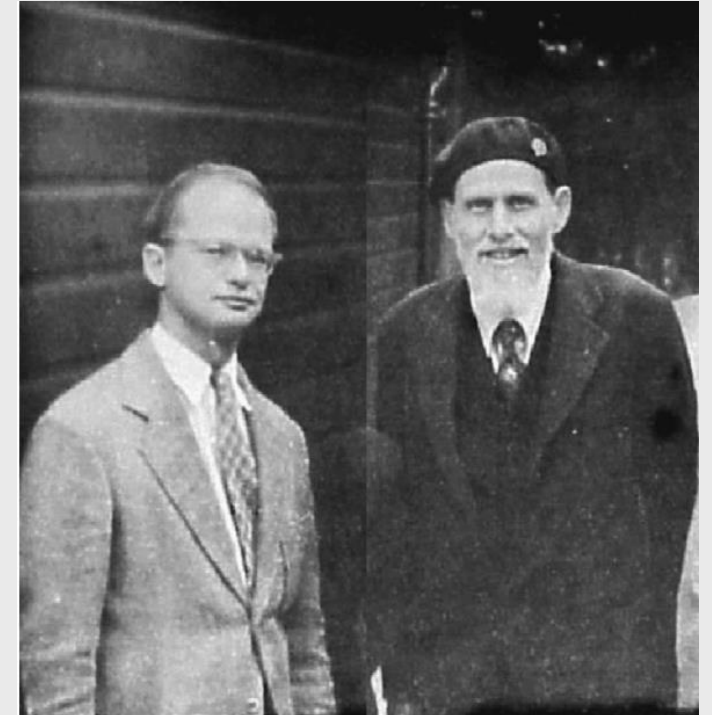
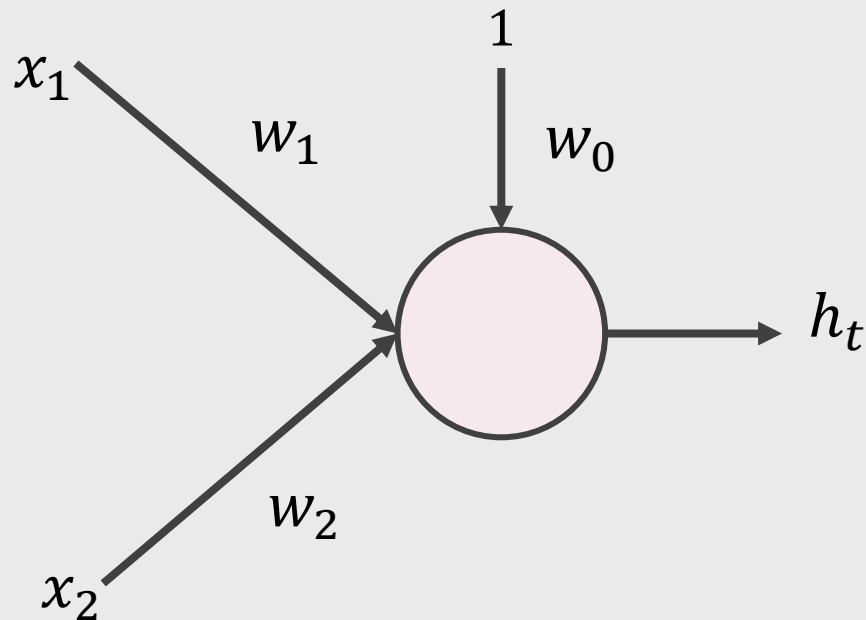
A line separating data can be considered a model which equivalent to a single neuron or a perceptron



This is basically equation of line or a linear regression

Represent a model (a neuron)

A line separating data can be considered a model which equivalent to a single neuron or a perceptron



McCulloch (right) and Pitts (left) in 1949

McCulloch & Pitts Publish the First Mathematical Model of a Neural Network

We will discuss more detail of this in part 3

Represent a model $h_t \in H$

A line separating data can be considered a model which equivalent to a single neuron or a perceptron

Perceptron is a simple linear combination of inputs, which is written as:

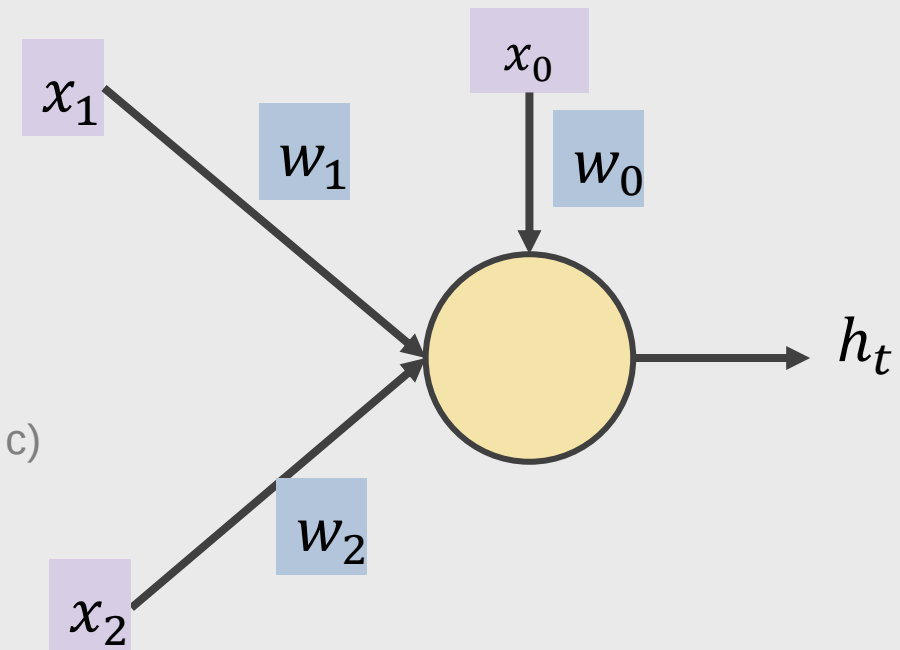
$$h_t = g(x) = \sum_{i=1}^d w_i x_i \geq x_0 w_0$$

This equation is also equivalent to linear regression ($y = mx + c$)

where w_0 is a threshold.

Real bottleneck of
Deep Learning

The model h_t has the weights w_i and the threshold w_0 as its **trainable parameters**.



Model h_t as a **perceptron / single neuron**.

Read: Sec 4.1 and Sec 5.1, Deep Learning by C Bishop

Represent a model $h_t \in H$

A line separating data can be considered as a model

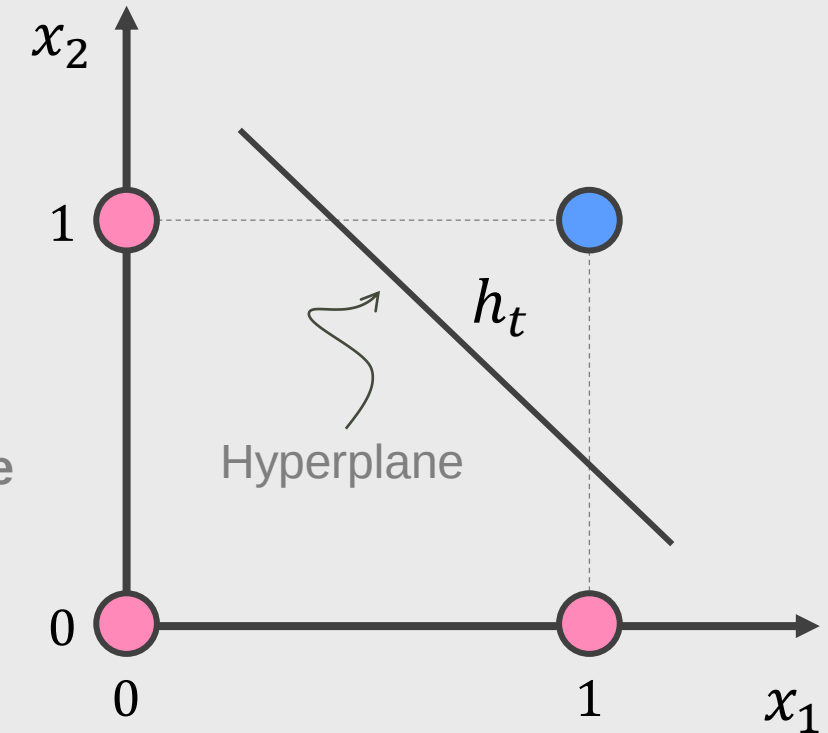
A model h_t as a perceptron.

$$\sum_{i=1}^d w_i x_i \geq x_0 w_0$$

$$\sum_{i=1}^d w_i x_i - x_0 w_0 = 0 \quad \text{This equation is also called a **hyperplane**}$$

For an artificial input (also called bias) $x_0 = 1$ we have:

$$\sum_{i=0}^d w_i x_i = 0 \quad \text{This is an equation of a single neuron}$$



hyperplane is a decision boundary

Which model $h_t \in H$ to pick?

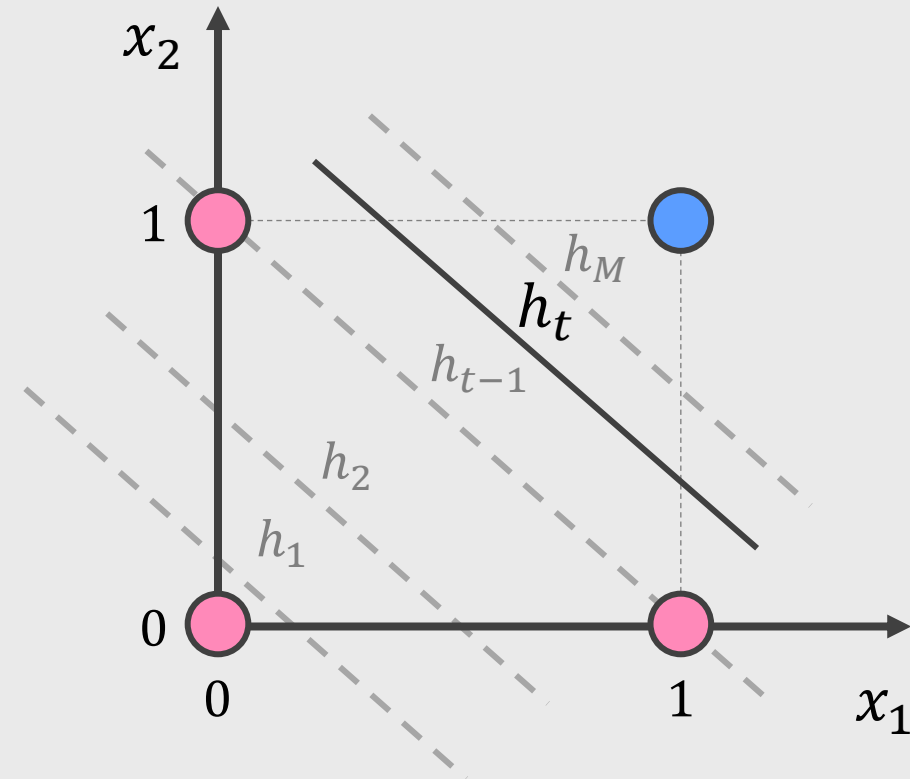
How to **evaluate** a model: compute cost of choosing a model

	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	0
3:	1	0	0
4:	1	1	1

$\mathcal{D}$

Cost function such as the error rate:

$$E(h_t(\mathcal{D})) = \frac{1}{N} \sum_{j=1}^N (g_{\mathbf{w}}(\mathbf{x}_j) \neq f(\mathbf{x}_j))$$



Optimise model $h_t \in H$ by minimizing error

How to optimize a model: compute cost of adjust the model weights

Function g of the model has parameter $\mathbf{w}$:

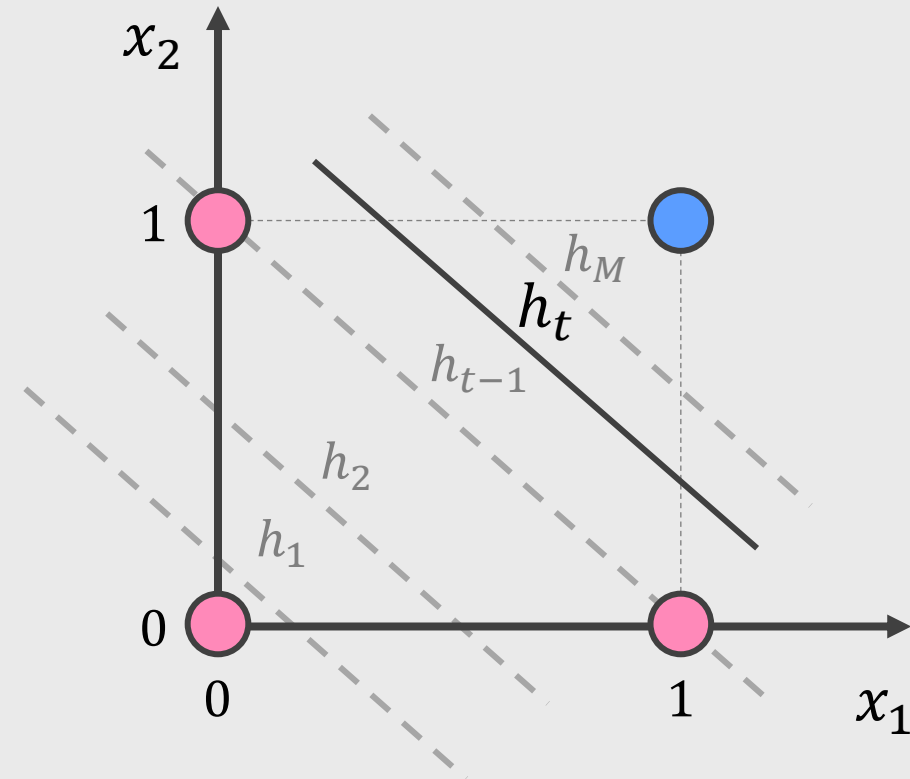
$$\hat{y} = g_{\mathbf{w}}(\mathbf{x}) = \sum_{i=0}^d w_i x_i = 0$$

Simple algorithm:

Repeat parameter $\mathbf{w}$ update for $t = 2, 3, \dots, M$ as:

$$\mathbf{w}_t = \mathbf{w}_{t-1} + \hat{y} \mathbf{x},$$

Until the error rate $E(h_t(\mathcal{D}))$ is acceptable or close to zero.



Does error $E(h_t(\mathcal{D}))$ minimization work?

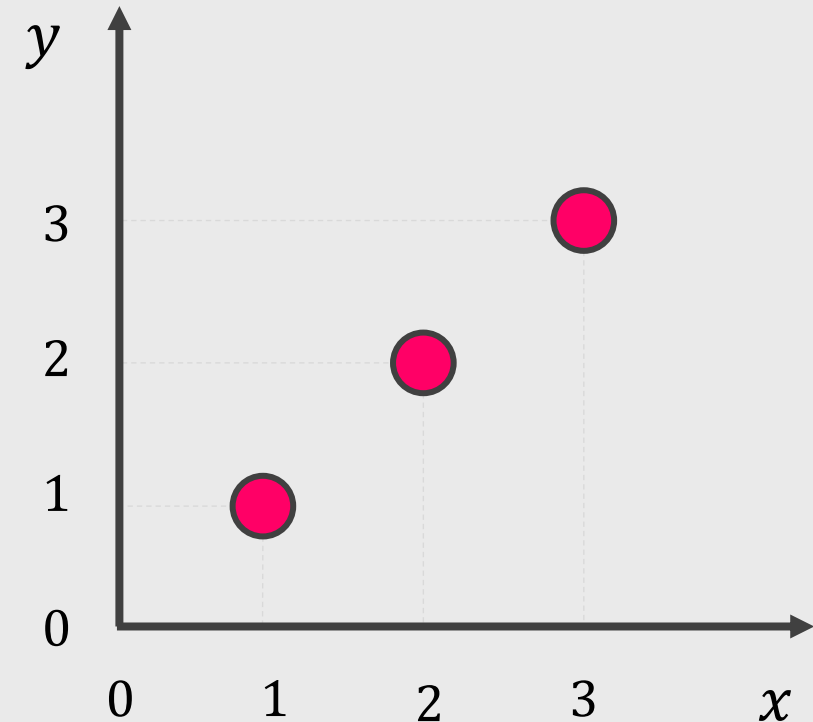
Let's see an example (house price):

	$x = \text{area}(m^2)$	$y = \text{price (in £)}$
1:	1000	100K
2:	2000	200K
3:	3000	300K

Now, the **cost function** is a **mean squared error**:

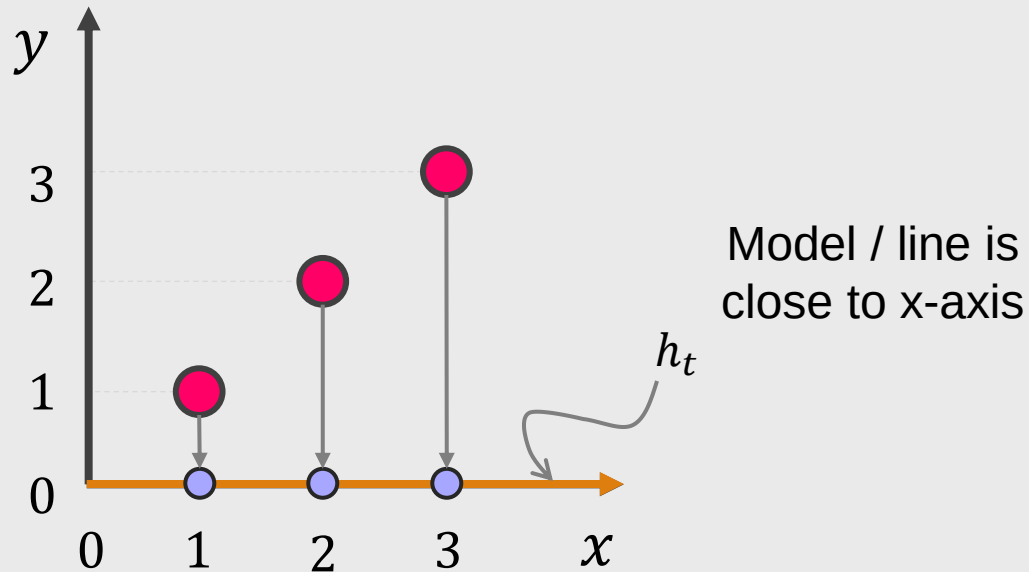
$$E(h_t(\mathbf{x})) = \frac{1}{2N} \sum_{j=1}^N (g_{\mathbf{w}}(\mathbf{x}_j) - f(\mathbf{x}_j))^2$$

Note that y and x values are simplified to 1, 2 and 3



Does error $E(h_t(\mathcal{D}))$ minimization work?

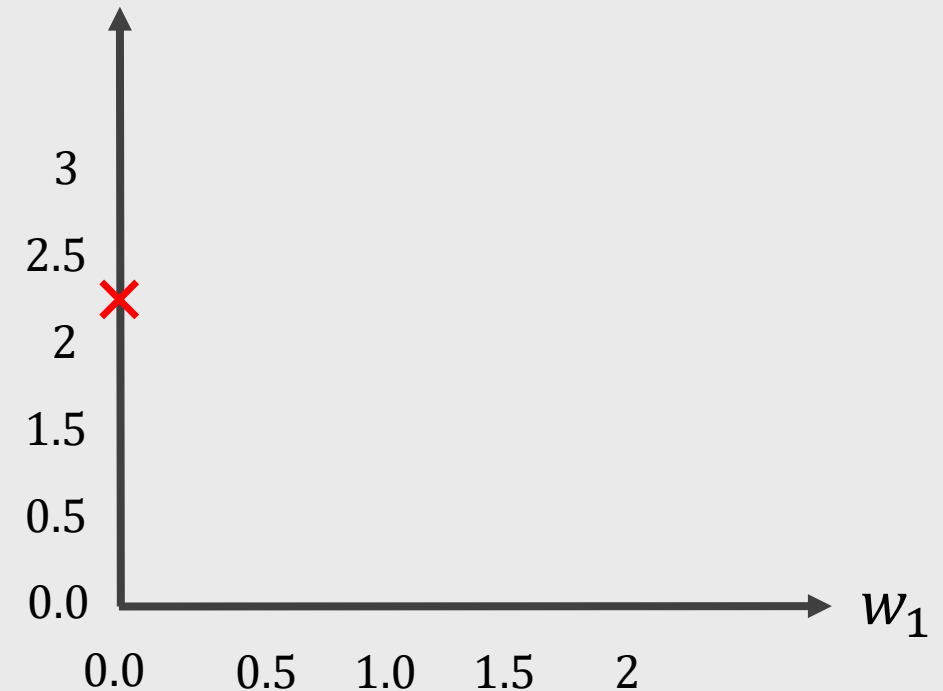
Note that y and x values are simplified to 1, 2 and 3



Model h_t for $w_0 = 0$ and $w_1 = 0.0$:

$$g_w(x_i) = w_0 + w_1 x_i \text{ for } i = 1, 2, 3$$

$$E(g_{w_1}(x)) = \frac{1}{2N} \sum_{j=1}^N (f(\mathbf{x}_j) - g_w(\mathbf{x}_j))^2$$

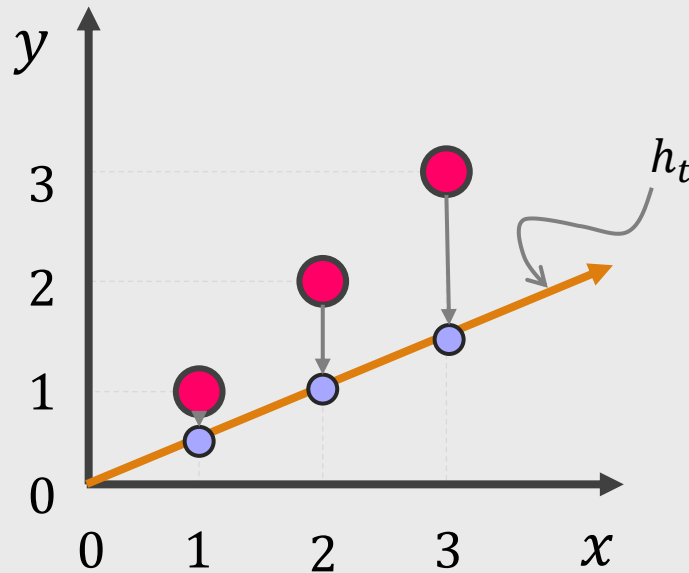


Error $E(w_1)$ for $w_0 = 0$ and $w_1 = 0.0$:

$$E(g_w(\mathbf{x})) = \frac{(1-0)^2 + (2-0)^2 + (3-0)^2}{2 \cdot 3} = 2.33$$

Does error $E(h_t(\mathcal{D}))$ minimization work?

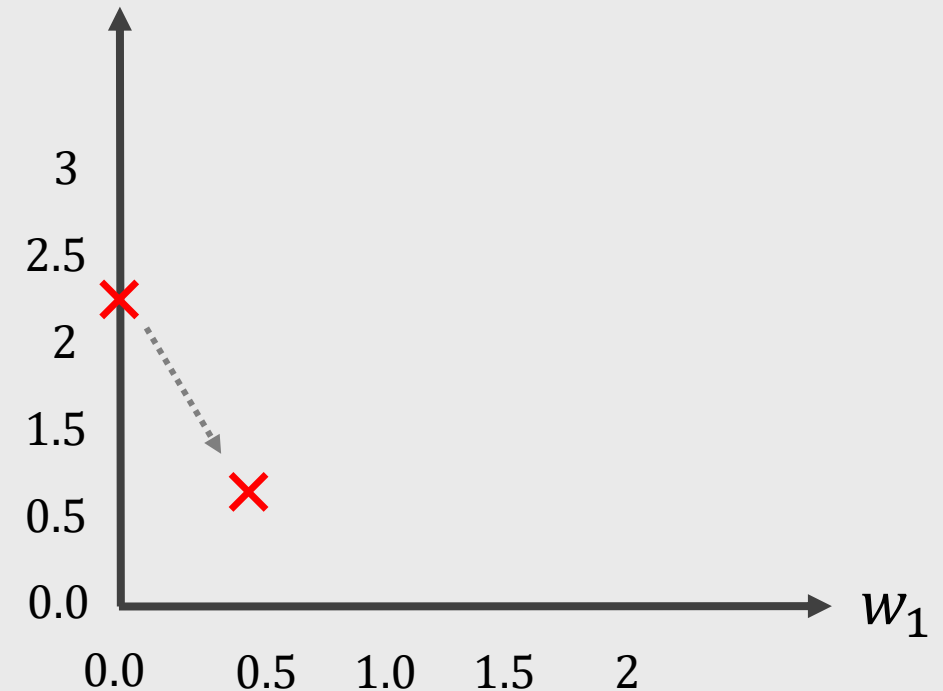
Note that y and x values are simplified to 1, 2 and 3



Model h_t for $w_0 = 0$ and $w_1 = 0.5$:

$$g_w(x_i) = w_0 + w_1 x_i \text{ for } i = 1, 2, 3$$

$$E(g_{w_1}(x)) = \frac{1}{2N} \sum_{j=1}^N (f(\mathbf{x}_j) - g_w(\mathbf{x}_j))^2$$

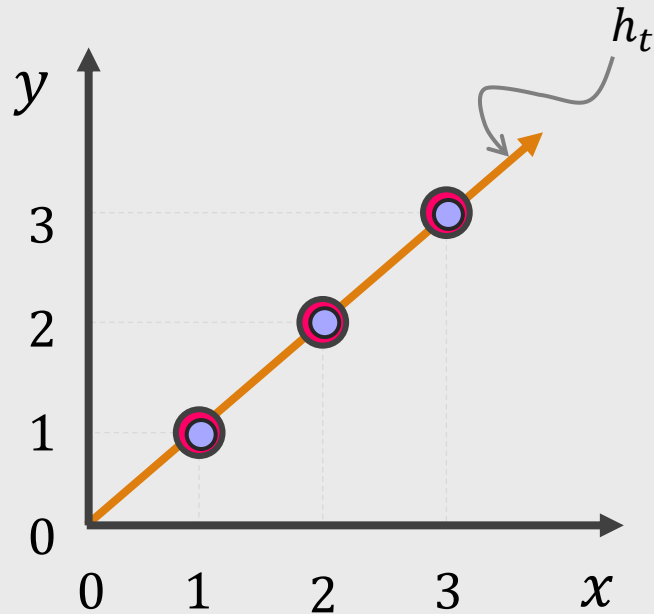


Error $E(w_1)$ for $w_0 = 0$ and $w_1 = 0.5$:

$$E(g_w(\mathbf{x})) = \frac{(1-0.5)^2 + (2-1)^2 + (3-1.5)^2}{2 \cdot 3} = 0.58$$

Does error $E(h_t(\mathcal{D}))$ minimization work?

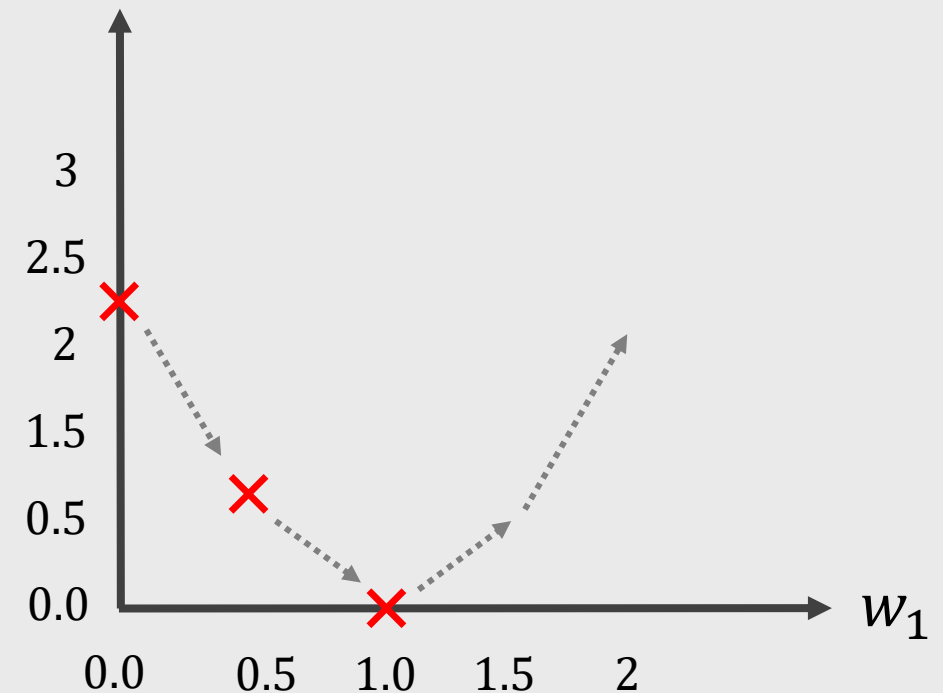
Note that y and x values are simplified to 1, 2 and 3



Model h_t for $w_0 = 0$ and $w_1 = 1$:

$$g_w(x_i) = w_0 + w_1 x_i \text{ for } i = 1, 2, 3$$

$$E(g_{w_1}(x)) = \frac{1}{2N} \sum_{j=1}^N (f(\mathbf{x}_j) - g_w(\mathbf{x}_j))^2$$

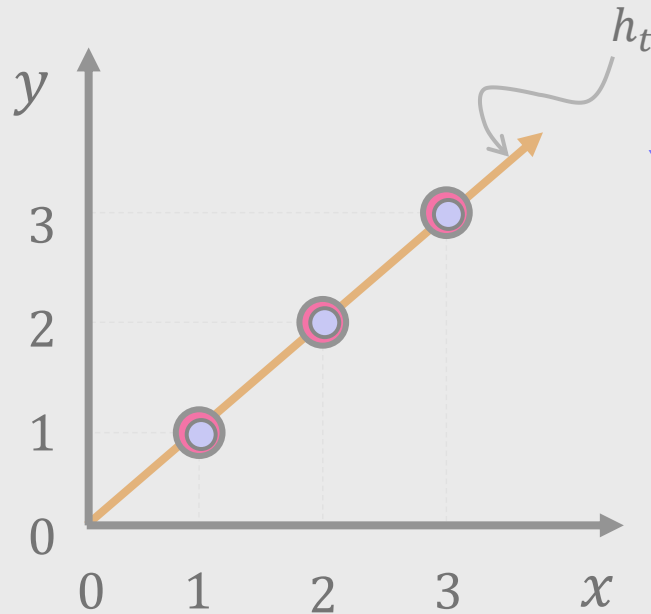


Error $E(w_1)$ for $w_0 = 0$ and $w_1 = 1$:

$$E(g_w(\mathbf{x})) = \frac{(1-1)^2 + (2-2)^2 + (3-3)^2}{2 \cdot 3} = 0.0$$

Does error $E(h_t(\mathcal{D}))$ minimization work?

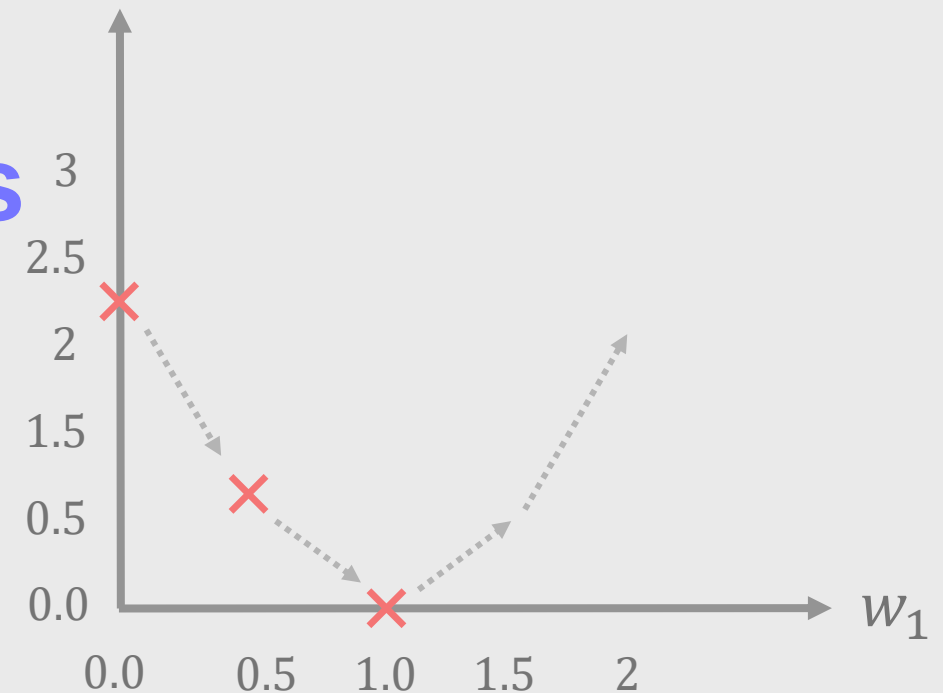
Note that y and x values are simplified to 1, 2 and 3



Yes, it does

But how much weight to change in each step?

$$E(g_{w_1}(x)) = \frac{1}{2N} \sum_{j=1}^N (f(\mathbf{x}_j) - g_w(\mathbf{x}_j))^2$$



Model h_t for $w_0 = 0$ and $w_1 = 1$:

$$g_w(x_i) = w_0 + w_1 x_i \text{ for } i = 1, 2, 3$$

Error $E(w_1)$ for $w_0 = 0$ and $w_1 = 1$:

$$E(g_w(\mathbf{x})) = \frac{(1-1)^2 + (2-2)^2 + (3-3)^2}{2 \cdot 3} = 0.0$$

Optimizer (Learning Algorithm)

Function g of the model has parameter $\mathbf{w}$:

$$g_{\mathbf{w}}(\mathbf{x}) = \sum_{i=0}^d w_i x_i = 0$$

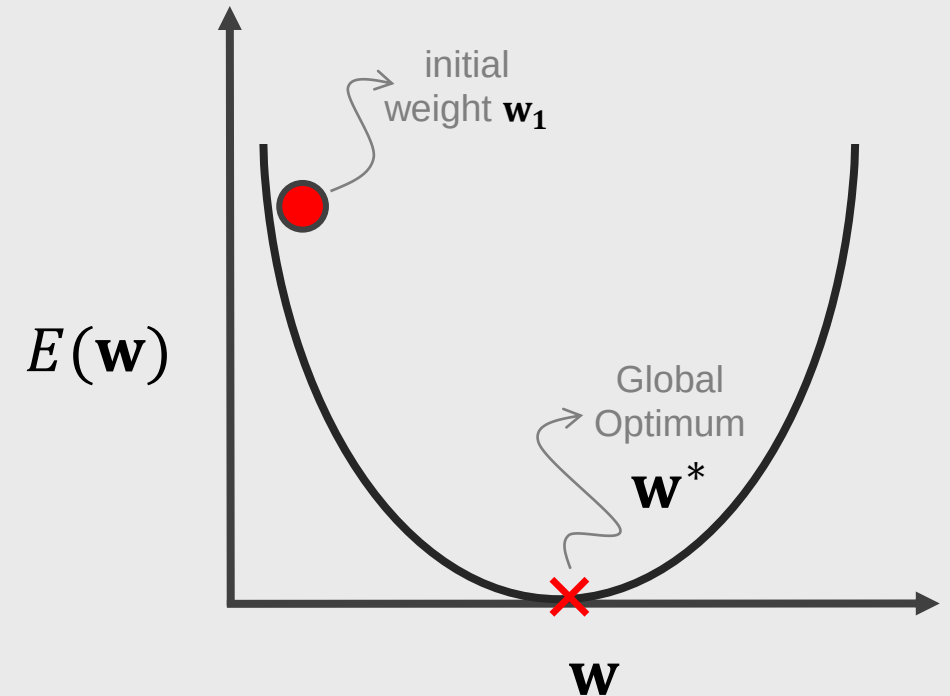
We consider cost function is continuously differentiable

E.g., the means squared error as the **cost function**

$$E(\mathbf{w}) = \frac{1}{2N} \sum_{j=1}^N (g_{\mathbf{w}}(x_j) - f(x_j))^2$$

Therefore, for optimization we want $E(\mathbf{w}^*) \leq E(\mathbf{w})$

We solve an unconstrained optimization problem that minimizes cost $E(\mathbf{w})$ with respect to weight vector $\mathbf{w}$ such that gradient of error/cost is zero, i.e., $\nabla E(\mathbf{w}^*) = \mathbf{0}$



Optimizer: Gradient Descent

We generate a sequence of $\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2 \dots$ such that:

$$E(\mathbf{w}_t) < E(\mathbf{w}_{t-1})$$

where $\mathbf{w}_{t-1}$ is old weight and $\mathbf{w}_t$ updated weight by a factor $\Delta\mathbf{w}_t$ (also called weight change (step) at t)

Learning Algorithm

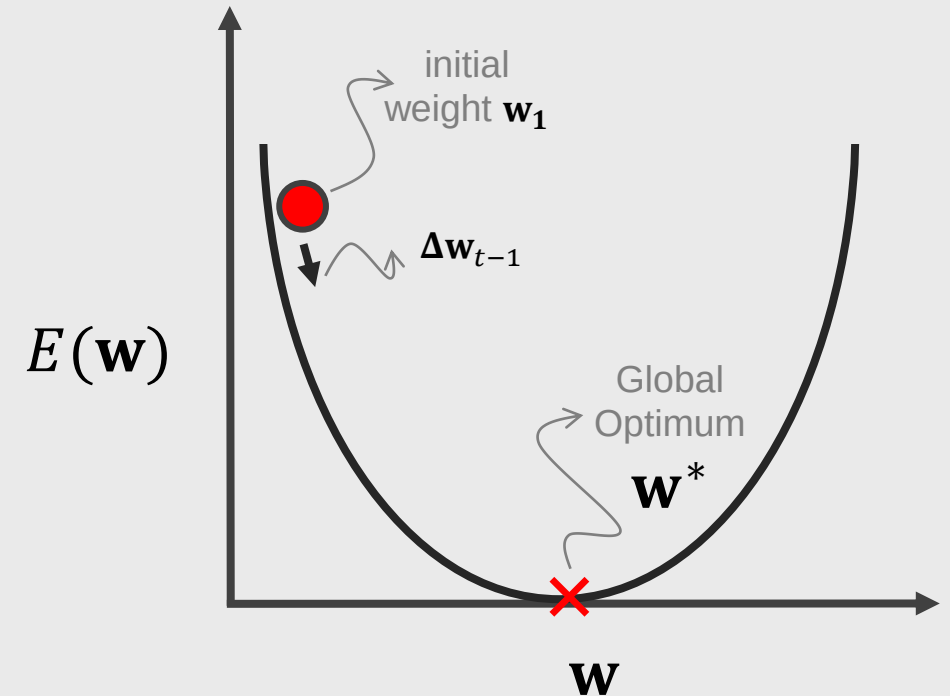
Repeat parameter $\mathbf{w}$ update for $t = 2, 3, \dots, M$.

$$\mathbf{w}_t = \mathbf{w}_{t-1} + \Delta\mathbf{w}_t$$

for learning rate η

See previous slides

Until error rate $E(g_{\mathbf{w}}(\mathbf{x}))$ is acceptable or goes to zero.



First gradient decent algorithms: A. Cauchy. M´ethode g´en´erale pour la r´esolution des syst`emes d´equations simultan´ees. C. R. Acad. Sci. Paris, 25:536–538, 1847. (in French)

Optimizer: Gradient Descent

Function g of the model has parameter $\mathbf{w}$:

$$g_{\mathbf{w}}(\mathbf{x}) = \sum_{i=0}^d w_i x_i = 0$$

Learning Algorithm

Repeat parameter $\mathbf{w}$ update for $t = 2, 3, \dots, M$.

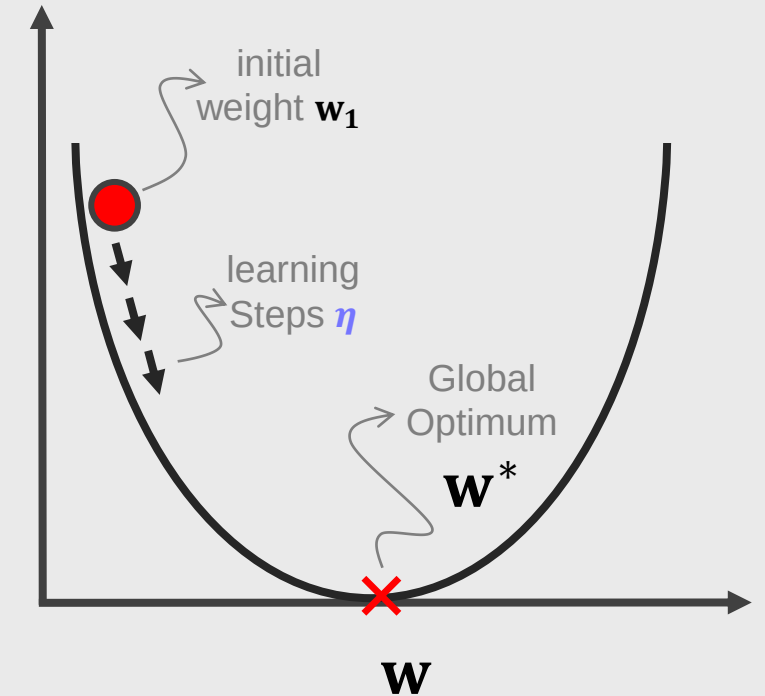
Gradient of error with respect to weights: $-\nabla E(W) = \Delta W$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\partial E(g_{\mathbf{w}}(\mathbf{x}))}{\partial \mathbf{w}_t} \mathbf{x} \text{ for learning rate } \eta$$

Considering function $g_{\mathbf{w}}(\mathbf{x})$, the differentiation $\frac{\partial g}{\partial \mathbf{w}_t} = \mathbf{x}$

Until error rate $E(g_{\mathbf{w}}(\mathbf{x}))$ is acceptable or goes to zero.

$E(\mathbf{w})$



Steepest descent

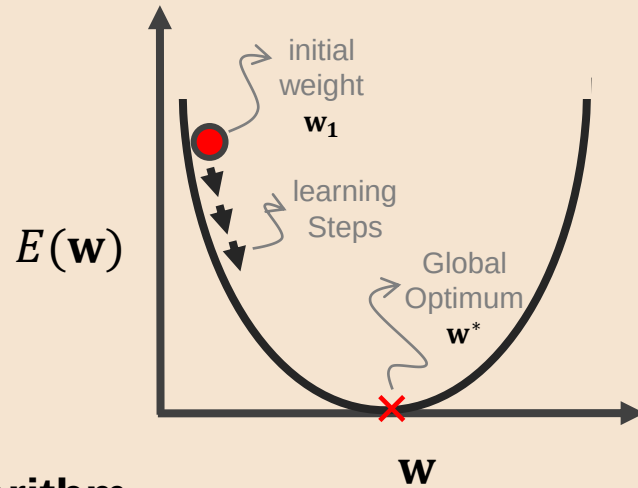


Image Source: <https://perceptrondemo.com/assets/1960-kelley-07dff188.pdf>

Gradient Theory of Optimal Flight Paths

HENRY J. KELLEY¹

Grumman Aircraft Engineering Corp.
Bethpage, N. Y.

An analytical development of flight performance optimization according to the method of gradients or "method of steepest descent" is presented. Construction of a minimizing sequence of flight paths by a stepwise process of descent along the local gradient direction is described as a computational scheme. Numerical application of the technique is illustrated in a simple example of orbital transfer via solar sail propulsion. Successive approximations to minimum time planar flight paths from Earth's orbit to the orbit of Mars are presented for cases corresponding to free and fixed boundary conditions on terminal velocity components.

Presented at the ARS Semi-Annual Meeting, May 9-12, 1960, Los Angeles, Calif.

¹ Section Leader, Systems Research. Member ARS.

² Numbers in parentheses indicate References at end of paper.

OCTOBER 1960

Algorithm

Initial guess w_0 at step $t = 0$

Repeat parameter w update for $t = 2, 3, \dots, M$.

it's the direction of steepest-descent (opposite to the gradient)

$$w_t = w_{t-1} - \eta \frac{\partial E(g_w(x))}{\partial w_t} x \text{ for learning rate } \eta$$

Until error rate $E(g_w(x))$ is acceptable or goes to zero

Least-mean-square (LMS) algorithms

Widrow-Hoff Rule / Delta Rule

Only works for single neuron learning. Hence this algorithm is also called Adaptive Linear Neuron (ADALINE) - Year 1960

LMS Algorithm

Initial guess w_0 at step $t = 0$

Repeat parameter w update for $t = 2, 3, \dots, M$.

$$w_t = w_{t-1} - \eta x_t (f(x_t) - g_w(x_t)x_t) \text{ for learning rate } \eta$$

target output predicted output
 ↖ ↖

Until error rate $E(g_w(x))$ is acceptable or goes to zero

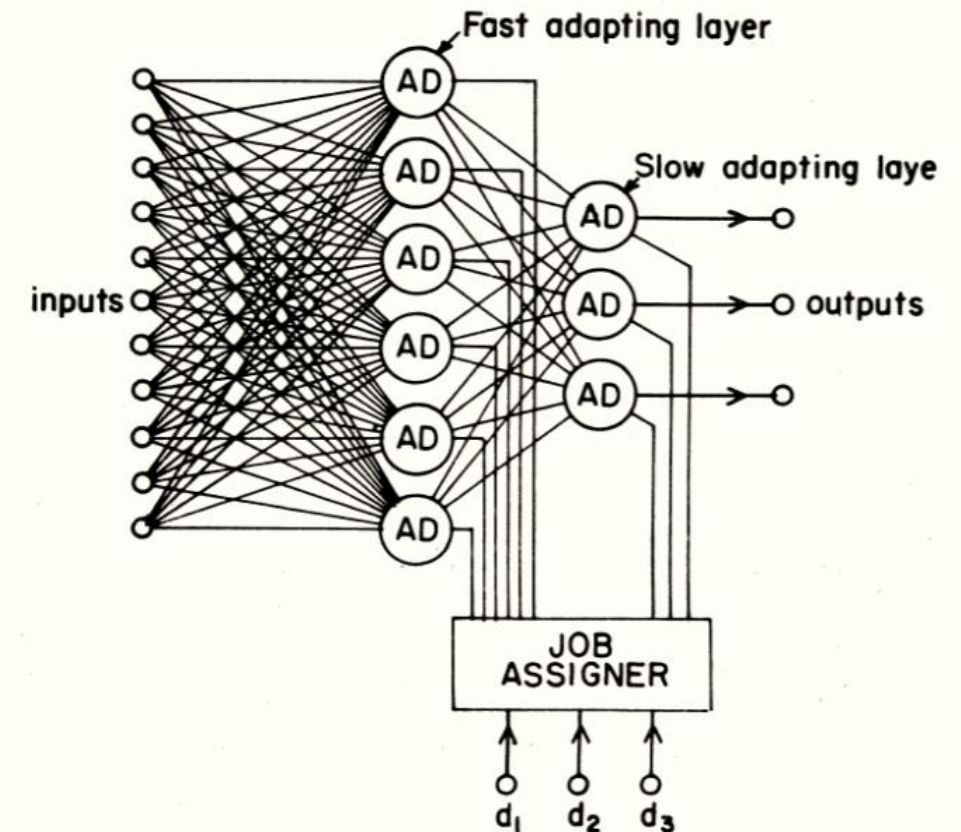


Figure 11. A Two-Layer Network of Adaptive ADALINES.

Widrow, Bernard. 1962. "Generalization and Information Storage in Networks of Adaline Neurons." *Self-Organizing Systems*, 435–61.

Versions of Gradient/Steepest Descent

Stochastic Gradient Descent

$t = 0$

$\mathbf{w}$ initial weights

for t in epochs **do**

$\mathcal{D} \leftarrow \text{shuffle}(\mathcal{D})$

for $\mathbf{x}_j \in \mathcal{D}$ **do** // for each sample

$\nabla \mathbf{w}_j = \partial E(g_{\mathbf{w}_t}(\mathbf{x}_j)) / (\partial \mathbf{w}_t)$ // gradient of
error *with respect to* weight $\mathbf{w}_j$

$$\mathbf{w}_j = \mathbf{w}_{j-1} - \eta \nabla \mathbf{w}_j \mathbf{x}_j$$

$t = t + 1$

Batch Gradient Descent

$t = 0$

$\mathbf{w}$ initial weights

for t in epochs **do**

$\mathcal{D} \leftarrow \text{shuffle}(\mathcal{D})$

for $\mathbf{x}_j \in \mathcal{D}$ **do** // for each sample

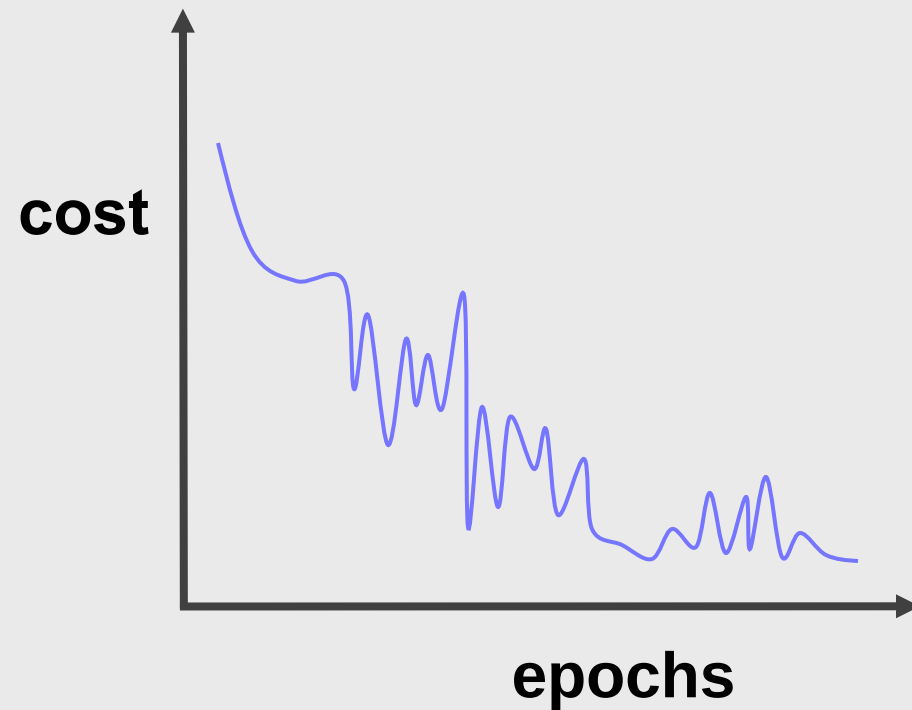
$\nabla \mathbf{w} = \nabla \mathbf{w} + \partial E(g_{\mathbf{w}}(\mathbf{x}_j)) / (\partial \mathbf{w}) \mathbf{x}_j$ // gradient of
error *with respect to* weight $\mathbf{w}_j$

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{\nabla \mathbf{w}}{|\mathcal{D}|}$$

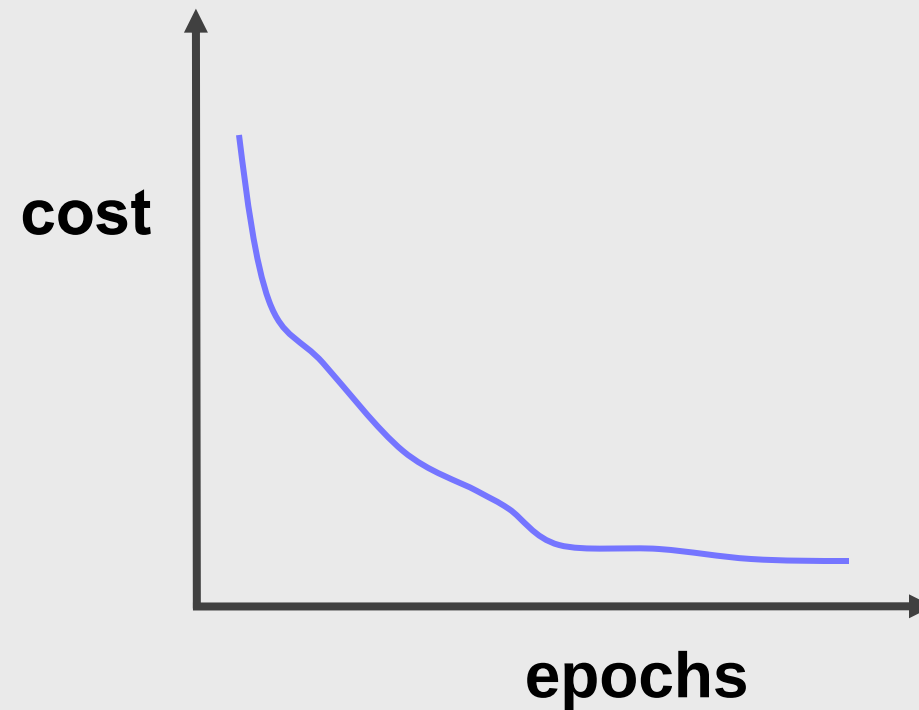
$t = t + 1$

Gradient Descent: Error/Loss Profile

Stochastic Gradient Descent



Batch Gradient Descent



First learning curve in 1960s by Widrow and Hoff

10:24
AM

Adaptive Linear Neuron (ADALINE) Algorithm

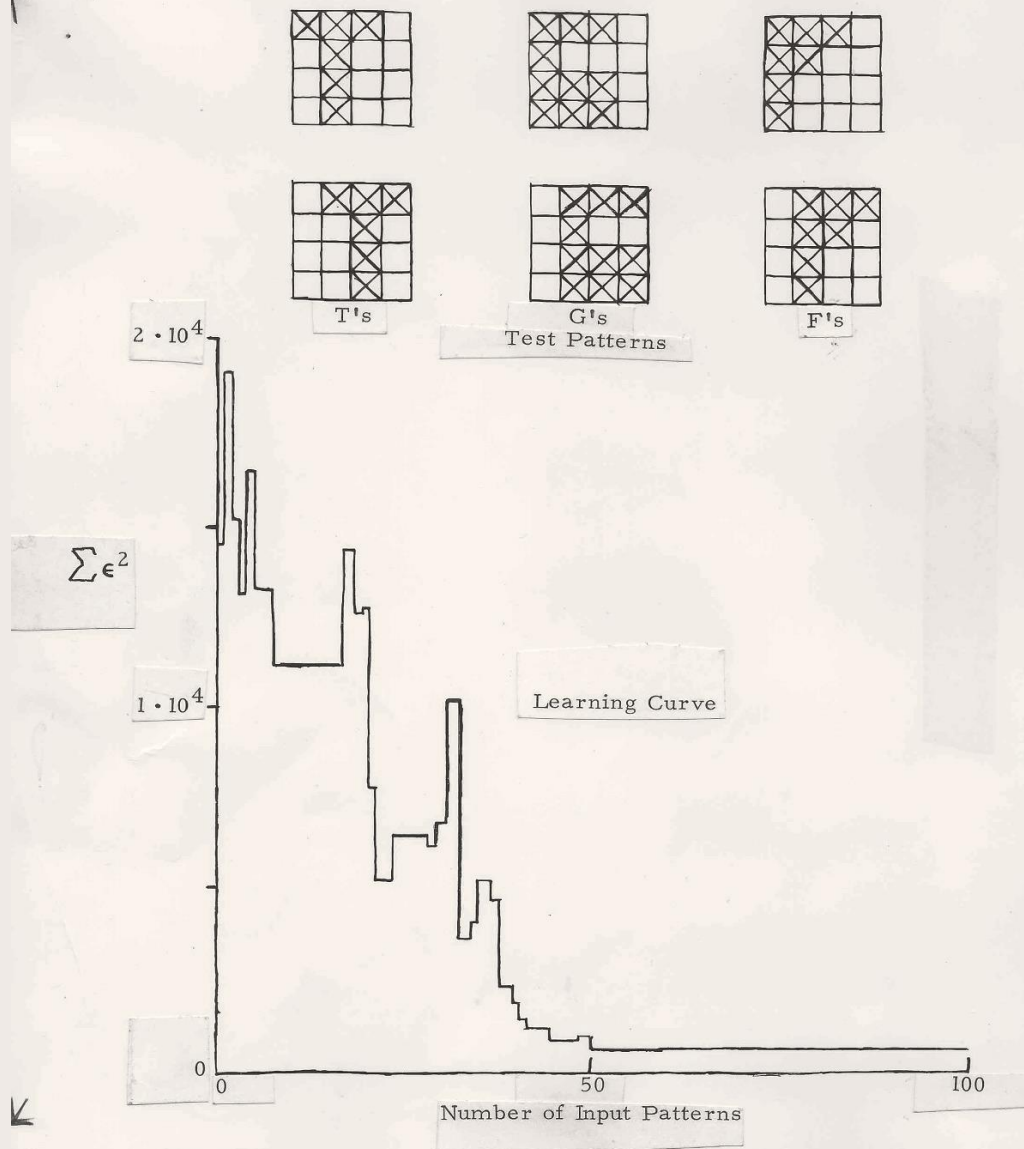


Fig. 6 Measurement of Rate of Adaption

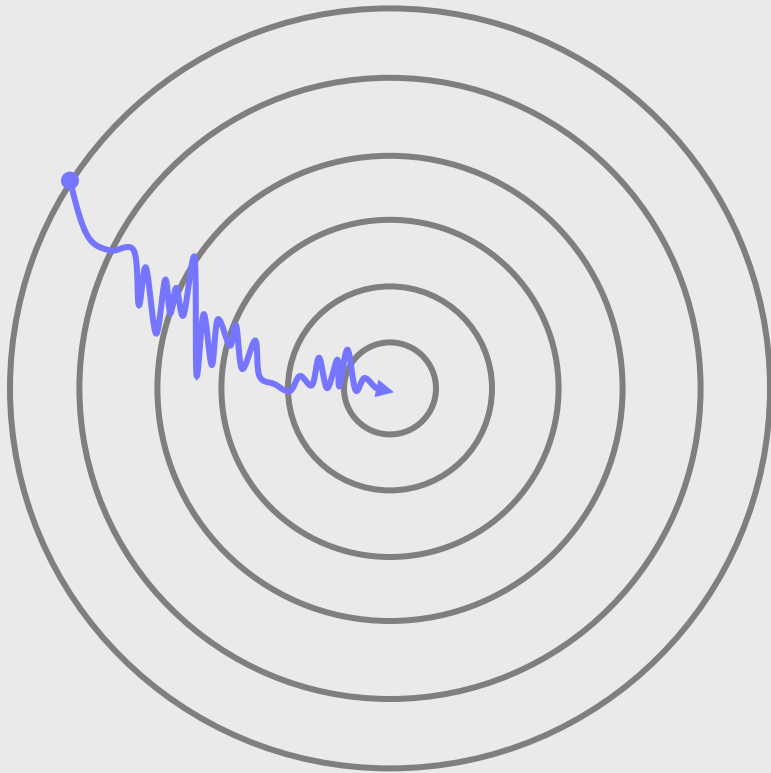
335

Image source: "Ancient History." In *Cybernetics 2.0: A General Theory of Adaptivity and Homeostasis in the Brain and in the Body*, 277–307. Springer Series on Bio- and Neurosystems. Cham: Springer

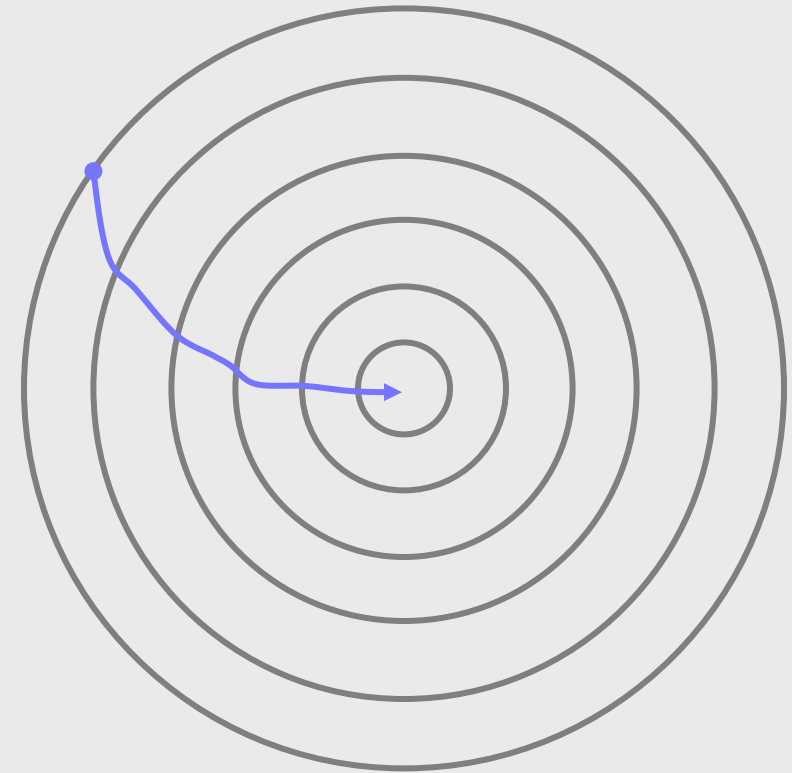
Gradient Descent: Error/Loss Profile

Geometric representation

Stochastic Gradient Descent



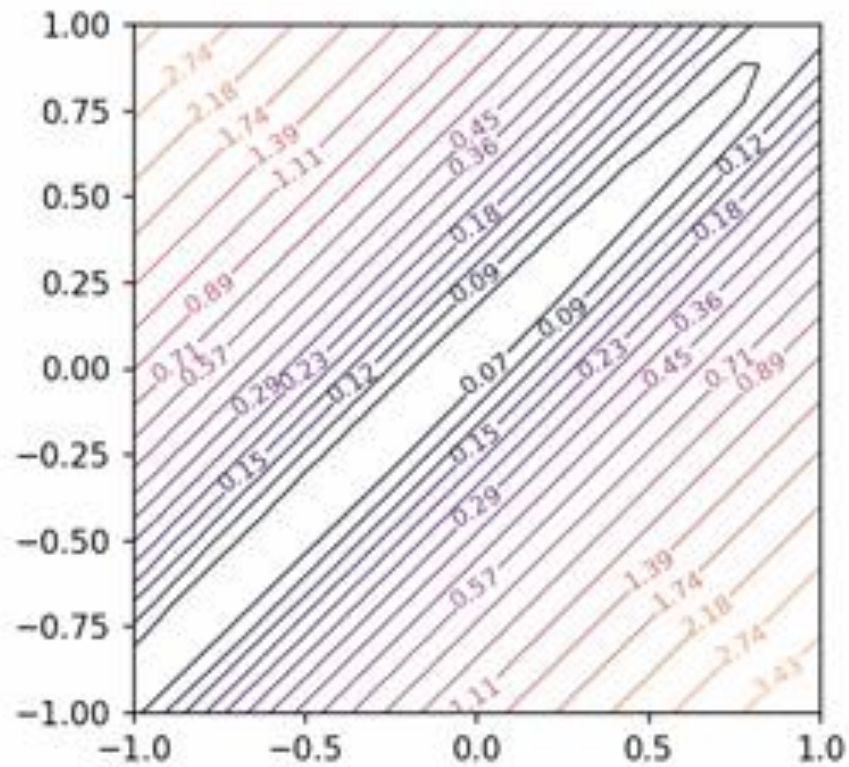
Batch Gradient Descent



Loss Profile / Loss Landscape

Visualizing the Loss Landscape of Neural Nets

Single neuron case



Multiple dimension case

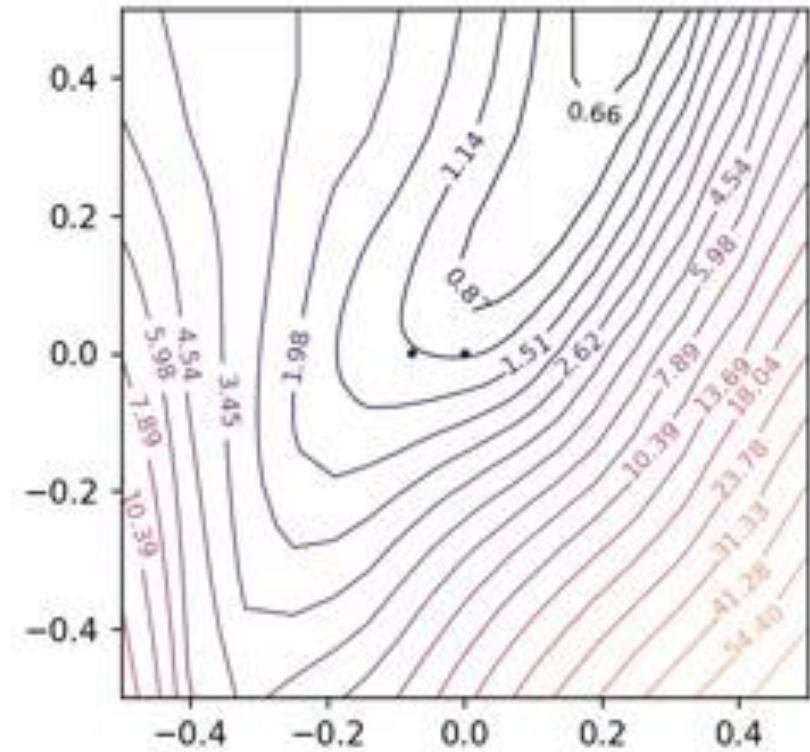
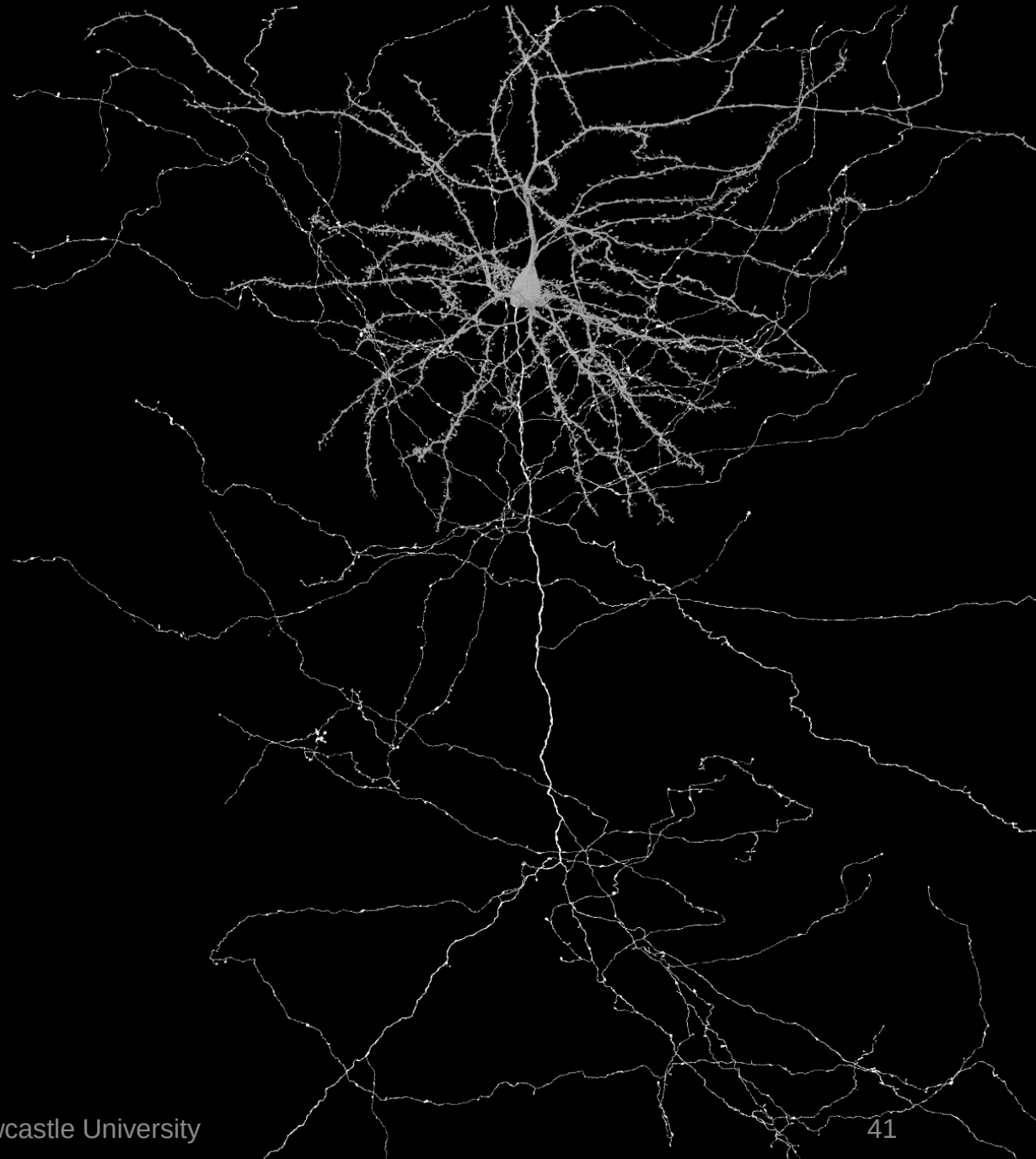


Image source: <https://mathformachines.com/posts/visualizing-the-loss-landscape/>

Part 3

Neural Network Architectures

Beyond single neuron

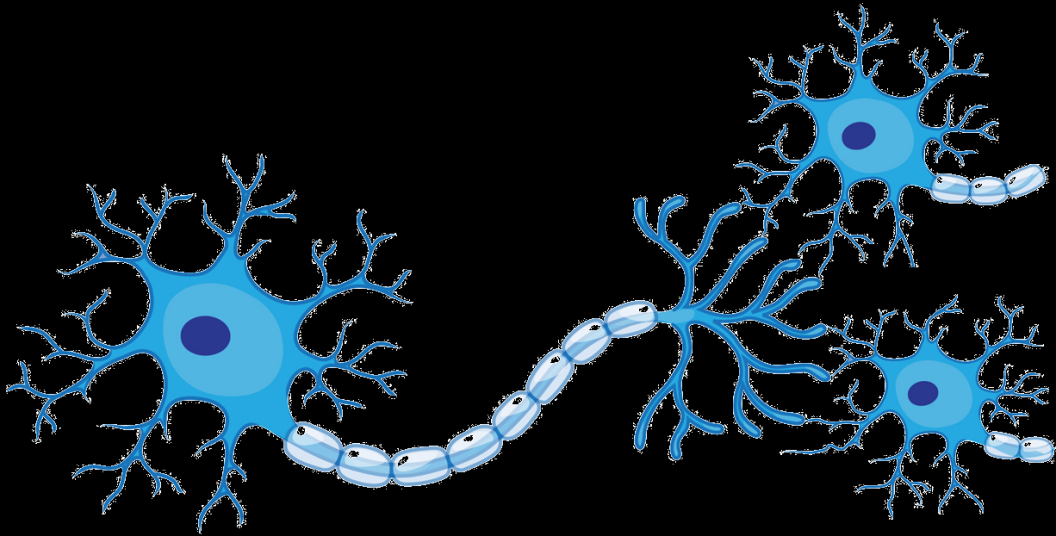


About Image:

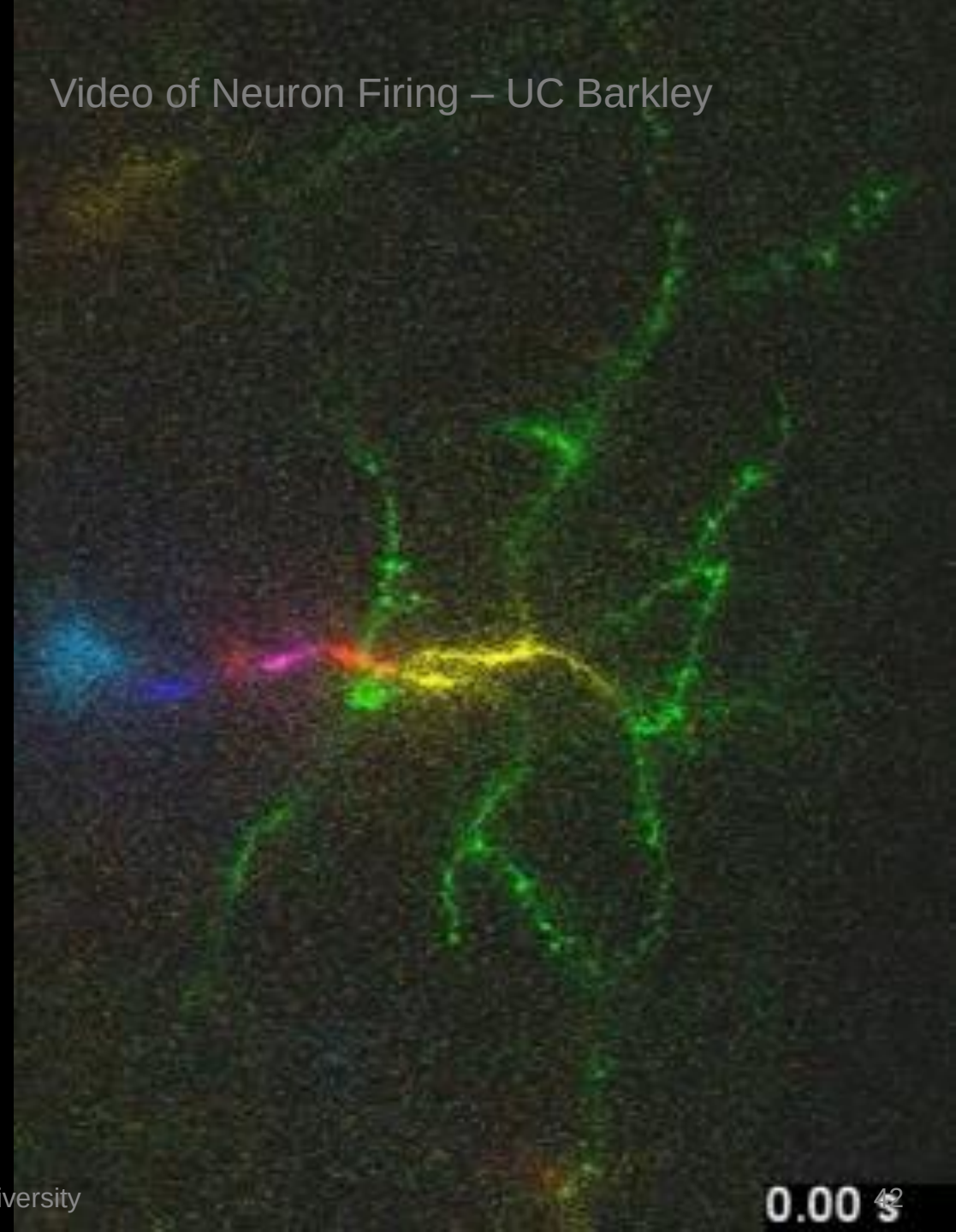
Single layer 2/3 pyramidal neuron from mouse visual cortex

Image Credit: Allen Institute

Biological Neural Networks

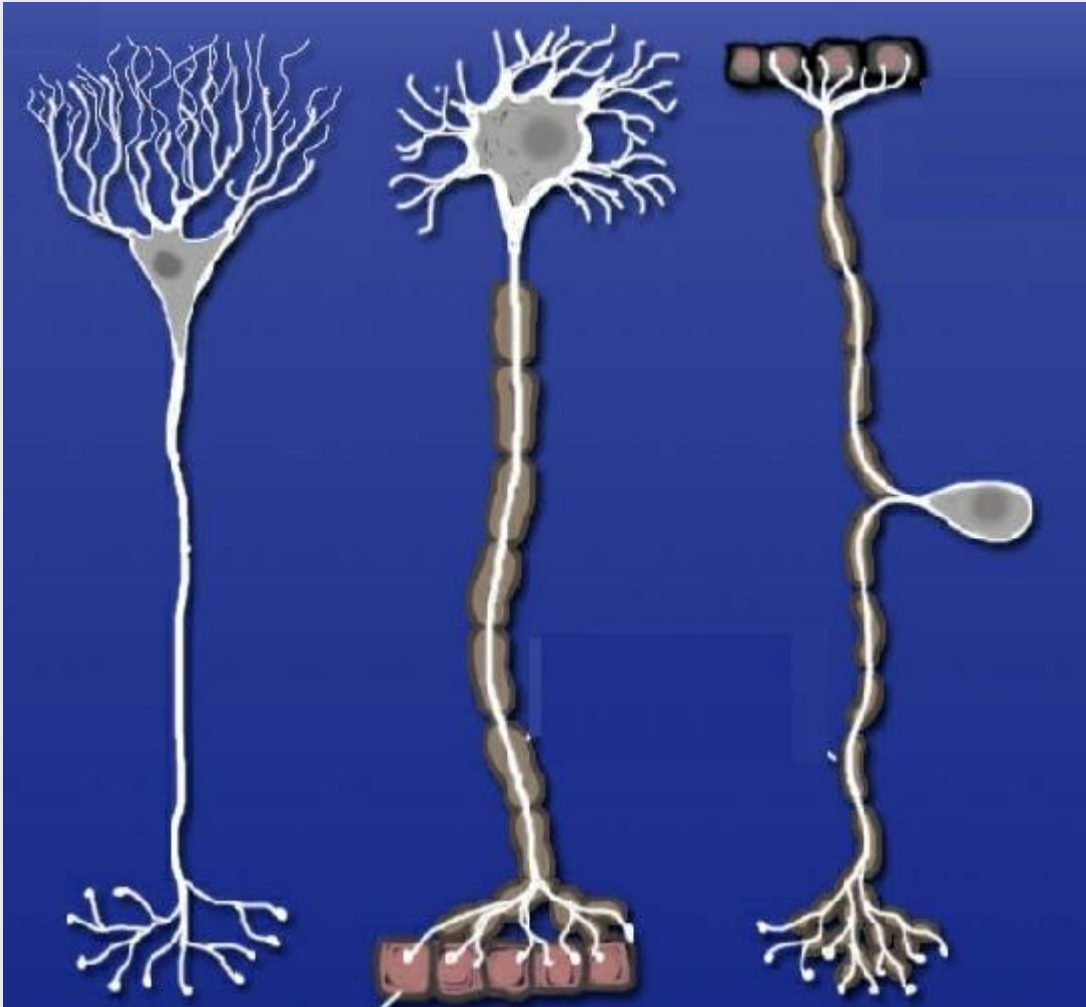


Video of Neuron Firing – UC Barkley



Neuron

10:24
AM



three types of Biological Neurons

Image source: Evolving Science

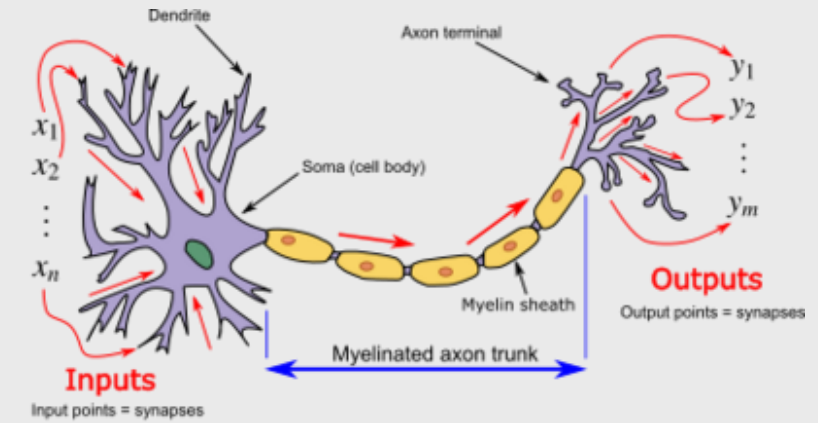
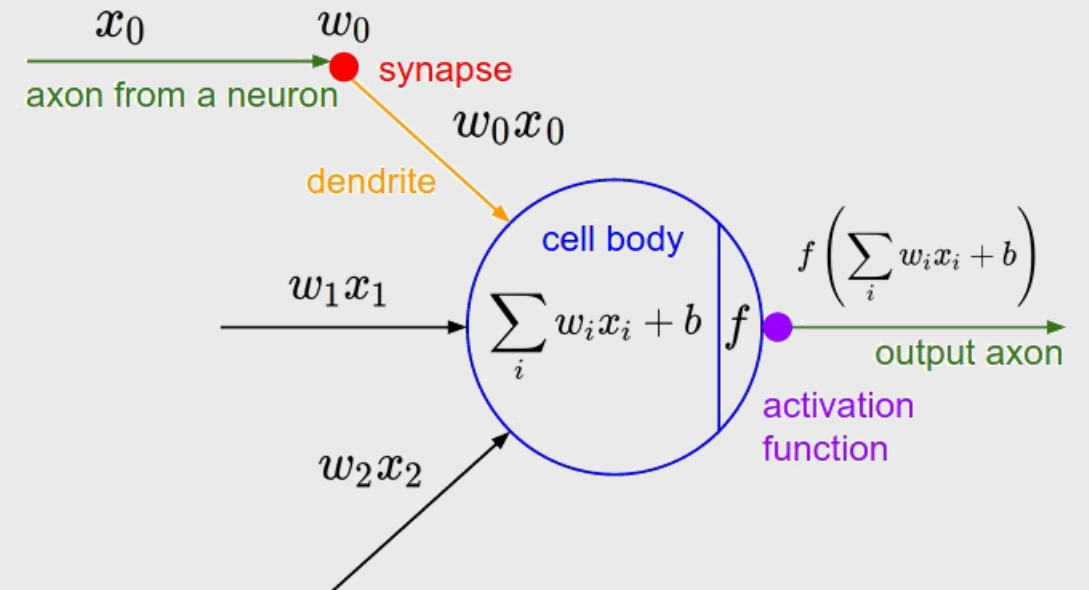


Image source: Wikipedia



A mathematical models of
McCulloch & Pitts's Neuron, 1943



THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

F. ROSENBLATT

Cornell Aeronautical Laboratory

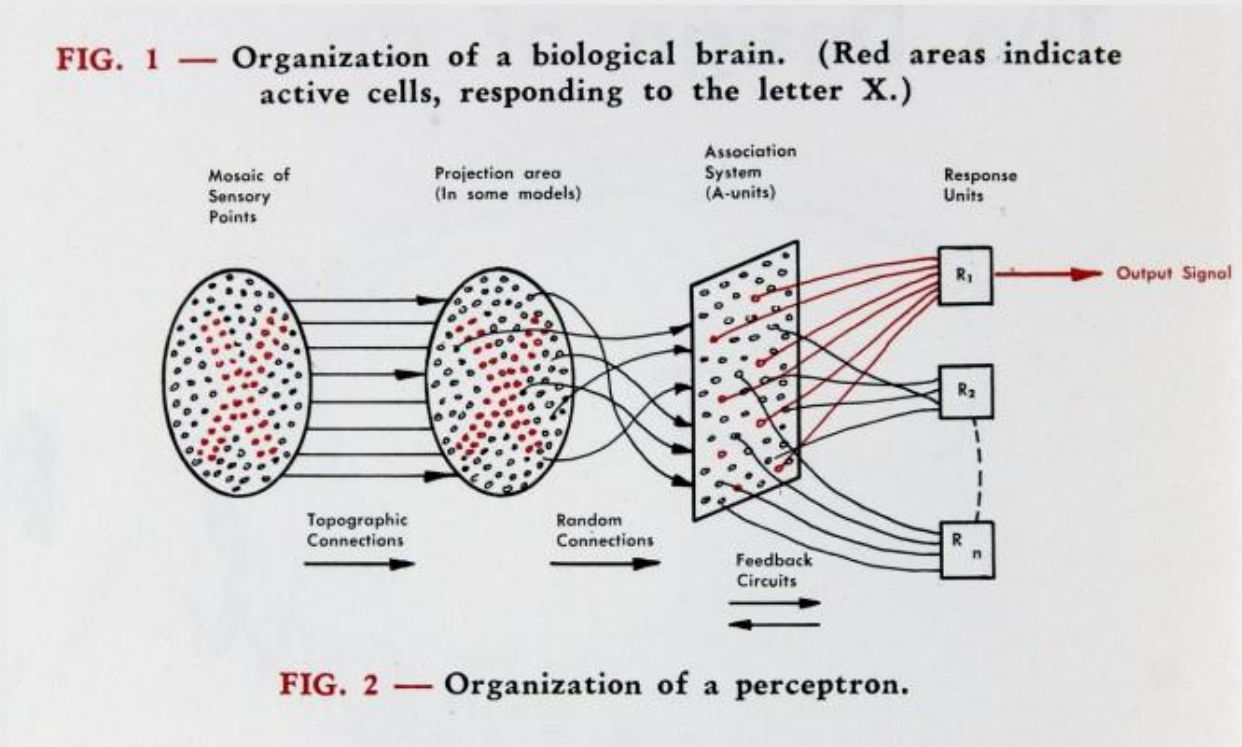
If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

1. How is information about the physical world sensed, or detected, by the biological system?
2. In what form is information stored, or remembered?
3. How does information contained in storage, or in memory, influence recognition and behavior?

Dr Varun Ojha, Newcastle University

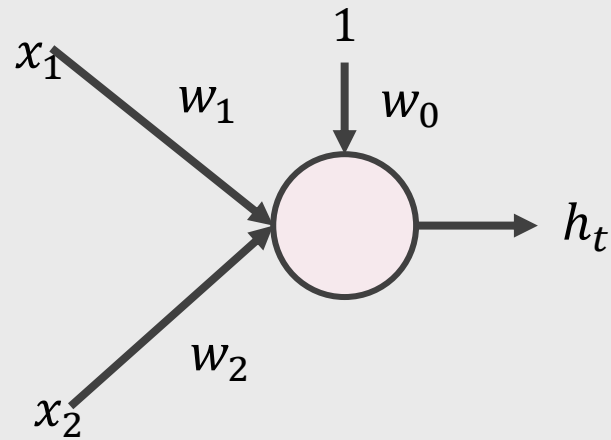
and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an organism remembers by reconstructing the original sensory patterns from the "memory traces" which they have left, much as we might develop a photographic negative, or translate the pattern of electrical charges in the "memory" of a digital computer. This hypothesis is appealing in its simplicity and ready intelligibility, and a large family of theoretical brain

The Rosenblatt's Perceptron

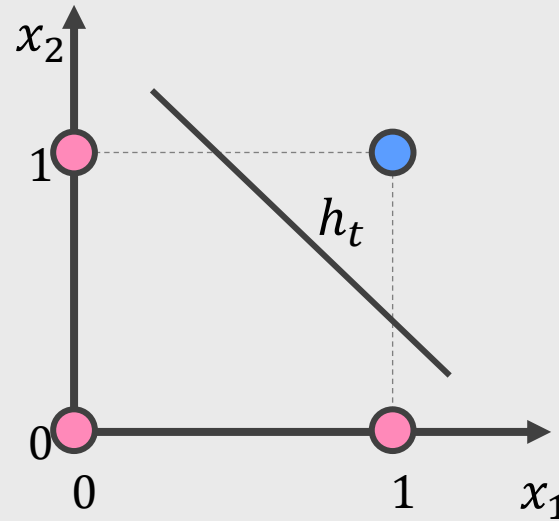


An image of the perceptron from Rosenblatt's "The Design of an Intelligent Automaton," Summer 1958.

Limitations of Perceptron's

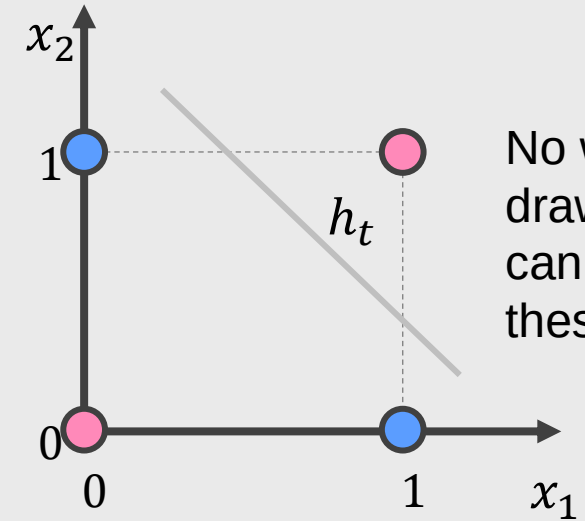


A perceptron



AND Problem

	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	0
3:	1	0	0
4:	1	1	1



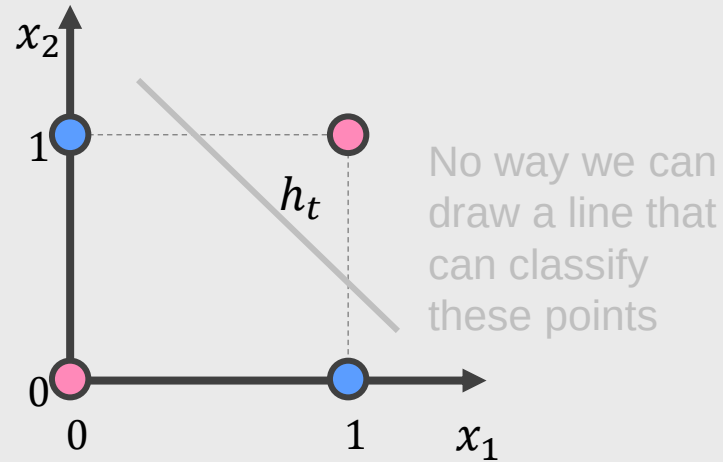
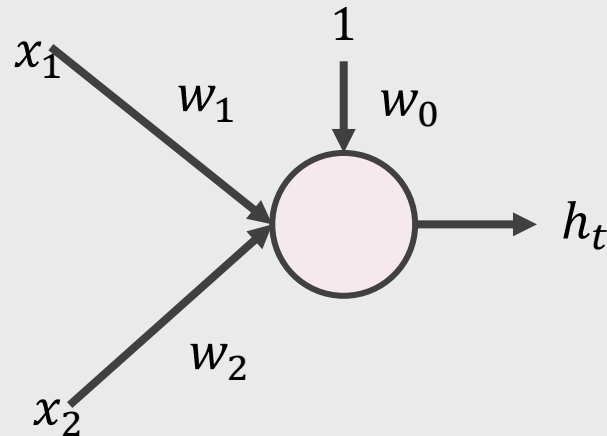
No way we can draw a line that can classify these points

XOR Problem

	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	1
3:	1	0	1
4:	1	1	0

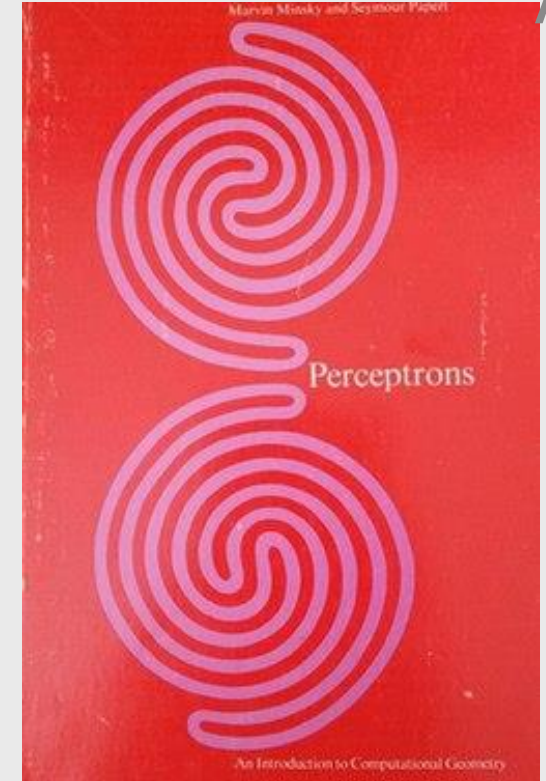
1969's Perceptron

“perceptrons could only learn ‘linearly separable’ patterns” And that it can not solve XOR problem that is non-linearly separable



XOR Problem

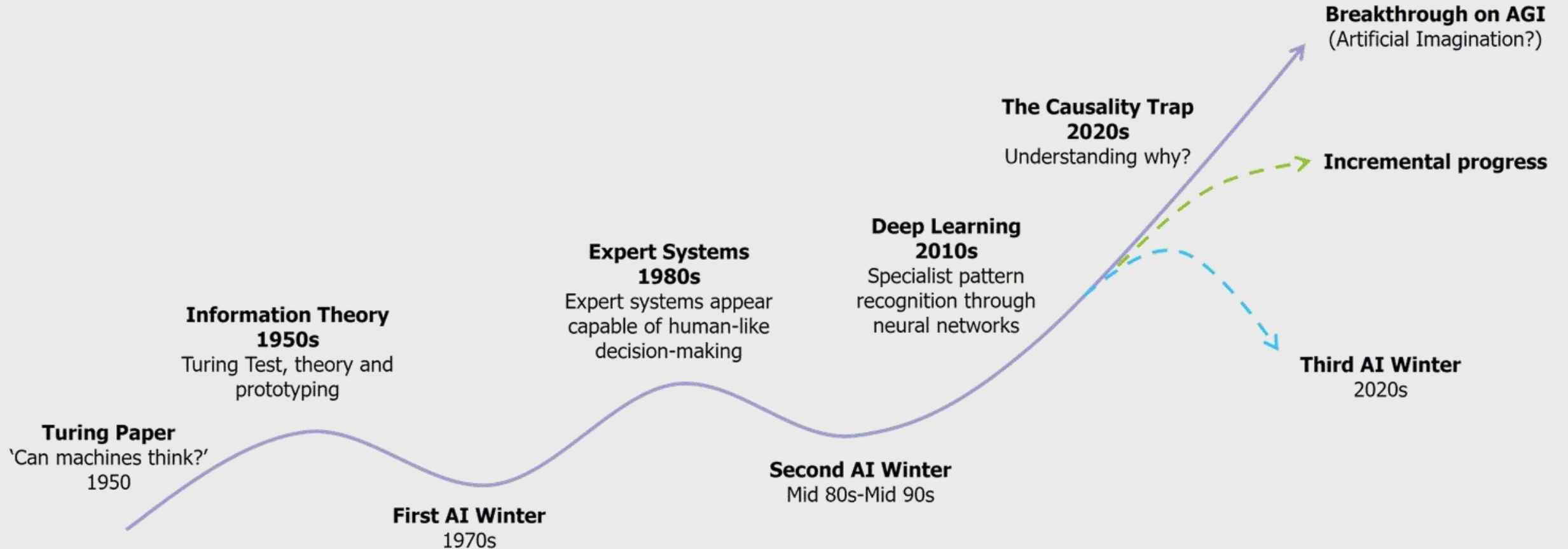
	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	1
3:	1	0	1
4:	1	1	0



Marvin Minsky and Seymour Papert

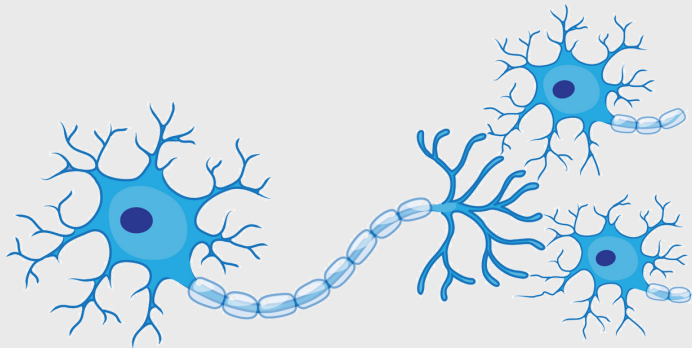
The AI Winter

Image Source: <https://transformainsights.com/ai-machine-learning>

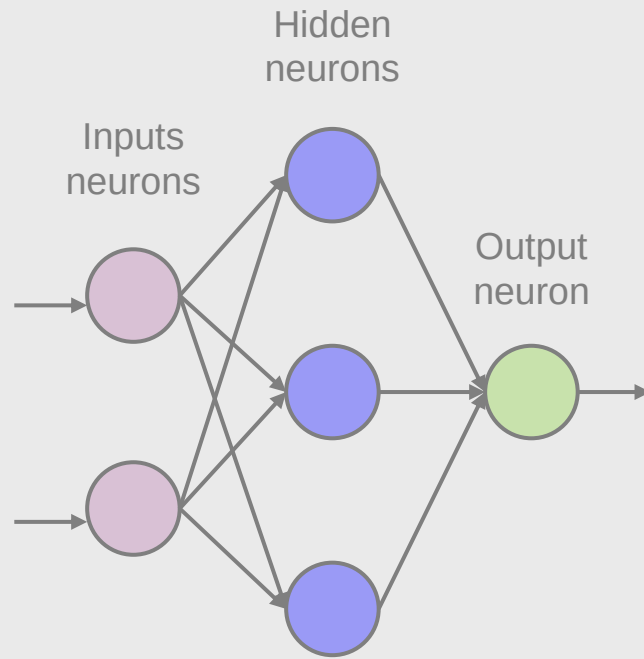


Multilayer Perceptron (Neural Networks)

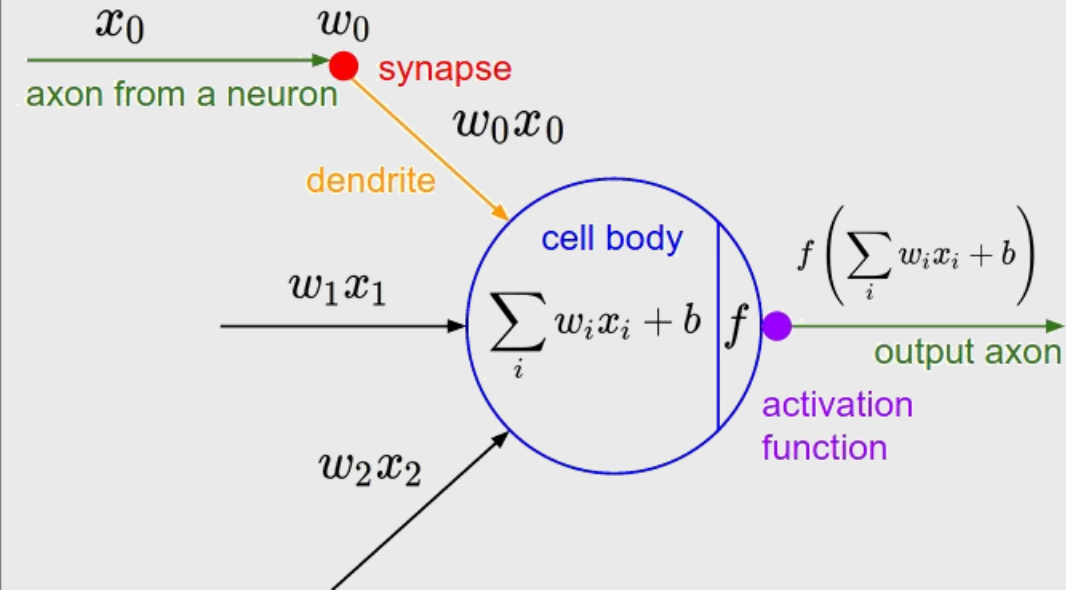
Solution to Single Layer perceptron (SLPs) problem is Multilayer Perceptron (MLP)



Biological networks of neurons in human brains



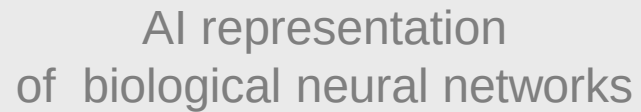
AI representation of biological neural networks



A mathematical models of McCulloch & Pitts's Neuron, 1943



2

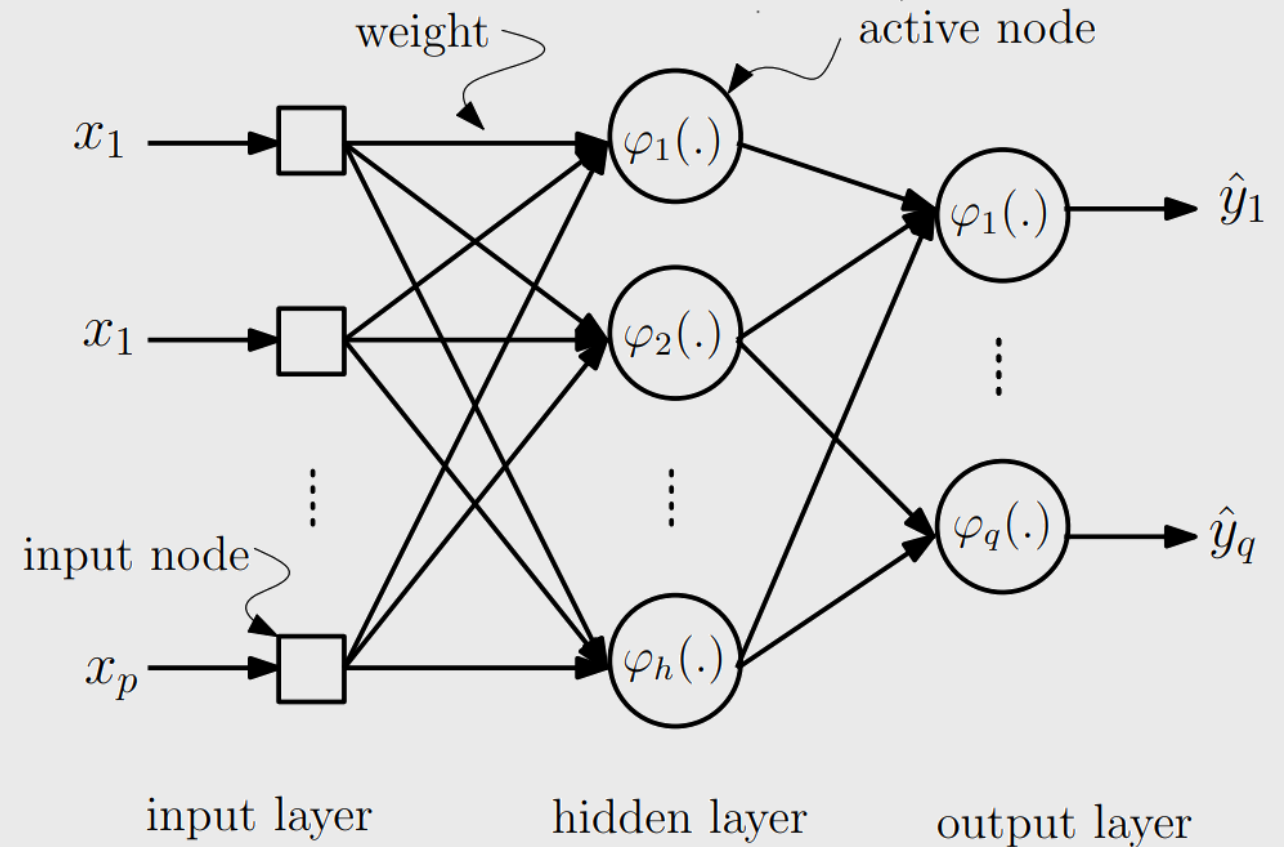


50

Neural Networks

NN components:

- Inputs
- Weights
- Architecture
- Activation functions
- Learning algorithms



NEURAL NETWORK

Architecture

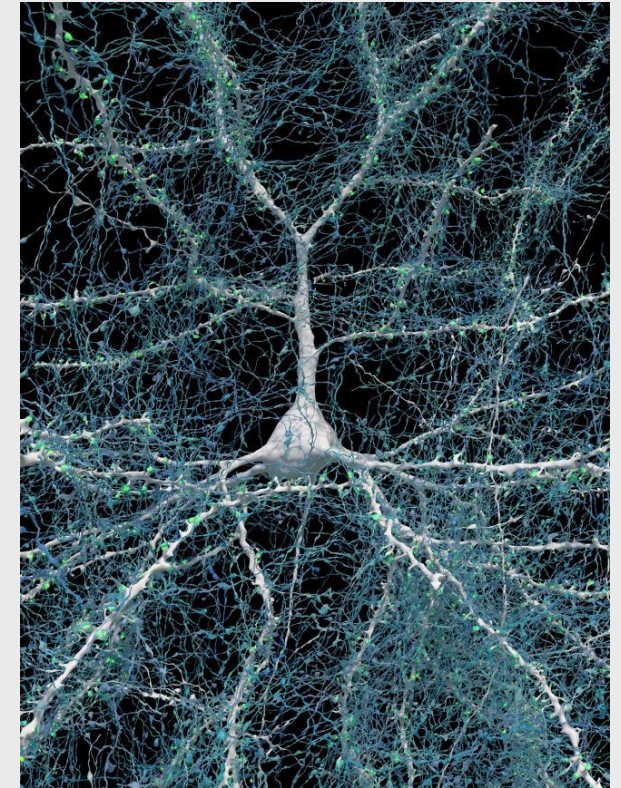
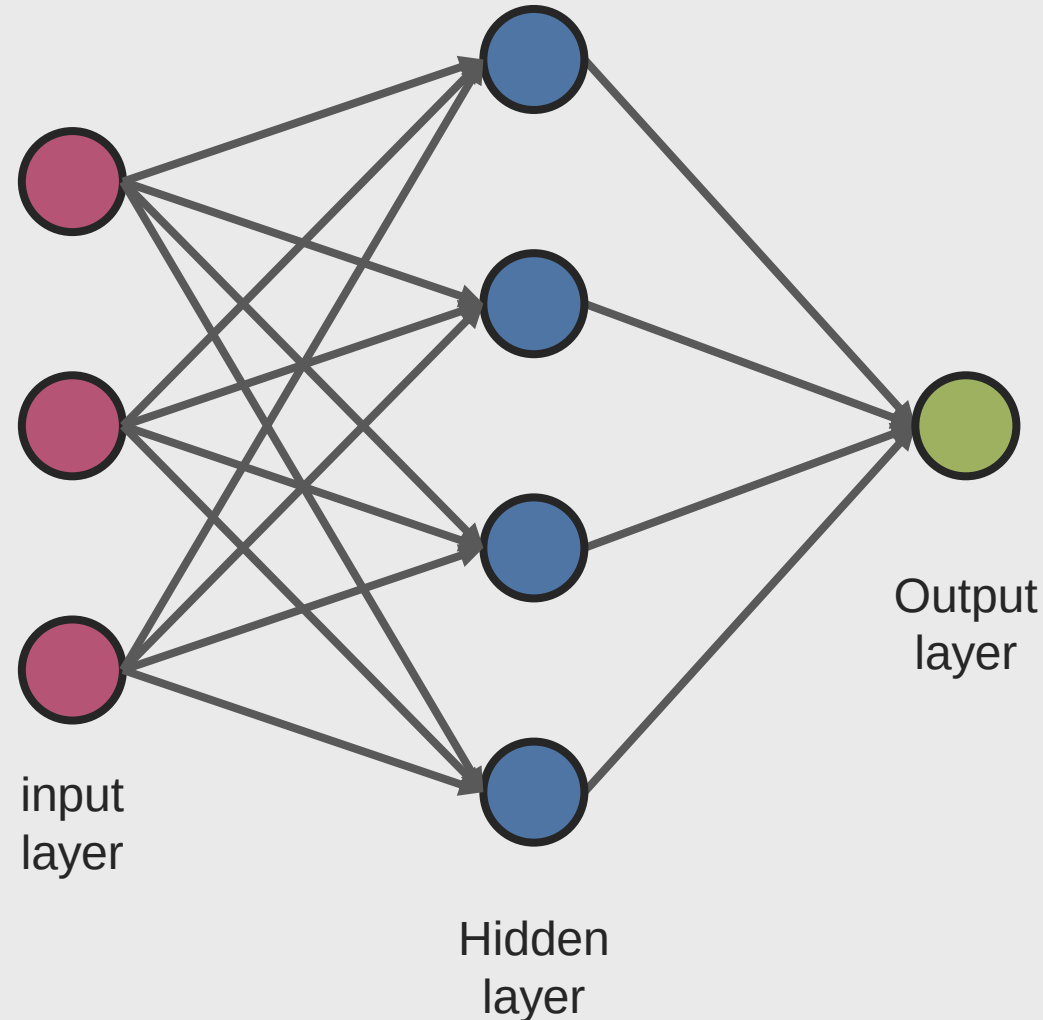
Number of neurons in the brain

The number of neurons in the brain is about 10^{11} (100 billion).

Number of synapses in the brain

The number of synapses in the brain is known much less precisely, but is probably about 10^{14} - 10^{15} (100 – 1000 trillion) synapses in the brain

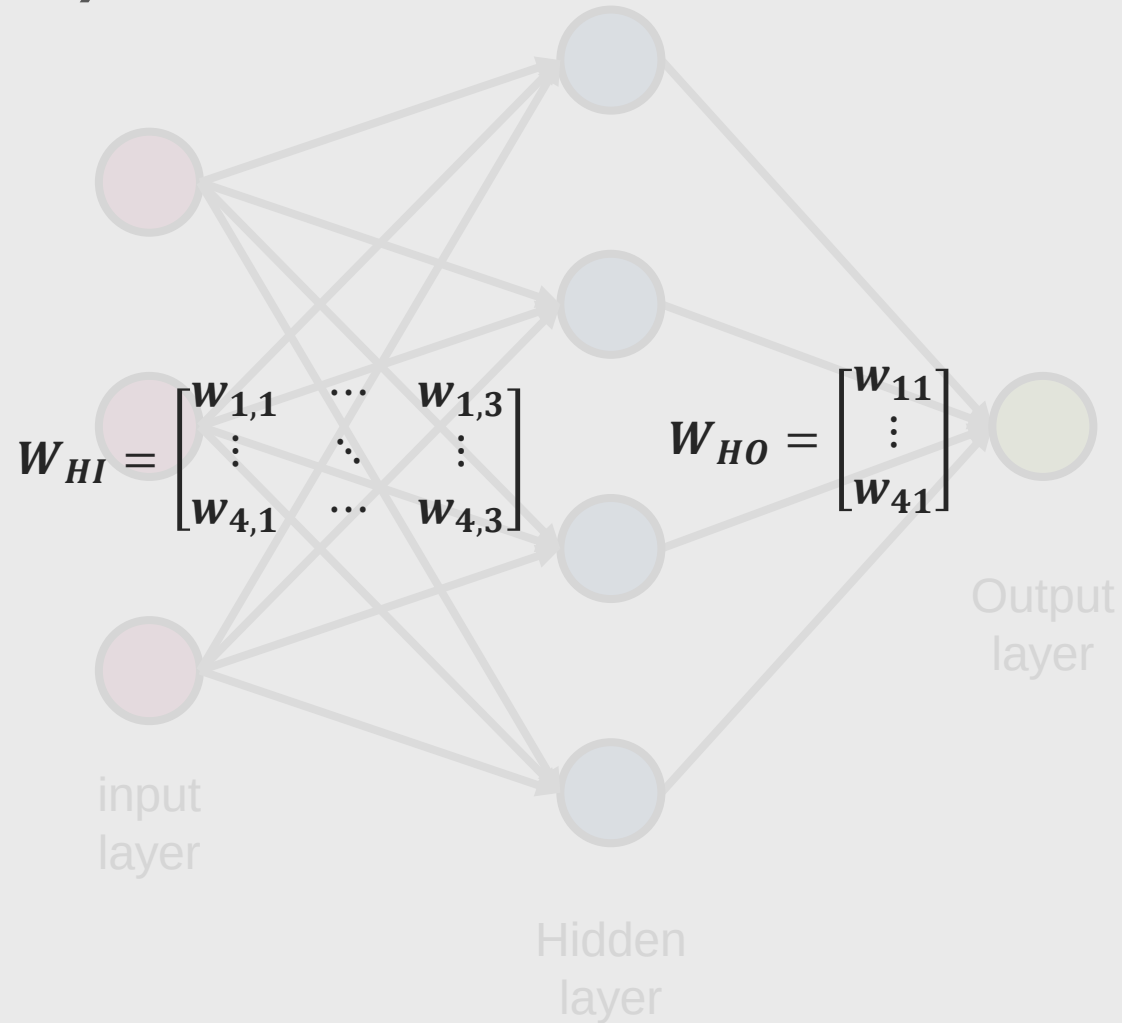
These are estimates and unverifiable

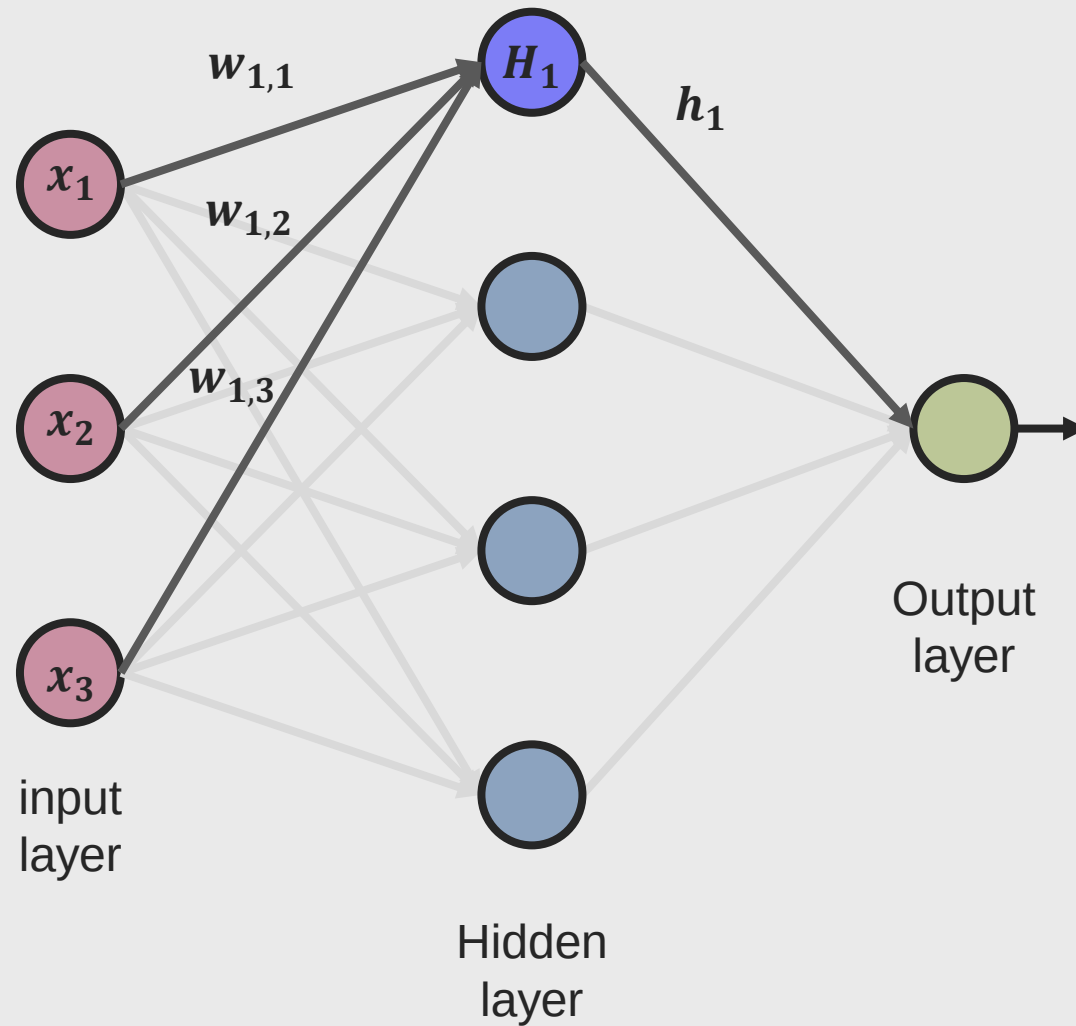


A single neuron with 5,600 nerve fibres, Google Research a7 Harvard University

NEURAL NETWORK

Weights (parameters)

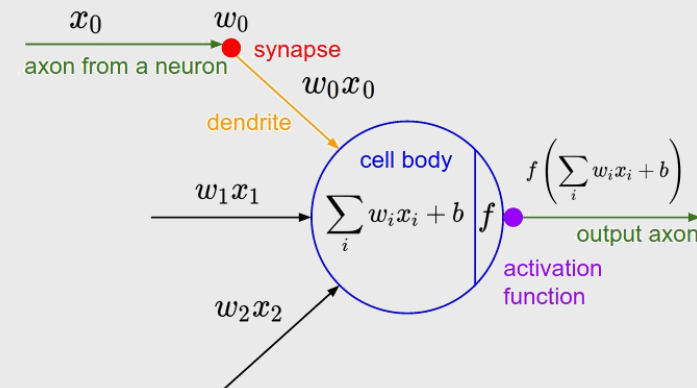




For n inputs, a hidden layer node's h_j output is expressed as:

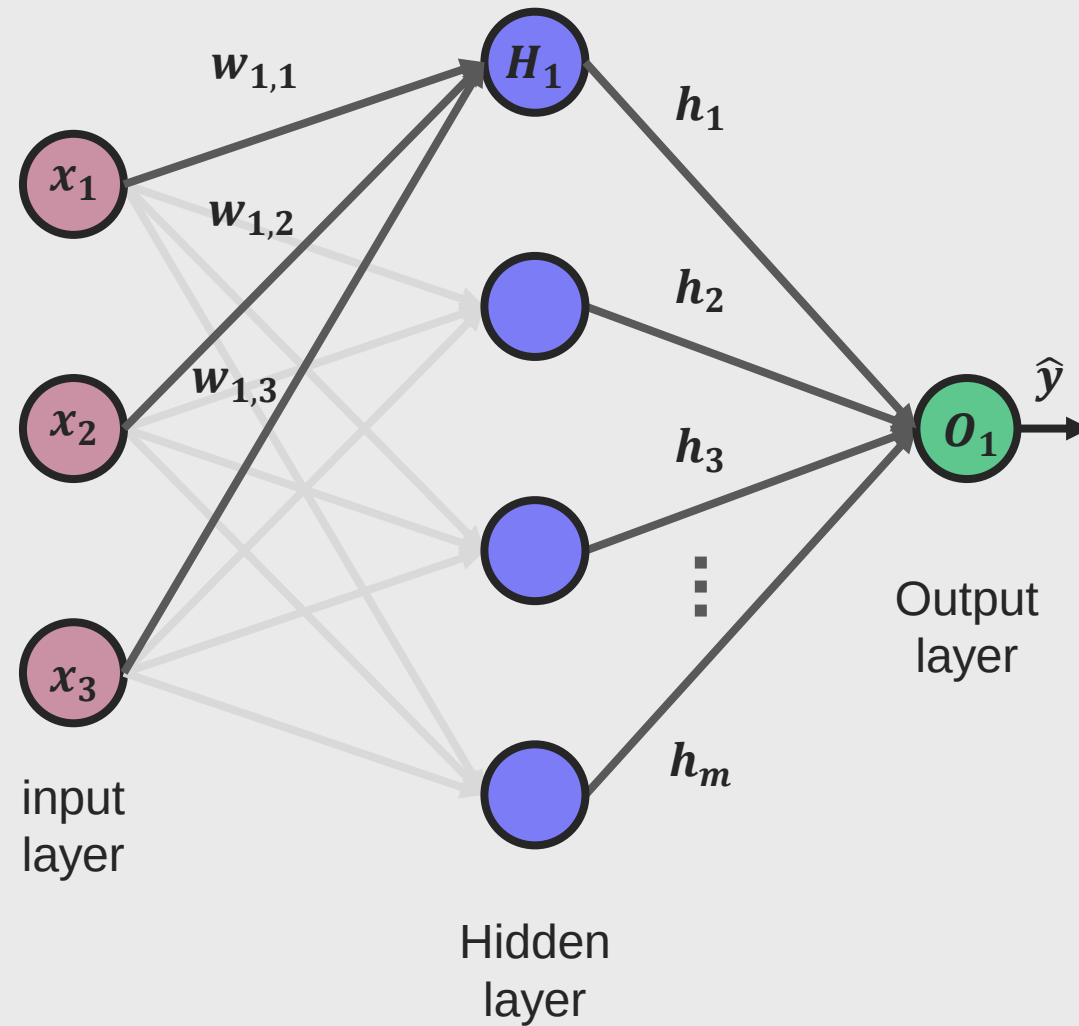
$$h_j = \varphi_h \left(\sum_{i=1}^n w_{ji} \cdot x_i \right)$$

Where φ_h is an activation function:



NEURAL NETWORK

Computation: Hidden layer



For n inputs, a hidden layer node's h_j output is expressed as:

$$h_j = \varphi_h \left(\sum_{i=1}^n w_{ji} \cdot x_i \right)$$

Where φ_h is an activation function:

For m hidden nodes and an output node, the output nodes output is expressed as:

$$\hat{y} = \varphi_o \left(\sum_{j=1}^m w_{jk} \cdot h_j \right)$$

NEURAL NETWORK

Computation: Output layer

Sigmoid activation

$$\hat{y} = \varphi(x) = \frac{1}{1 + e^{-x}}$$

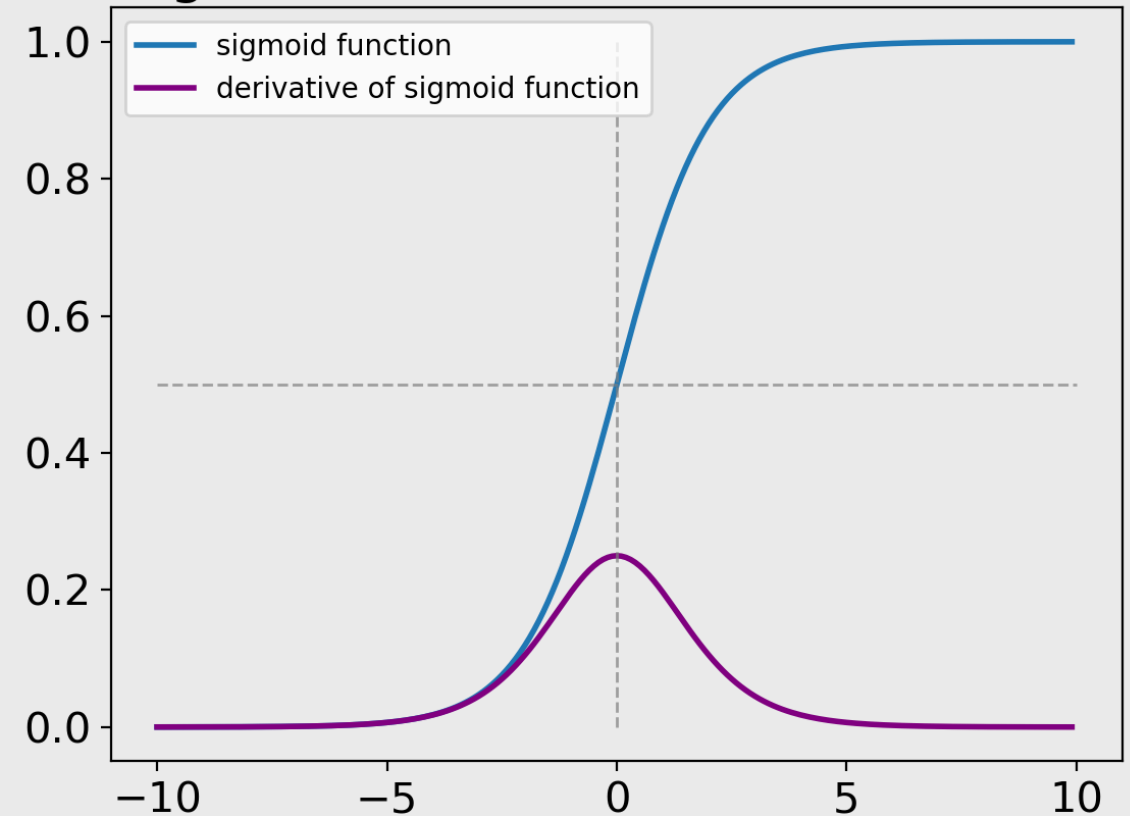
derivative

$$\varphi'(x) = \hat{y}(1 - \hat{y})$$

NEURAL NETWORK
Activation function

<https://www.derivative-calculator.net/>

sigmoid function and its derivative

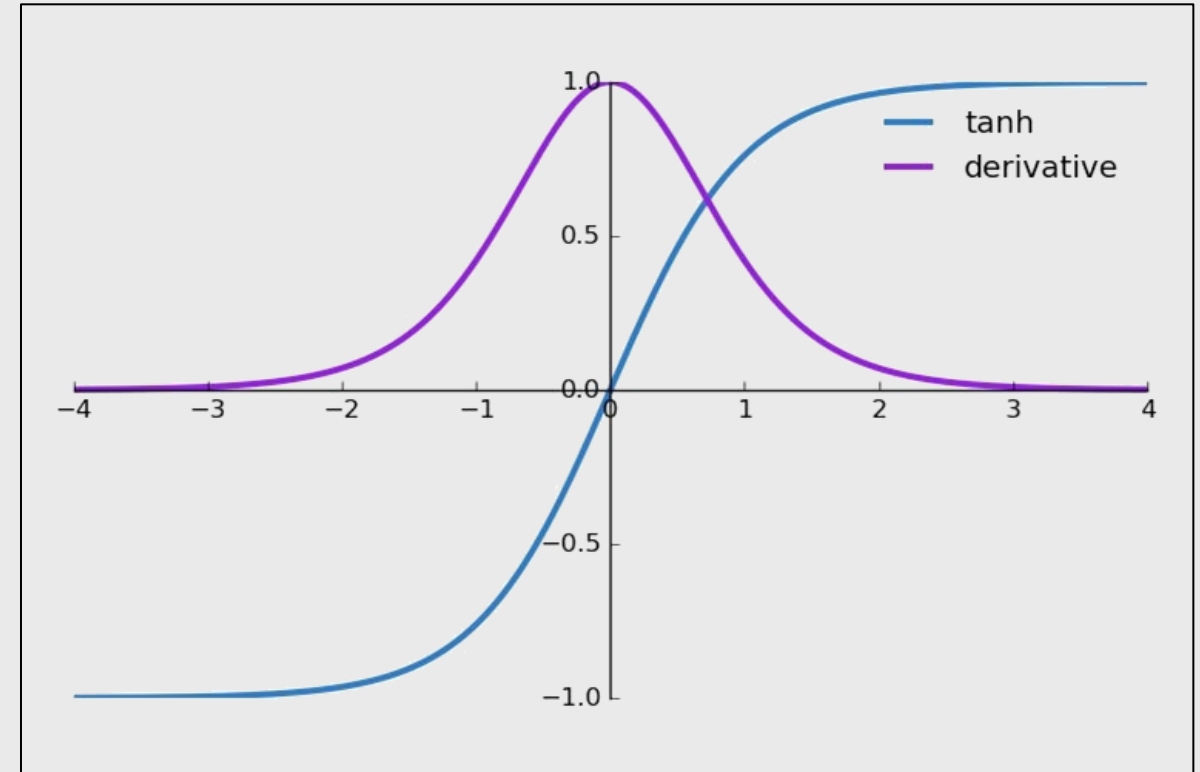


Tangent hyperbolic activation

$$\varphi(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

derivative

$$\varphi'(x) = 1 - \hat{y}^2$$



NEURAL NETWORK
Activation function

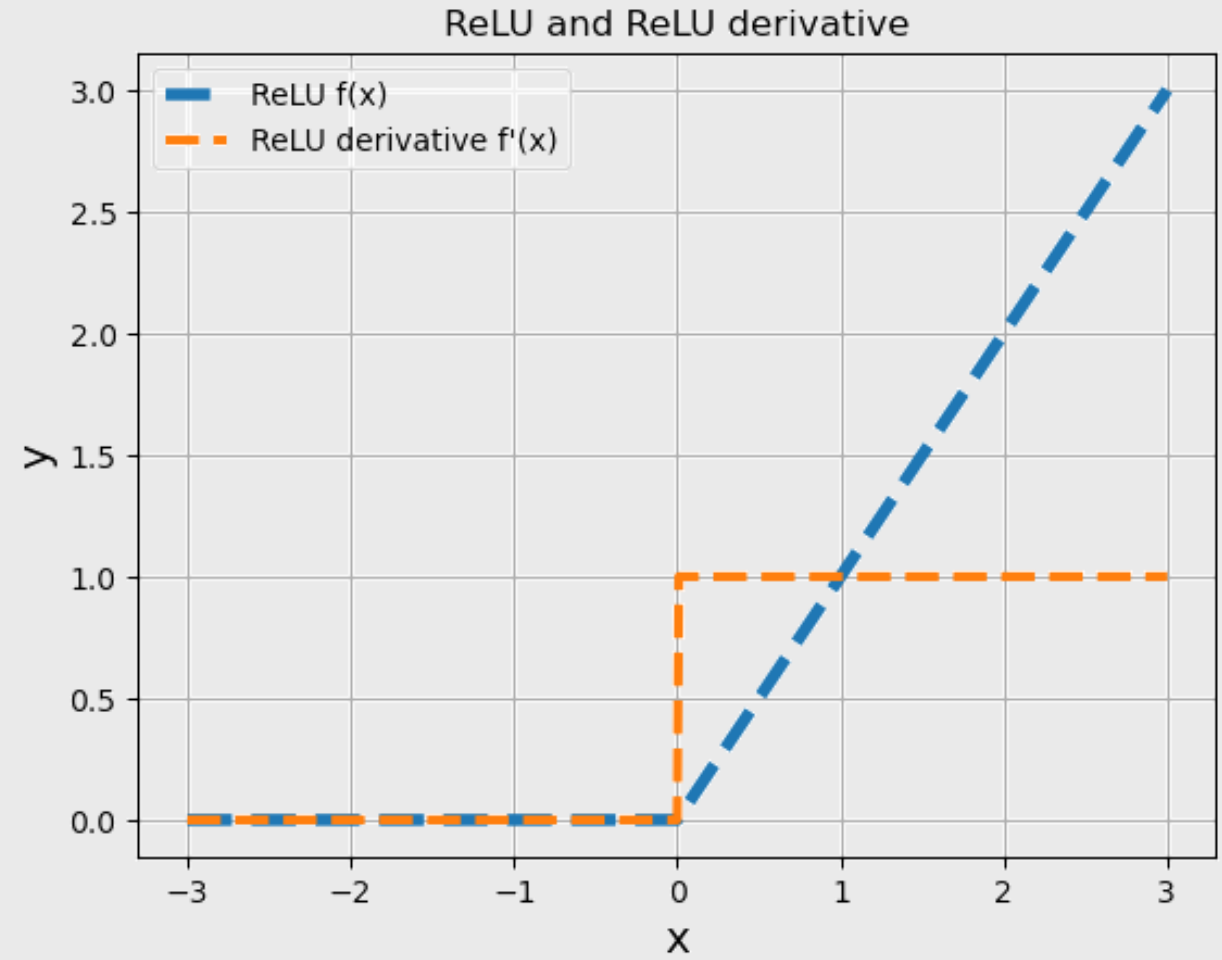
Rectified Linear Unit (ReLU)

$$\varphi(x) = \max(0, x)$$

derivative

$$\varphi'(x) = 1 \text{ if } x > 0 \text{ else } 0$$

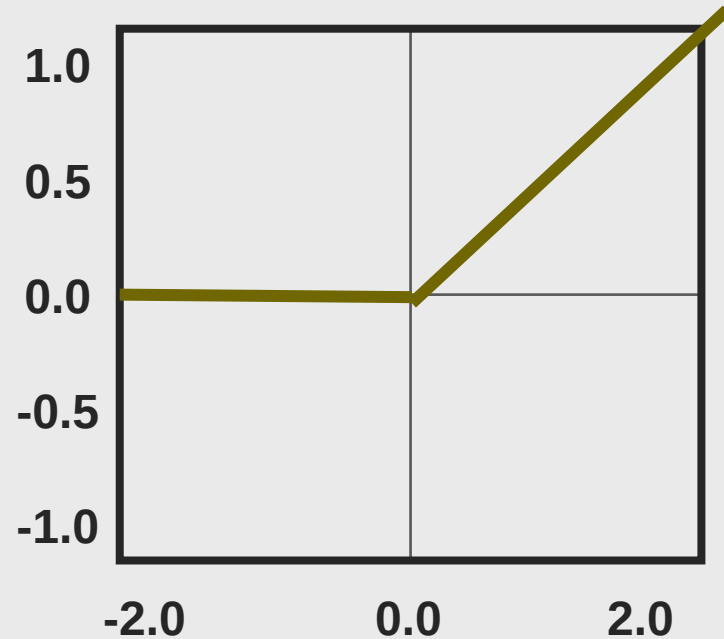
NEURAL NETWORK
Activation function



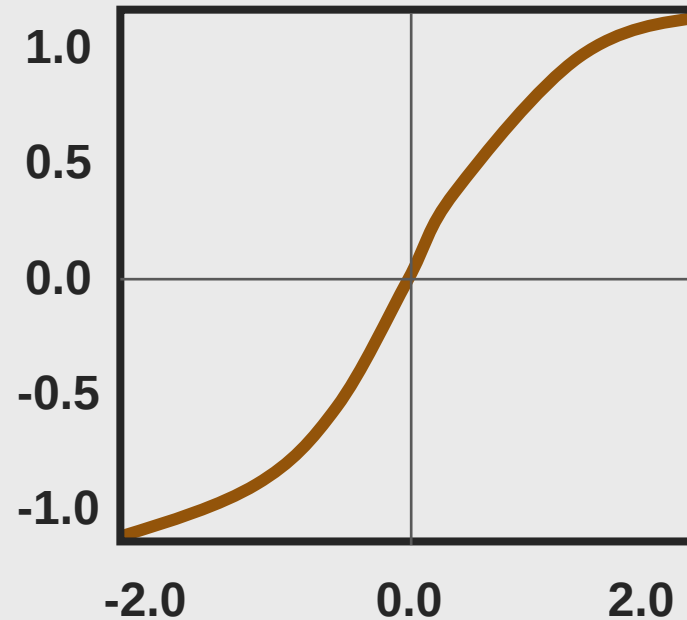
NEURAL NETWORK

Activation functions

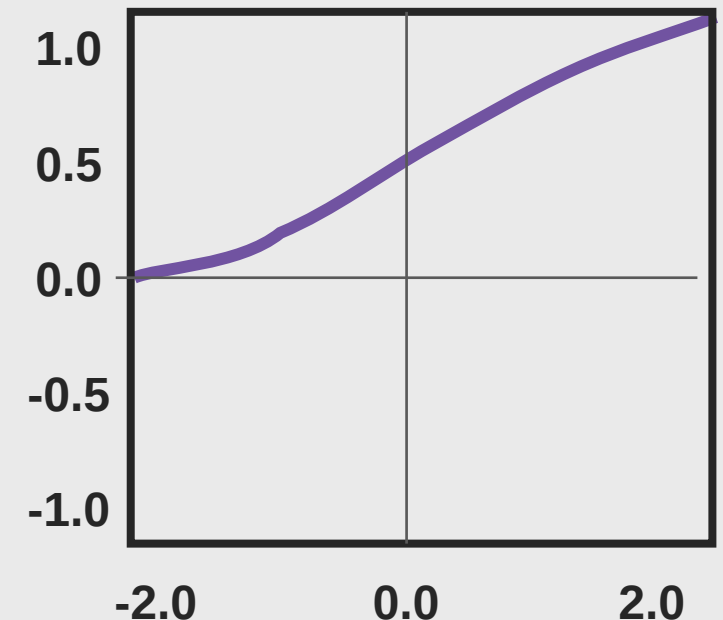
ReLU



Tanh



Sigmoid

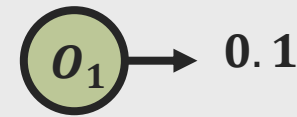


SoftMax Activation

$$\hat{y}_i = \varphi(x_i) = \frac{e^{x_i}}{\sum_j^k e^{x_j}} \text{ for } k \text{ units}$$

derivative

$$\varphi'(x_i) = \begin{cases} \hat{y}_i(1 - \hat{y}_i) & \text{if } i = j \\ -\hat{y}_i\hat{y}_j & \text{if } i \neq j \end{cases}$$



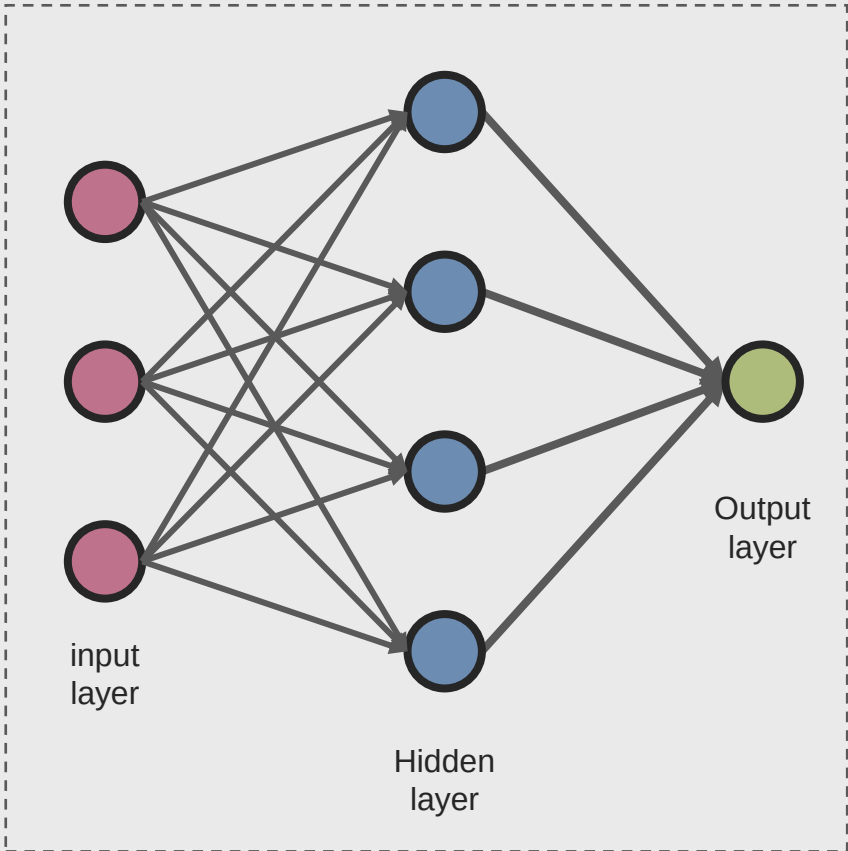
PROBABILITIES
DISTRIBUTION OF ALL
LABELS

NEURAL NETWORK

Activation function

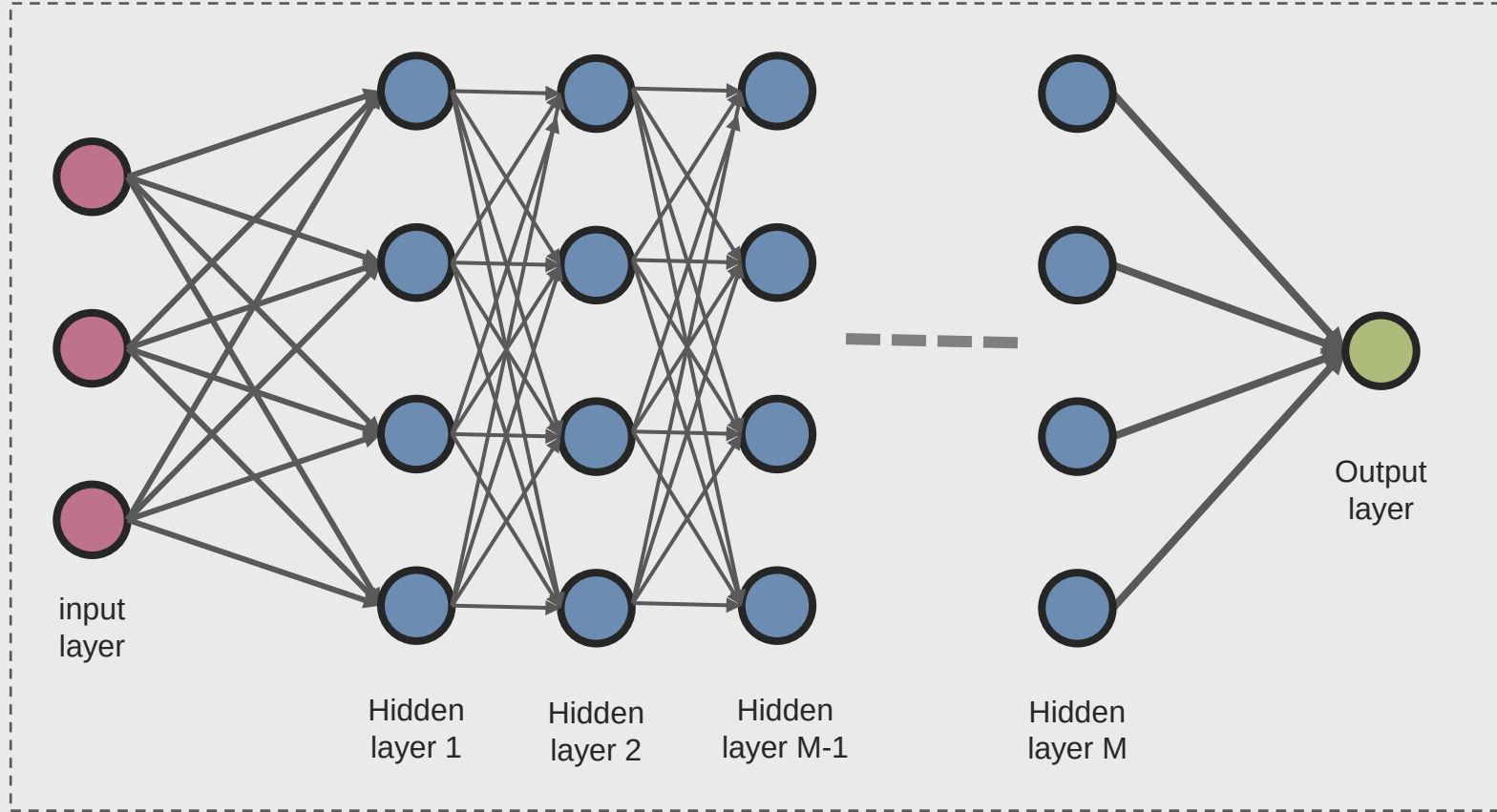
NEURAL NETWORK: Architecture

A regular neural network architecture



SHALLOW LEARNING

A deep neural network architecture



DEEP LEARNING

Deep Learning

Deep learning is an artificial intelligence function that imitates the workings of the human brain in processing data and creating patterns for use in decision making.

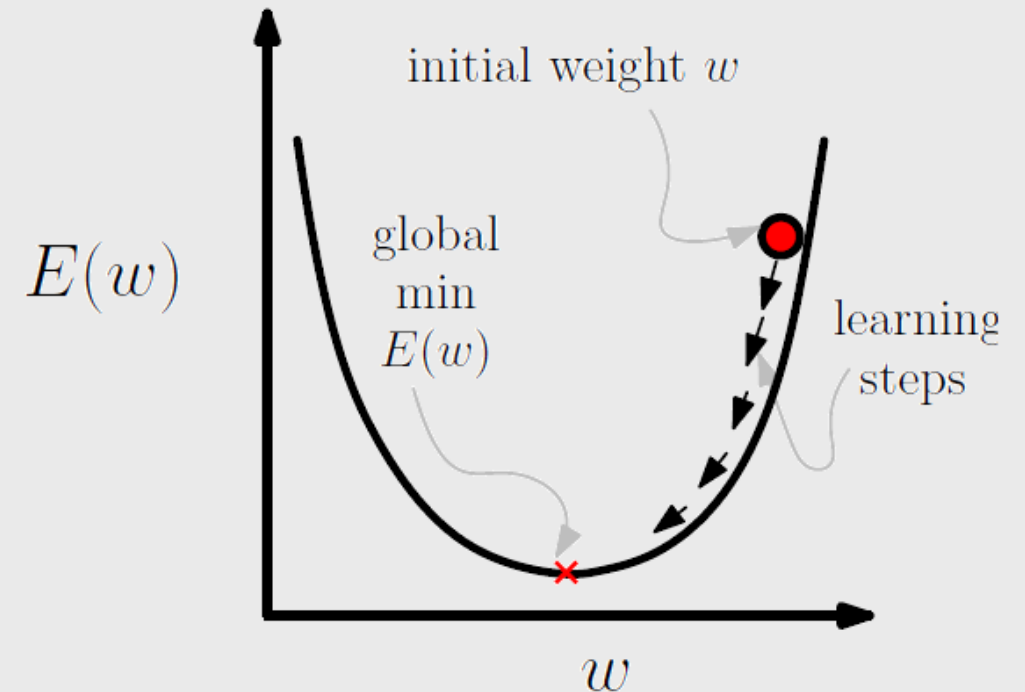
Deep learning is a **subset of machine learning in artificial intelligence** that has networks capable of learning supervised/unsupervised from data that is structured/unstructured or labelled/unlabelled.

Source: <https://www.investopedia.com/terms/d/deep-learning.asp>

(DEEP) NEURAL NETWORK Optimisation

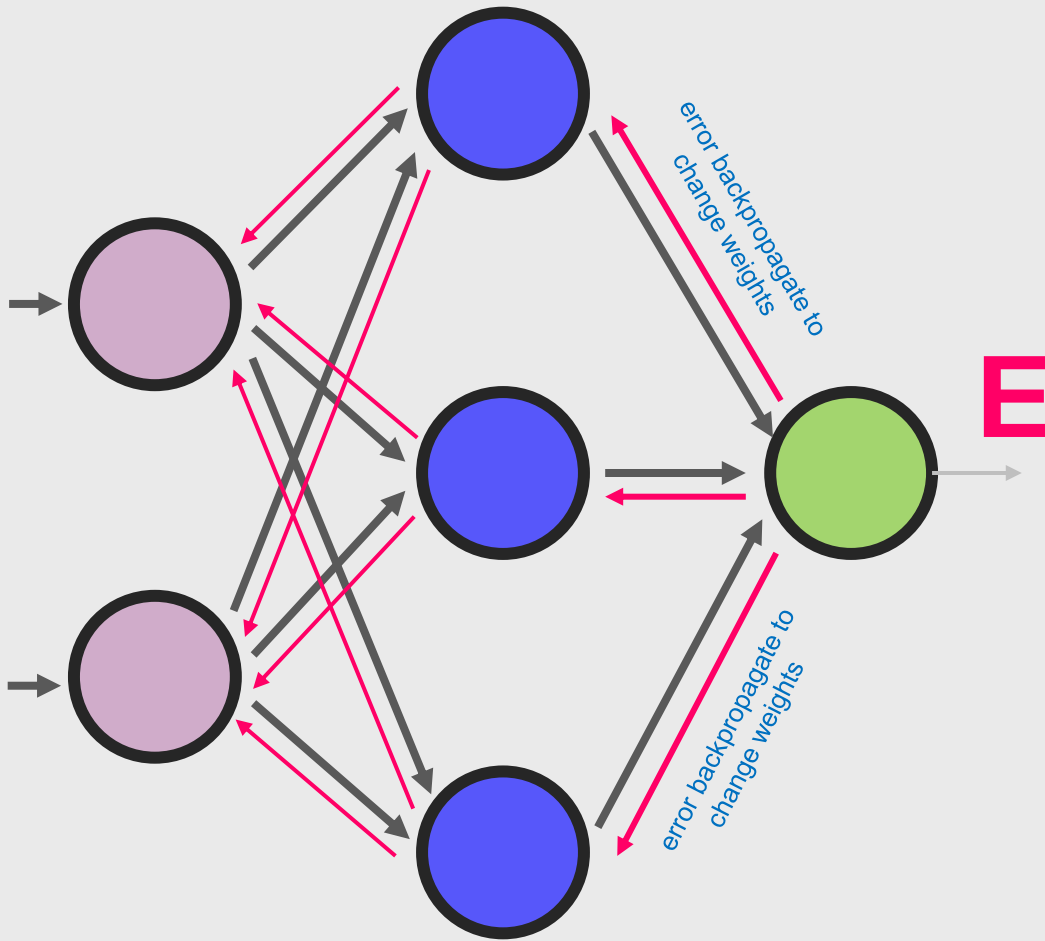
- Stochastic gradient descent
- Mini-batch gradient descent
- Batch gradient descent
- **Backpropagation**

(The algorithm at the heart of AI Revolution)



Backpropagation

Algorithm for Updating Learning Systems



David Rumelhart



Geoffrey Hinton



Ronald Williams

Learning Systems Update

$$W_{\text{NEW}} \leftarrow W_{\text{OLD}} + W_{\text{CHANGE}}$$

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors². Learning becomes more interesting but

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_j , to unit j is a linear function of the outputs, y_i , of the units that are connected to j and of the weights, w_{ji} , on these connections

$$x_j = \sum_i y_i w_{ji} \quad (1)$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

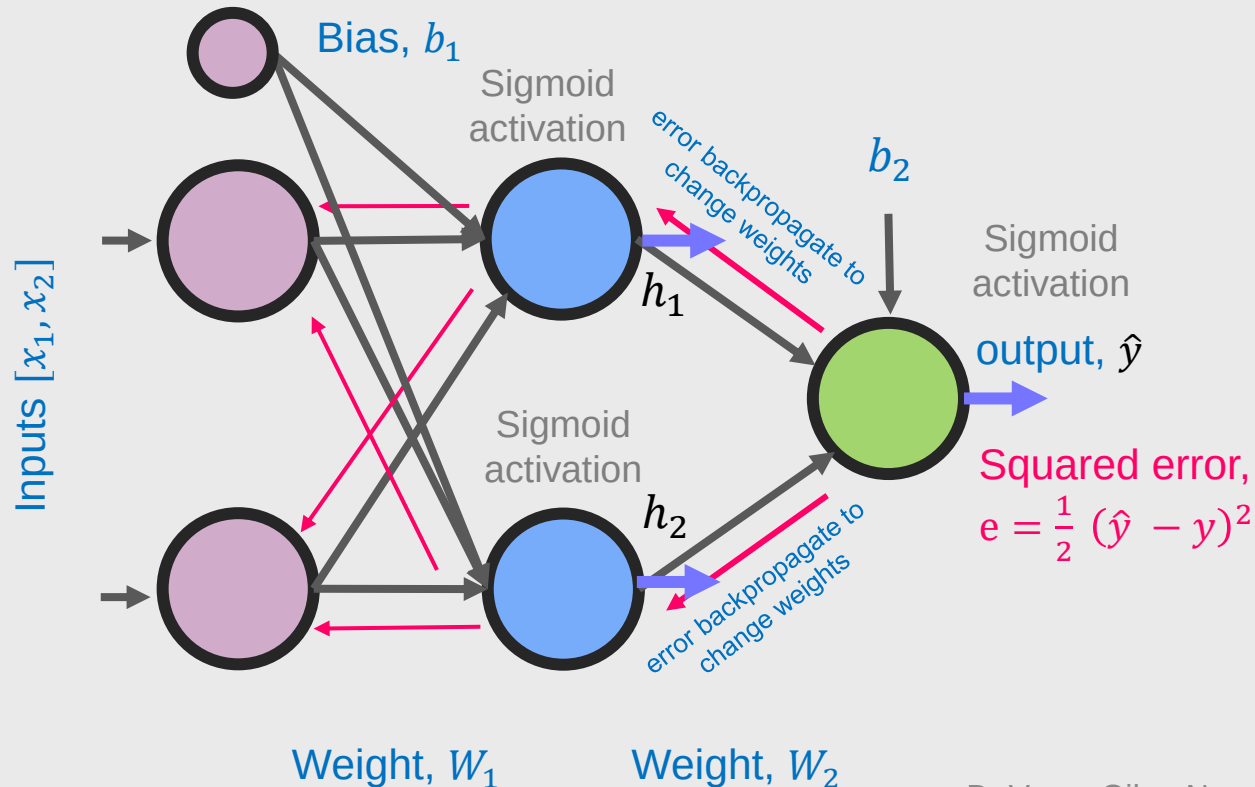
A unit has a real-valued output, y_j , which is a non-linear function of its total input

$$y_j = \frac{1}{1 + e^{-x_j}} \quad (2)$$

† To whom correspondence should be addressed.

Chain Rule

Chain rule state that the derivative of function $y = h(g(x))$, where g is a function of x and f is a function of g , then the derivative of y with respect to x is given by $\frac{dy}{dx} = \frac{dh}{dg} \cdot \frac{dg}{dx}$



Forward pass:

Hidden layer output, $h = \sigma(W_1x + b_1)$

Output layer output, $\hat{y} = \sigma(W_2h + b_2)$

Backward pass :

Output layer gradient

$$\frac{de}{d\hat{y}} = \hat{y} - y$$

Gradient output w.r.t weights, W_2 and :

$$\frac{d\hat{y}}{dW_2} = \hat{y}(1 - \hat{y})$$

Chain Rule applied to Output Layer Weights:

$$\frac{de}{dW_2} = \frac{de}{d\hat{y}} \cdot \frac{d\hat{y}}{dW_2}$$

Weight updates at output layer is as follows:

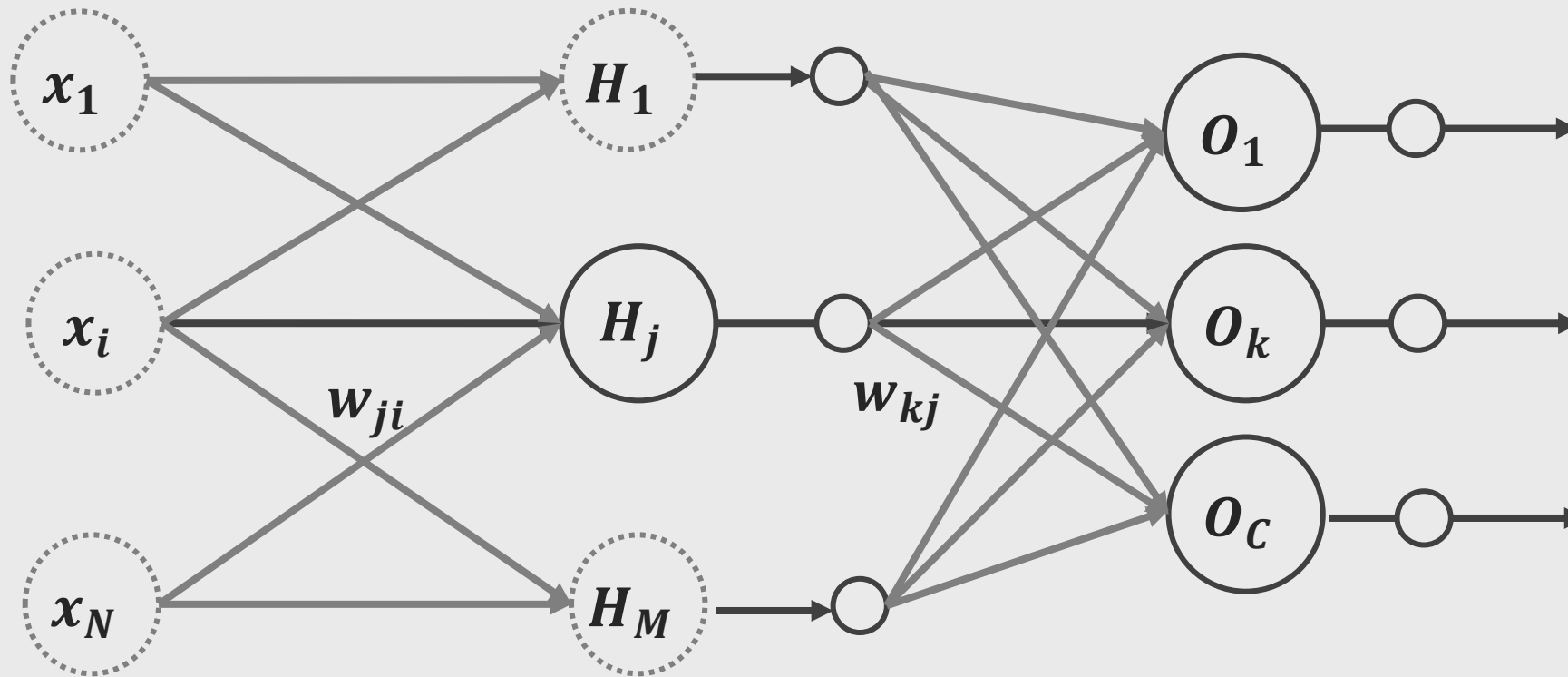
$$W_2 = W_2 - \eta \frac{de}{dW_2} h$$

Similarly other weights and biases are updated

Linnainmaa, 1970, first to demonstrate error backpropagation

Backpropagation Neural Network

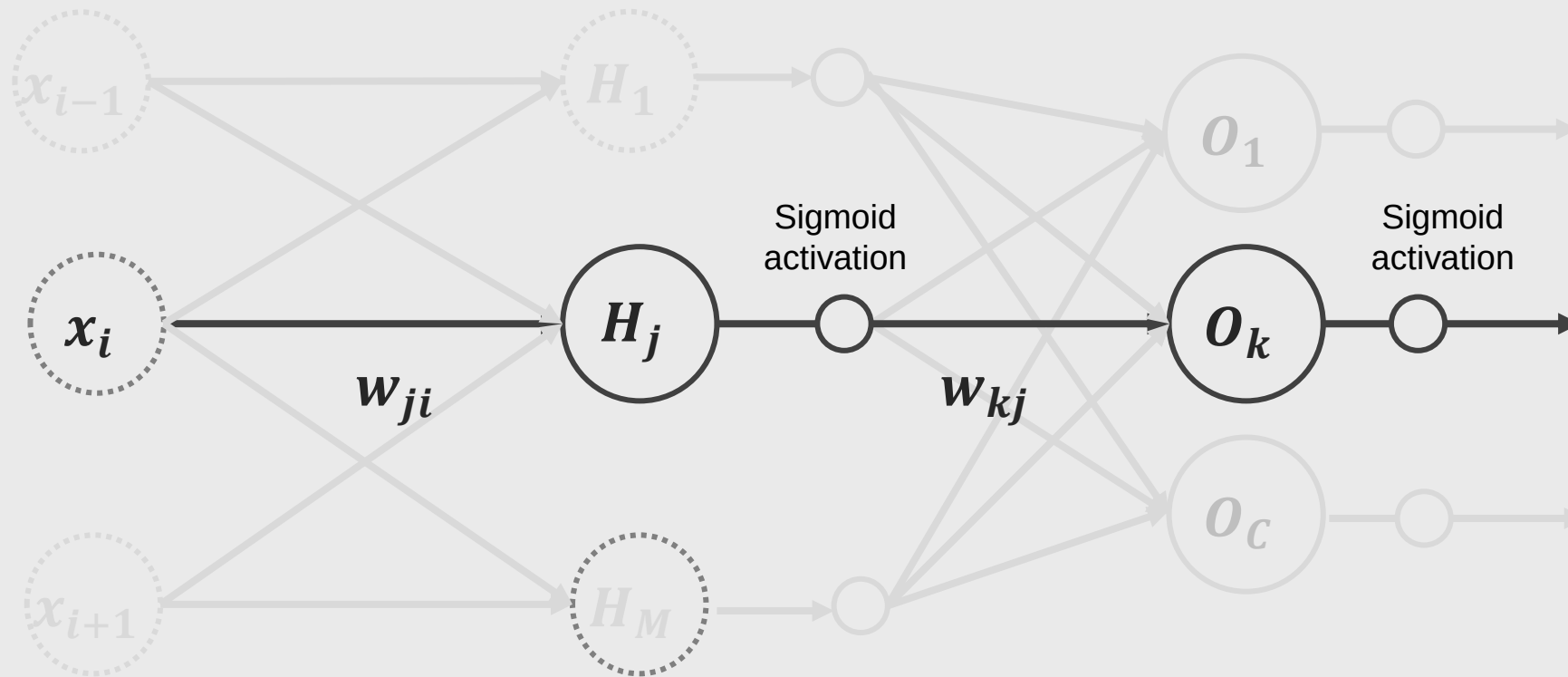
Two-layer neural network with initialised weights w_{ij} (hidden- input) and weights w_{kj} (output-hidden)



Rumelhart et al. Backpropagation

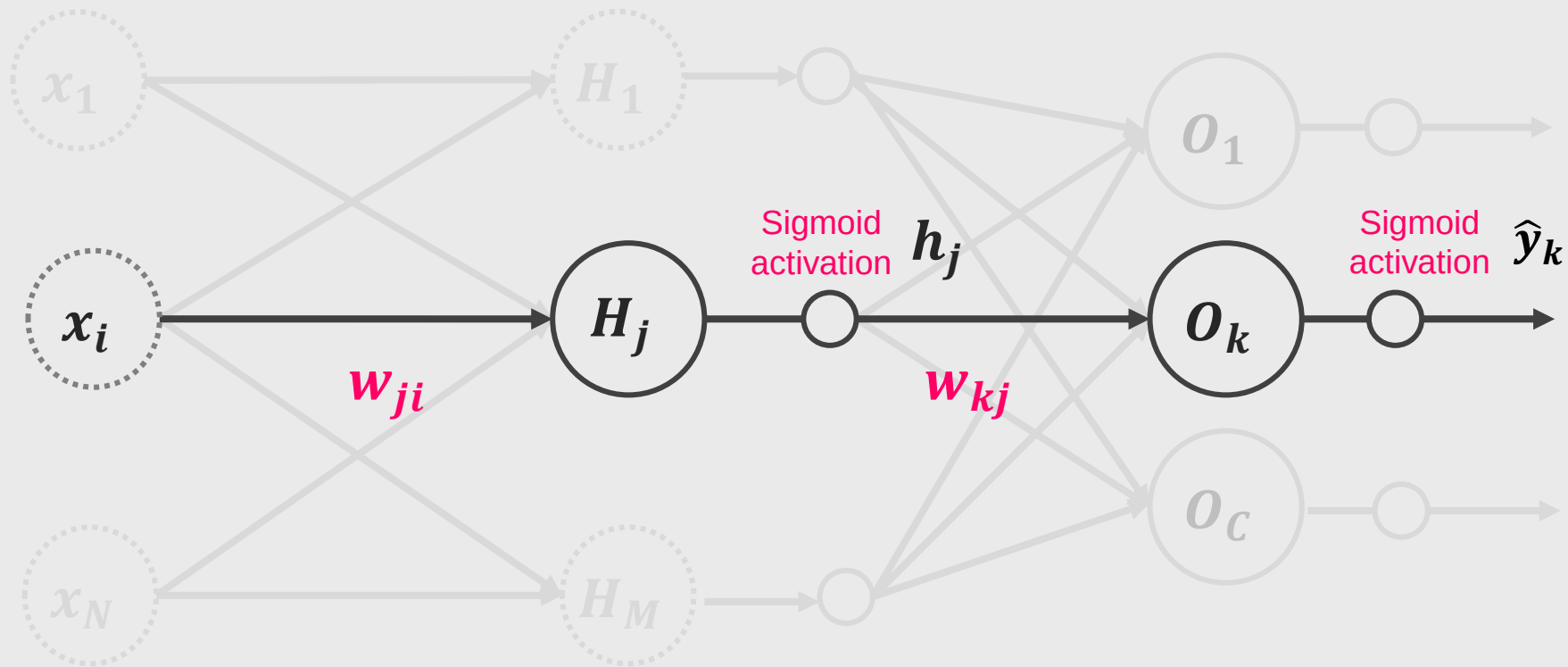
Backpropagation Neural Network

Let's take only single line of information processing on neural network for illustration



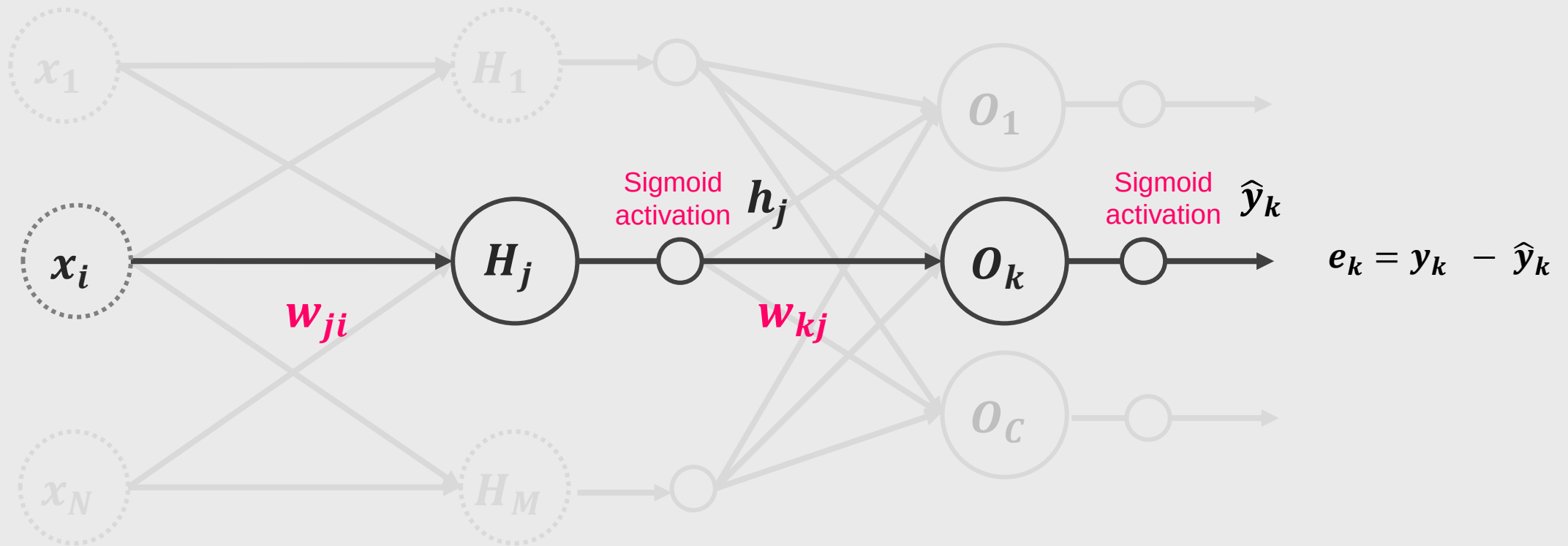
Backpropagation: Forward Pass

Hidden outputs/activation h_j and network total output are computed $\hat{y}_k$



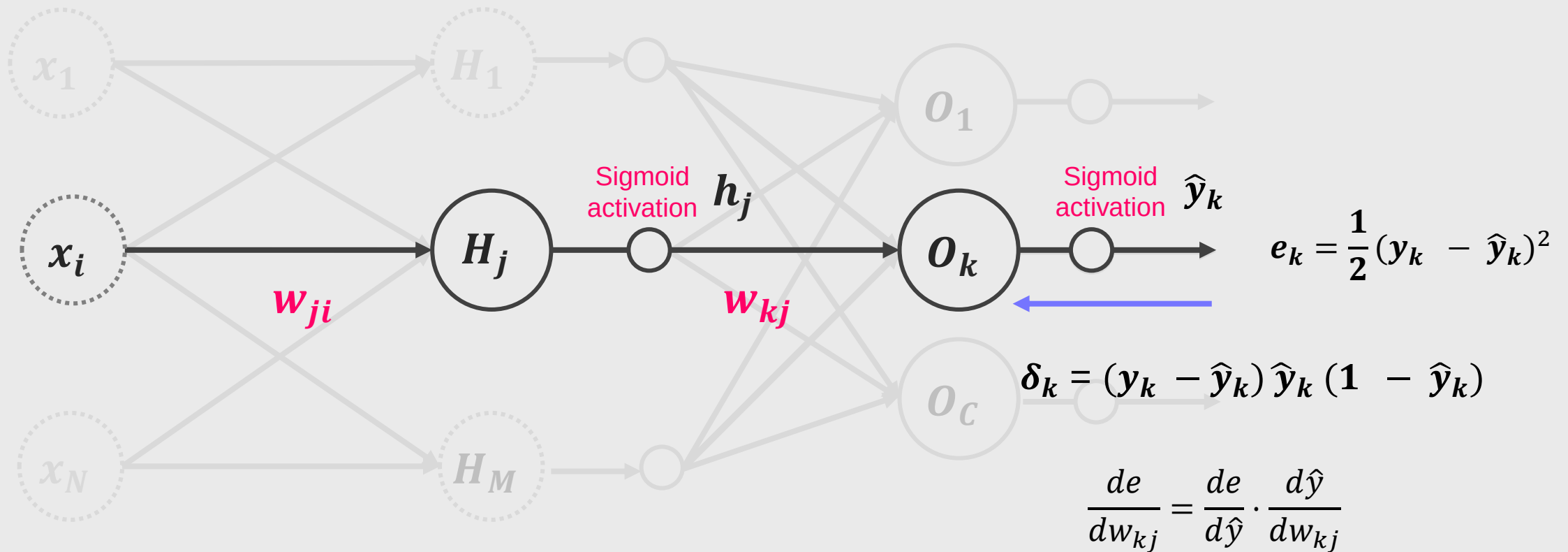
Backpropagation: Error at Output layer

Based on known target, the error $e_k = y_k - \hat{y}_k$ at output layer is computed



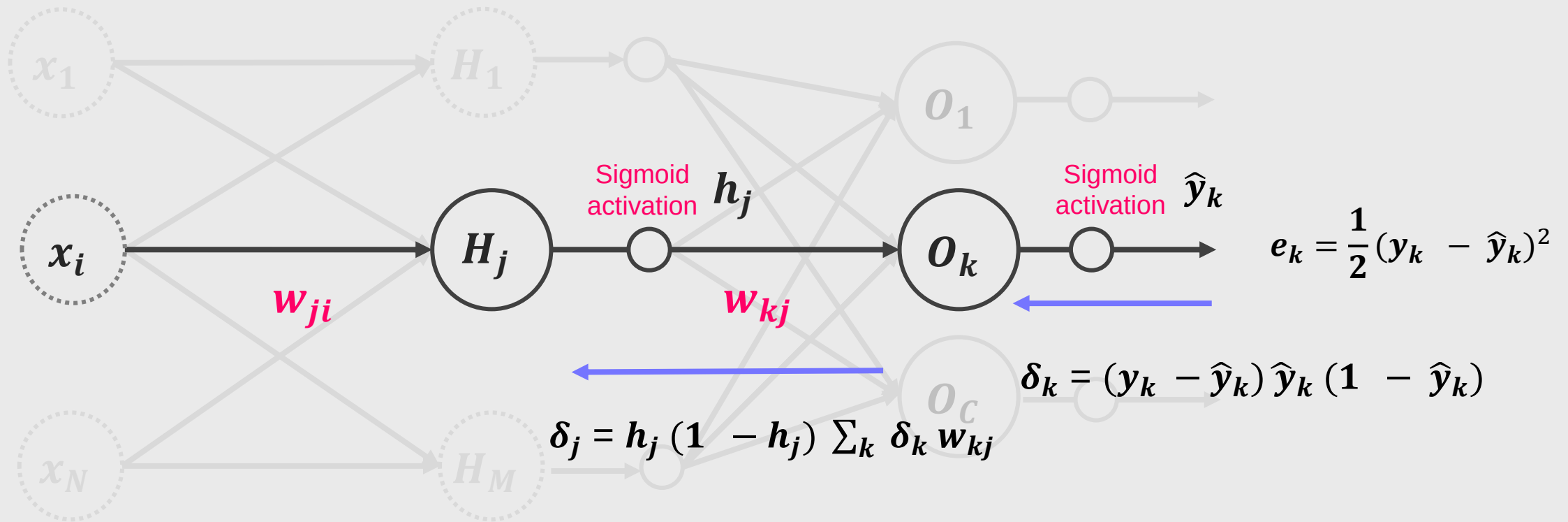
Backpropagation: Backward pass

Output layer gradient [δ_k] considering sigmoidal output node(s) is computed



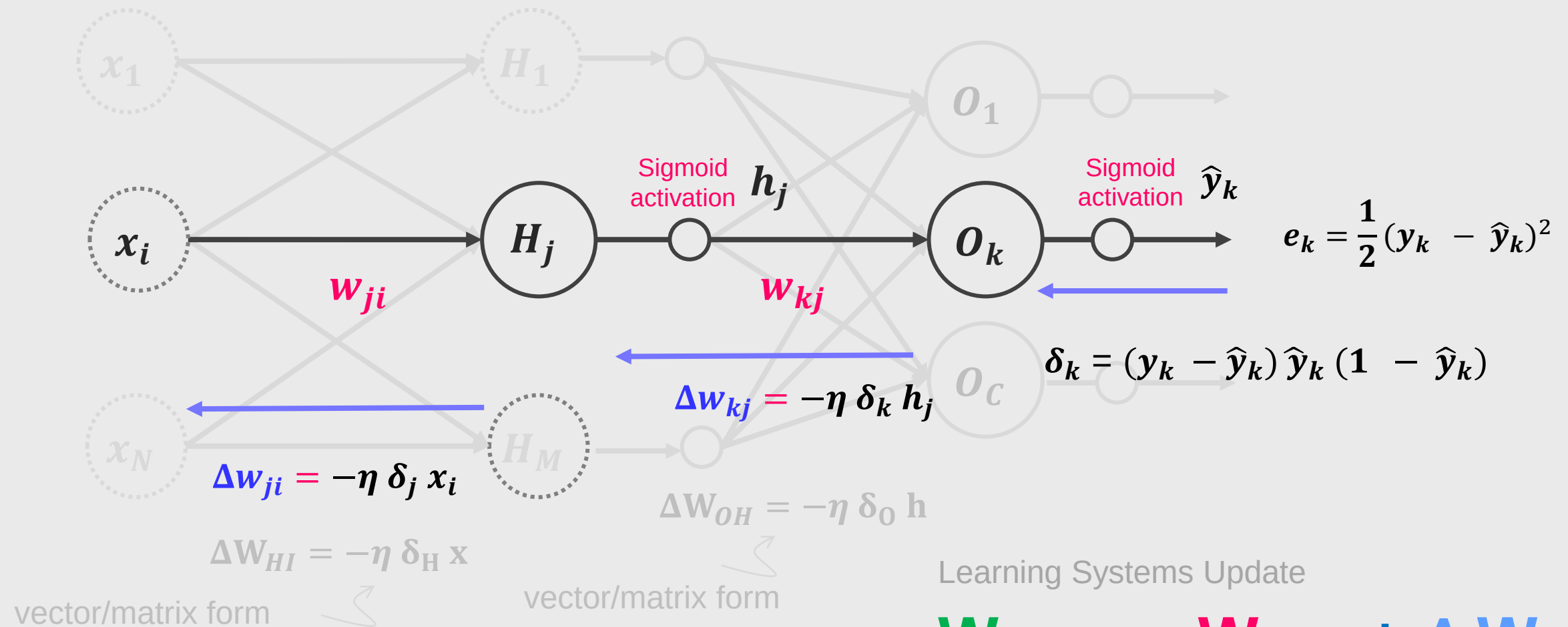
Backpropagation: Backward pass

Hidden layer gradient [delta (δ_j)] considering sigmoidal hidden node(s) is computed



Backpropagation: Backward pass

Weights change Δw_{kj} (output-hidden) Δw_{ji} (hidden-input) are computed

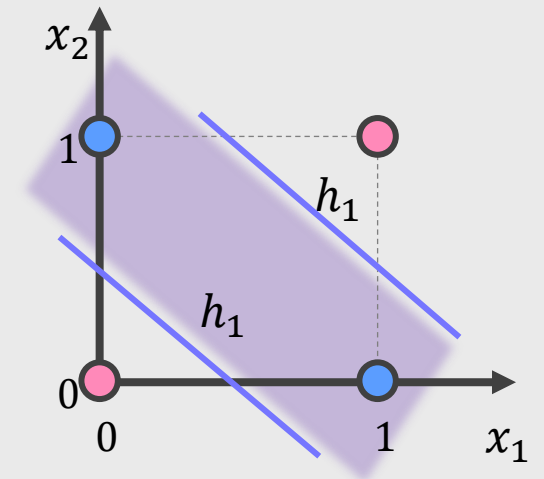
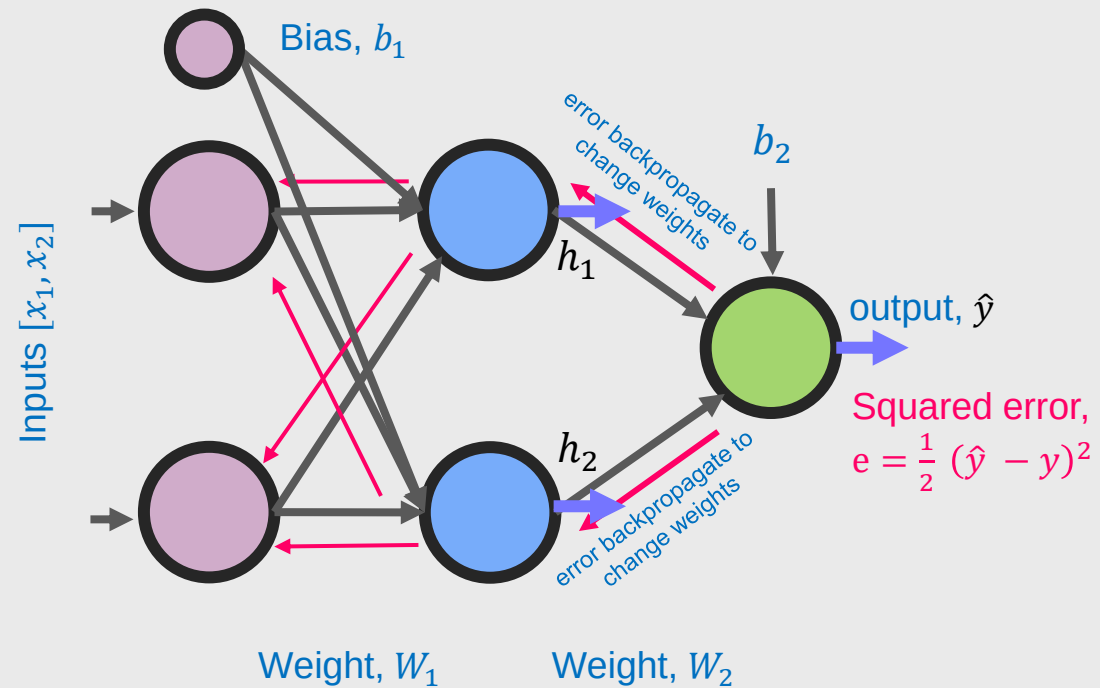


Backpropagation

Error backpropagation can solve non-linearly separable patterns

XOR Problem

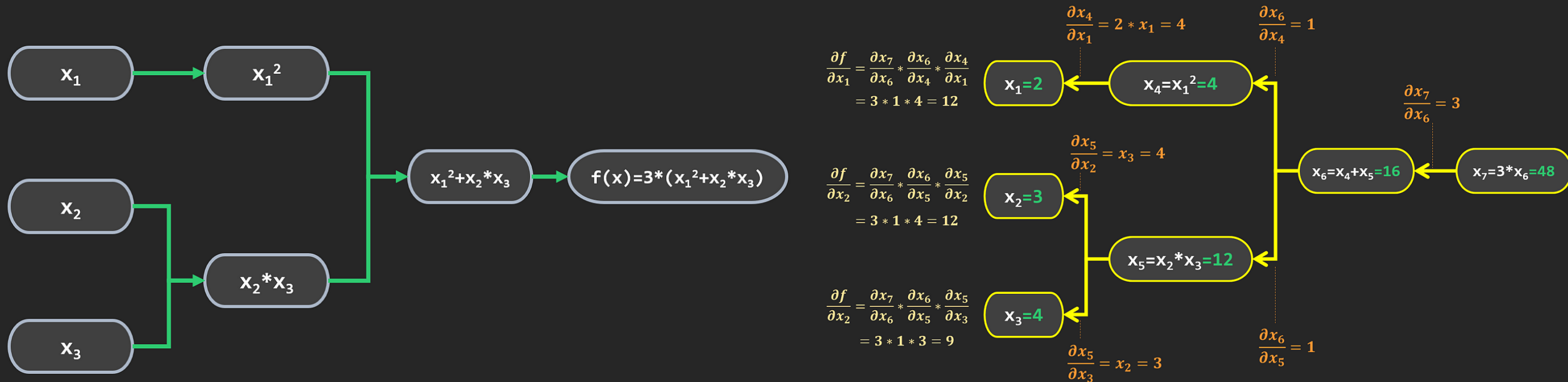
	x_1	x_2	$y = f(\mathbf{x})$
1:	0	0	0
2:	0	1	1
3:	1	0	1
4:	1	1	0



Automatic Differentiation

Linnainmaa, 1970, proposed reverse mode automatic differentiation

Tensor flow and pytorch produce computational graph for neural networks hence they use Automatic differentiation



Computational Graph / Direct Acyclic Graph (DAG)

Final derivative of DAG (see chain rule)

Example Source: <https://medium.com/@rhome/automatic-differentiation-26d5a993692b>

Backpropagation Neural Tree

Dr Varun Ojha's Work

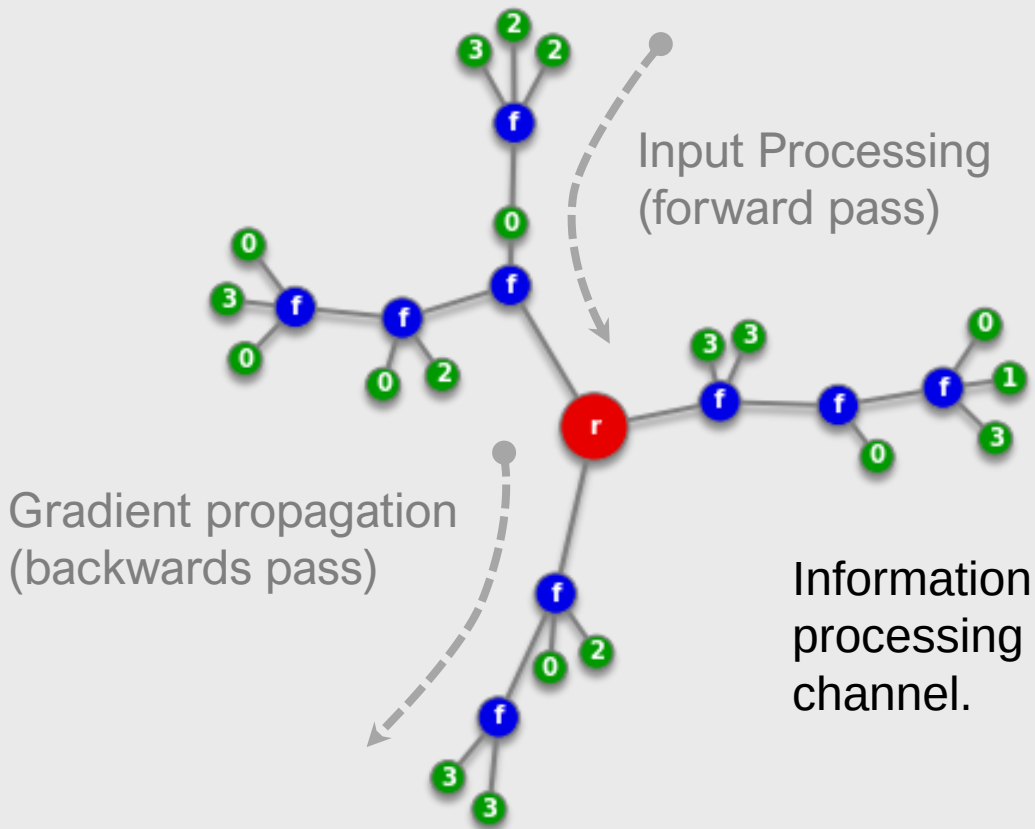
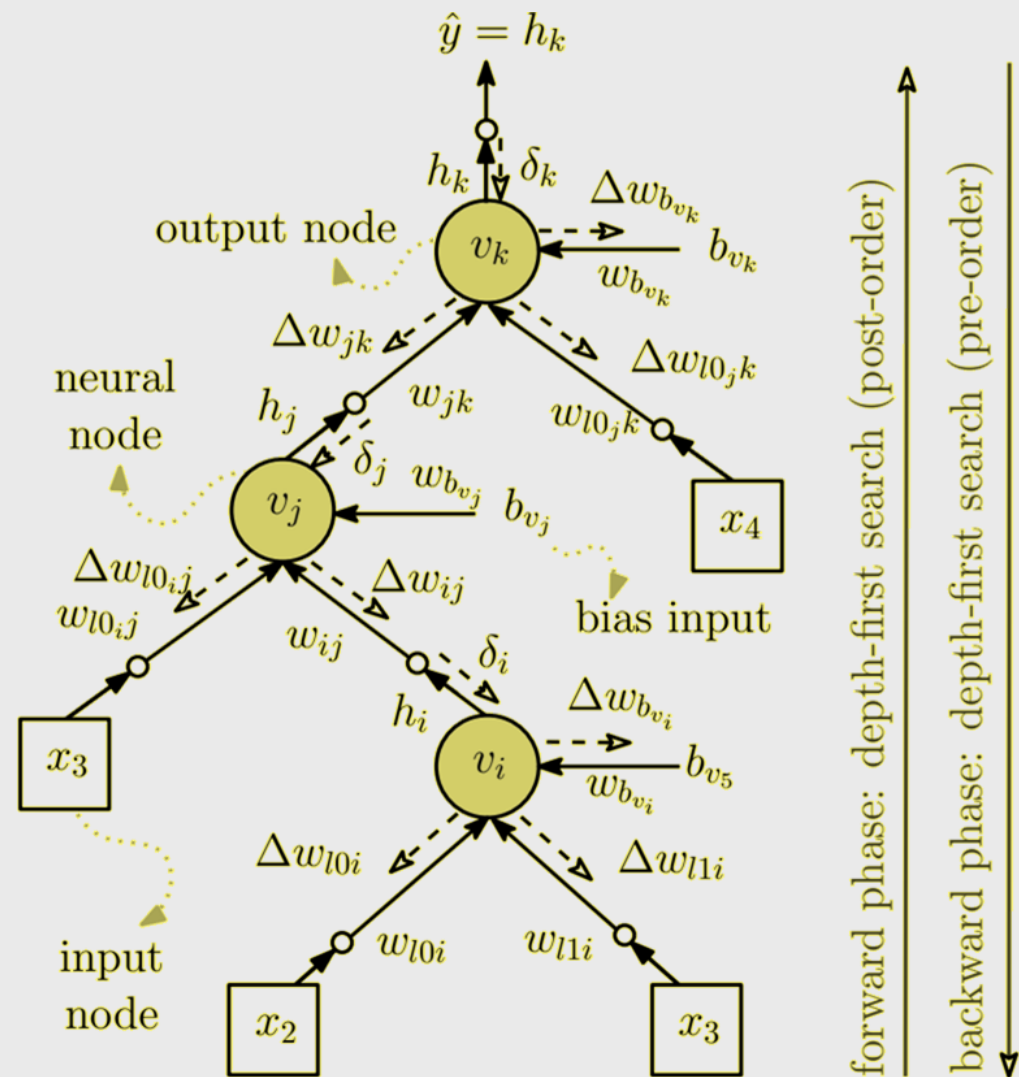
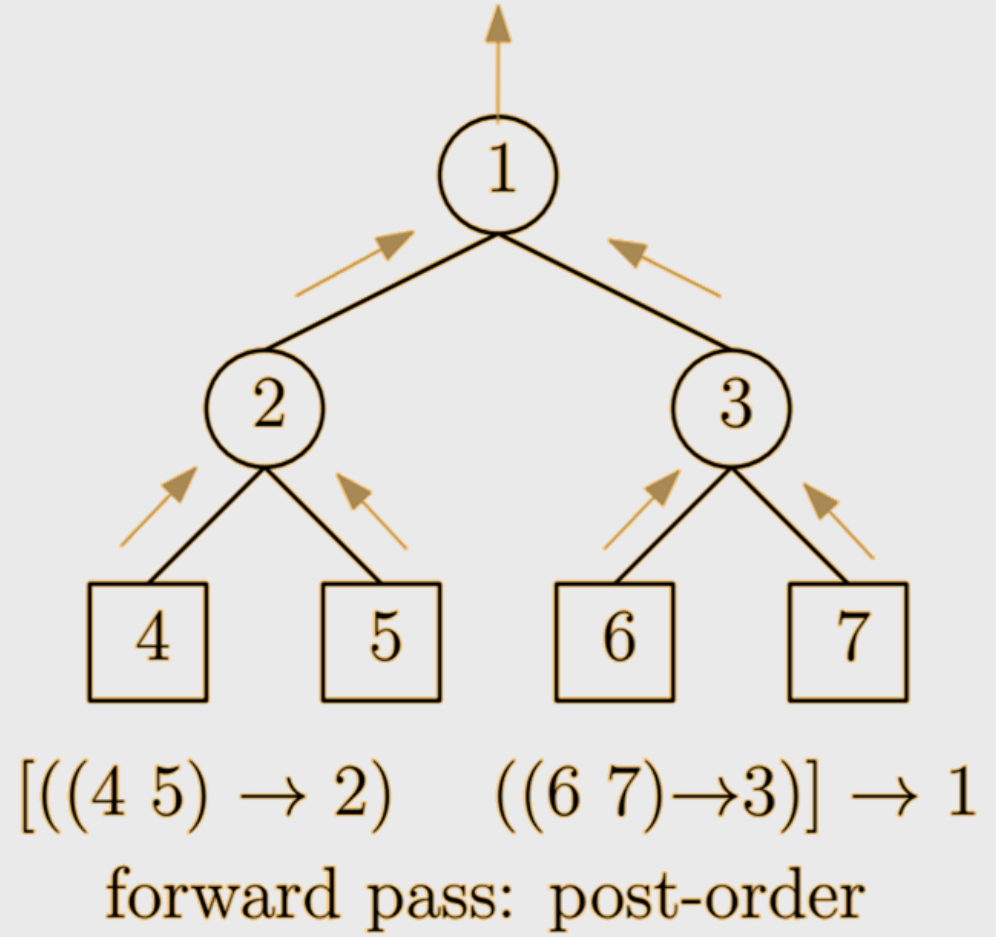
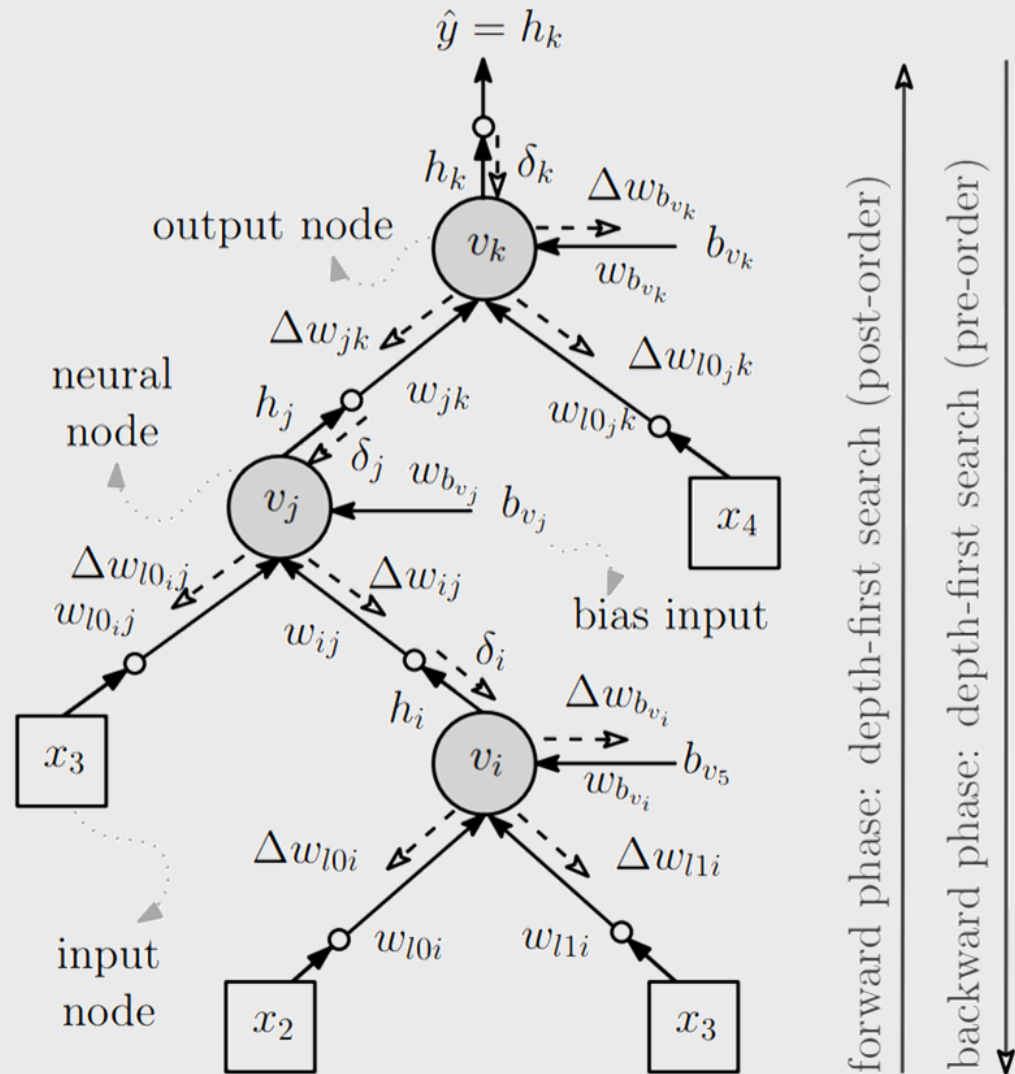


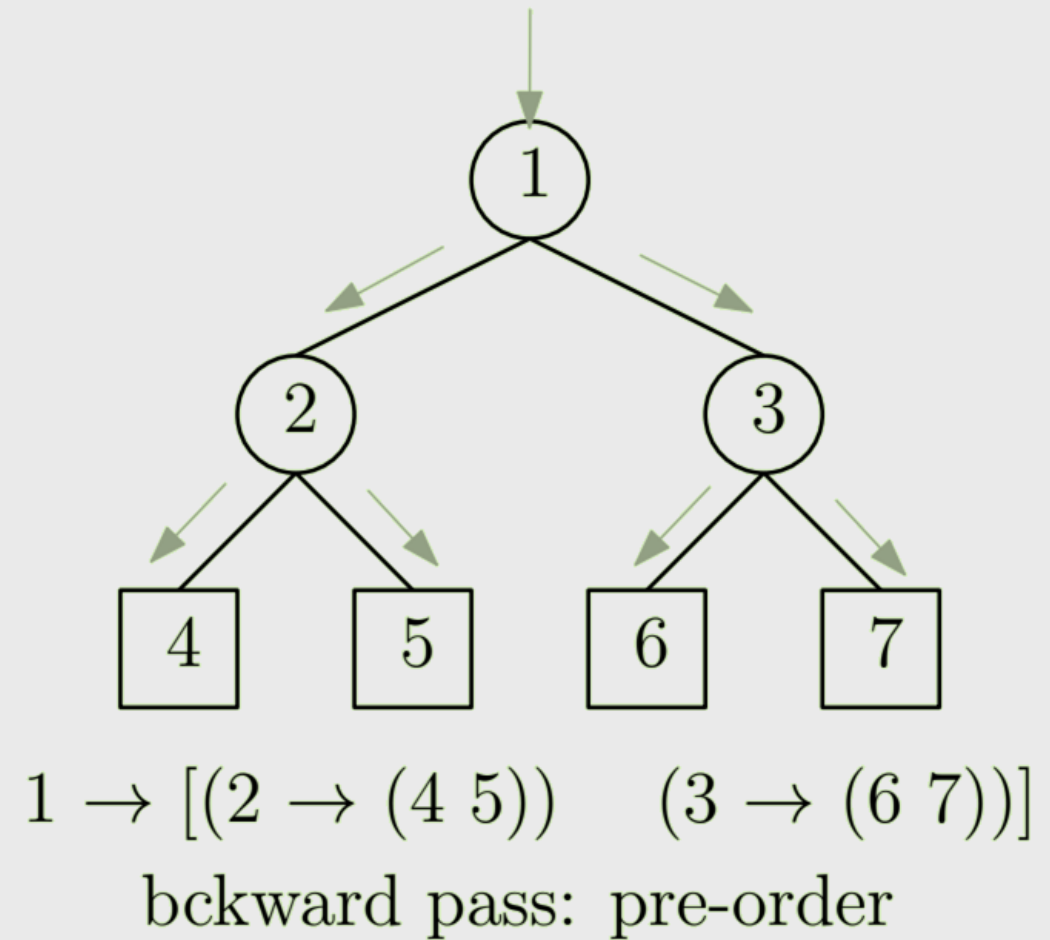
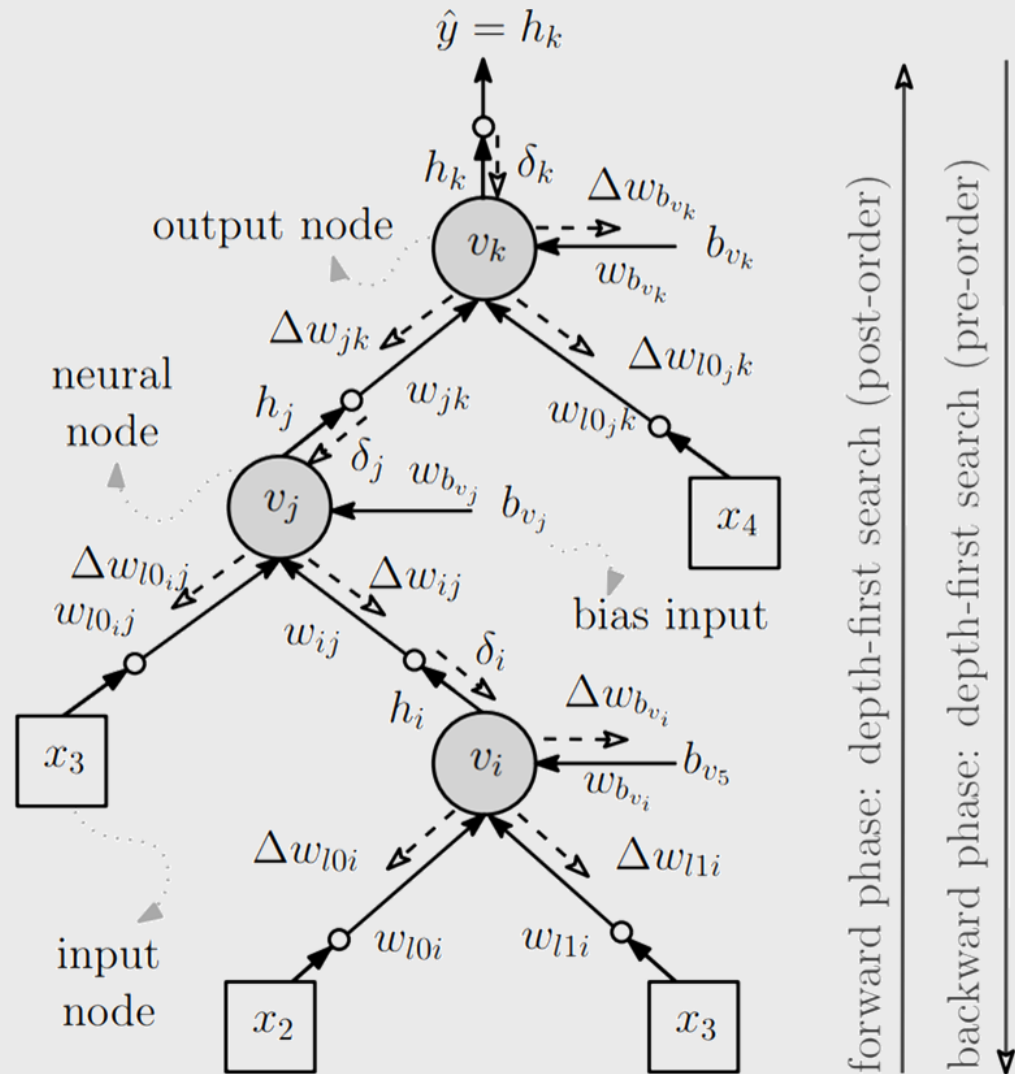
Fig A. Forward pass and gradient backpropagation



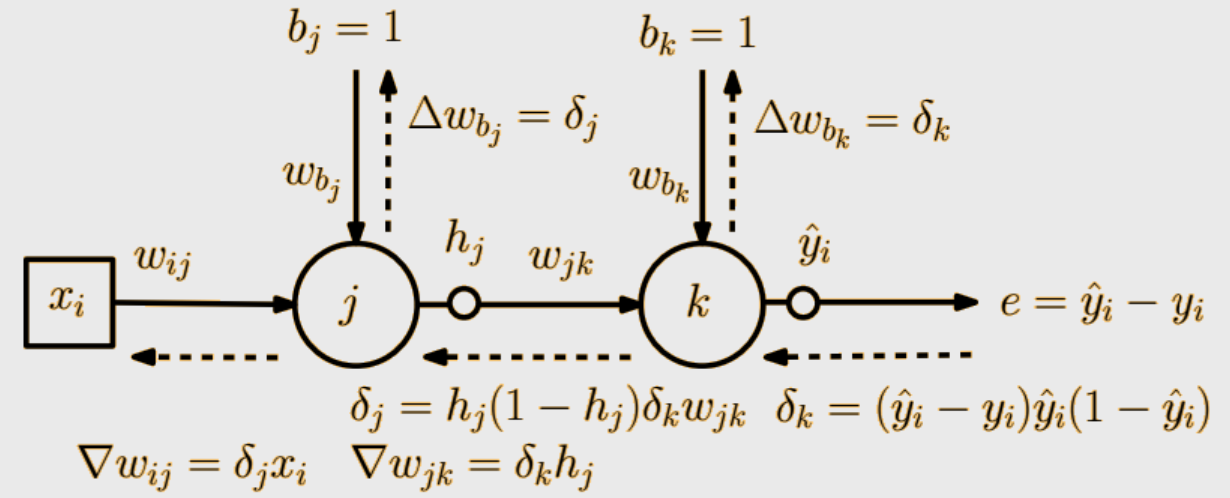
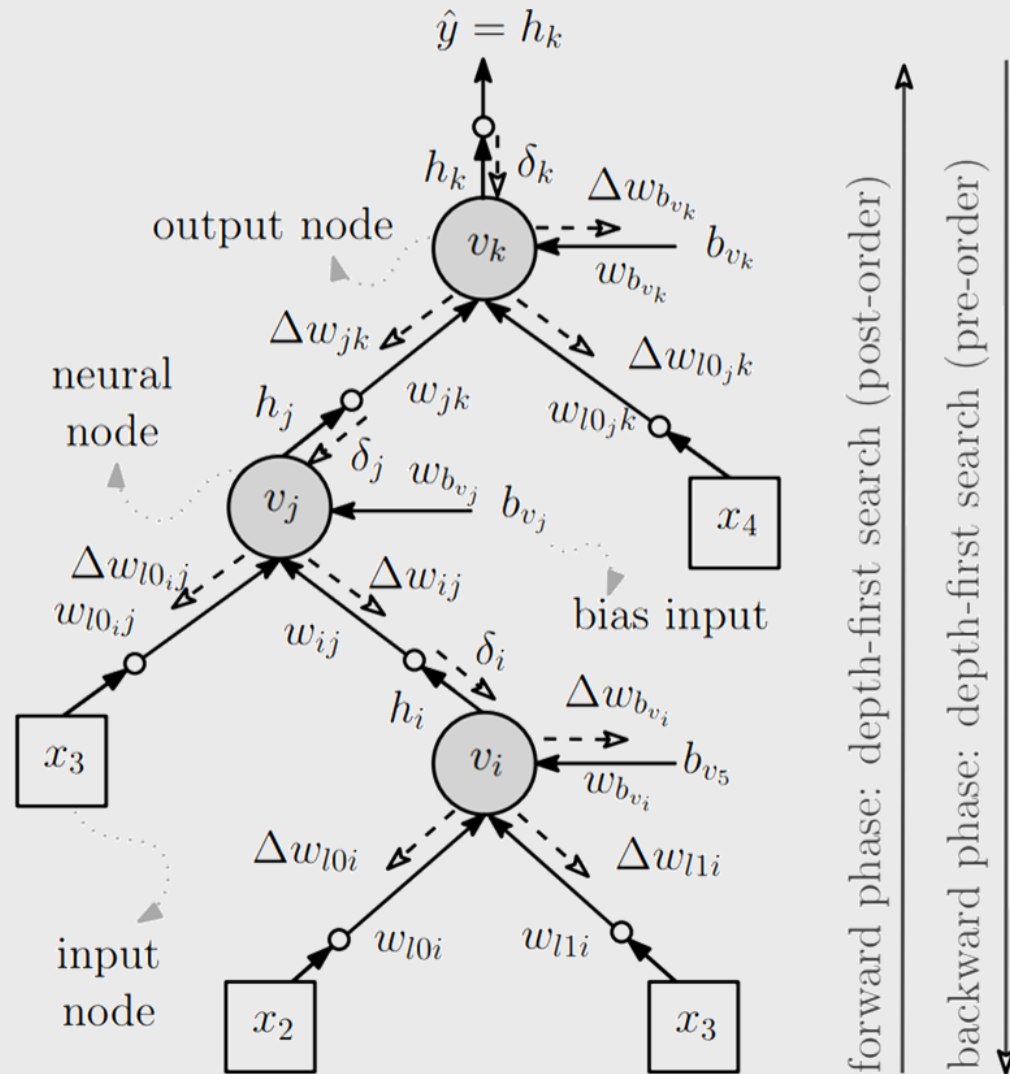
Backpropagation Neural Tree: Forward Pass



Backpropagation Neural Tree: Backward Pass



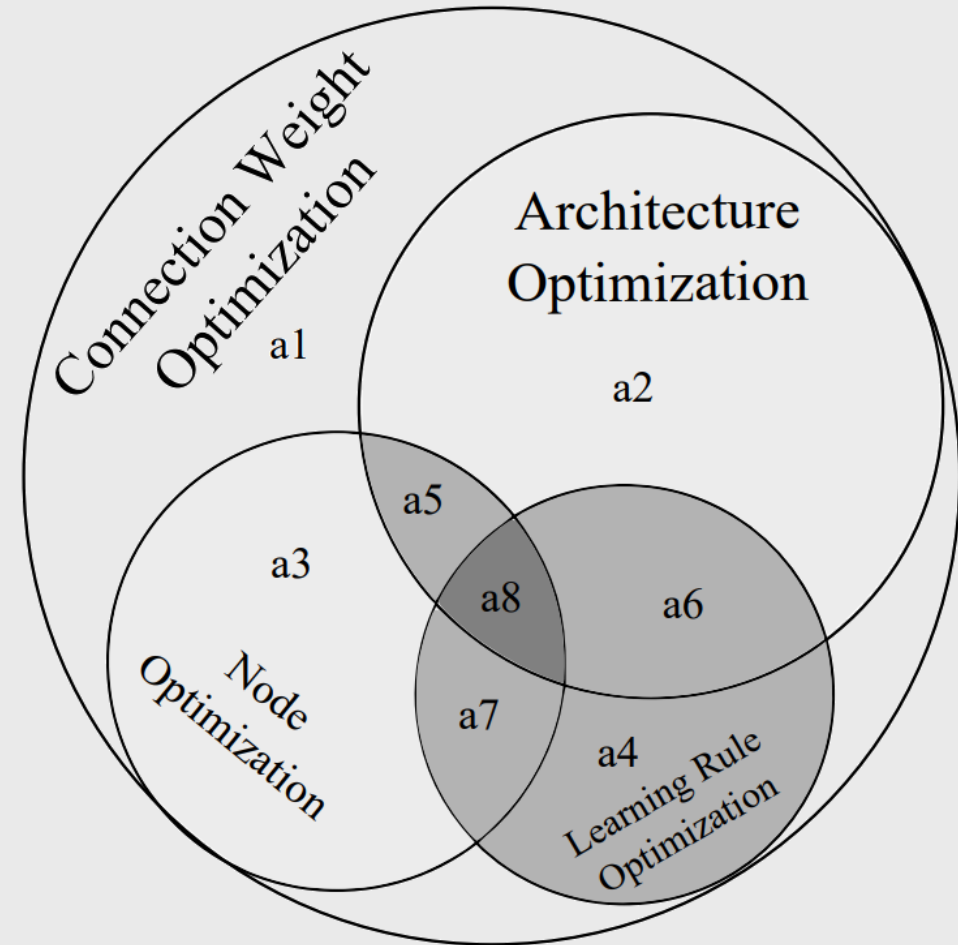
Recursive Backpropagation



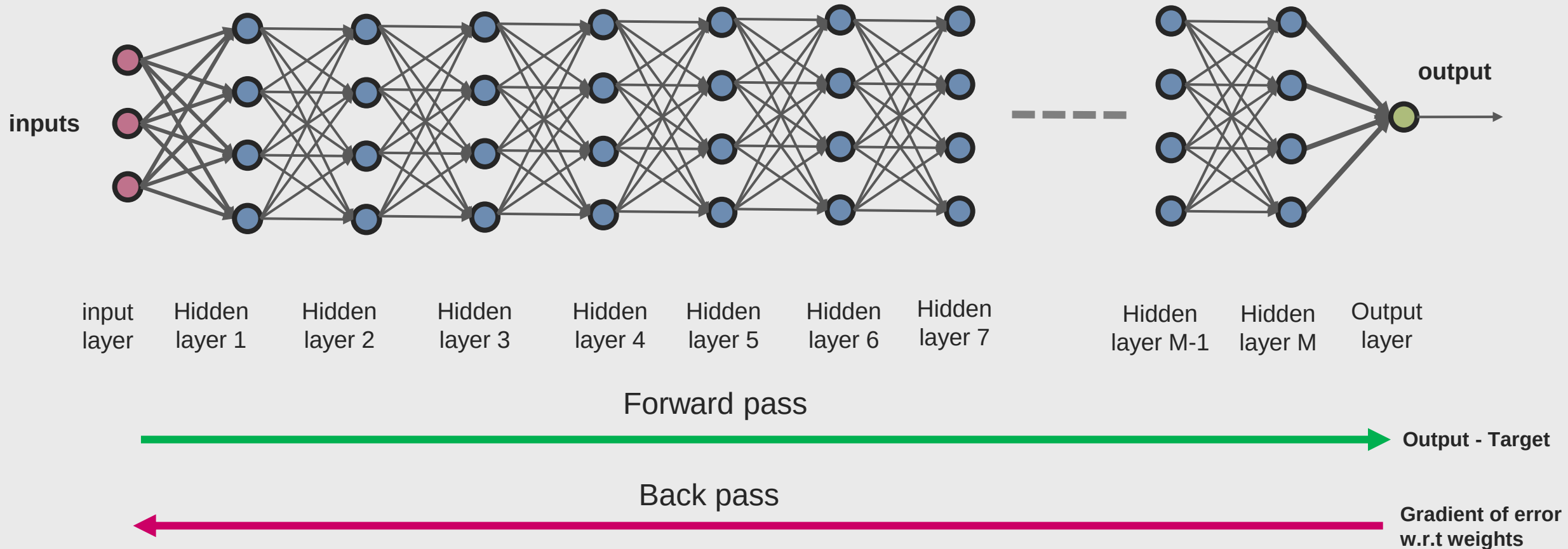
Recursive Backpropagation

Challenges faced by deep learning:

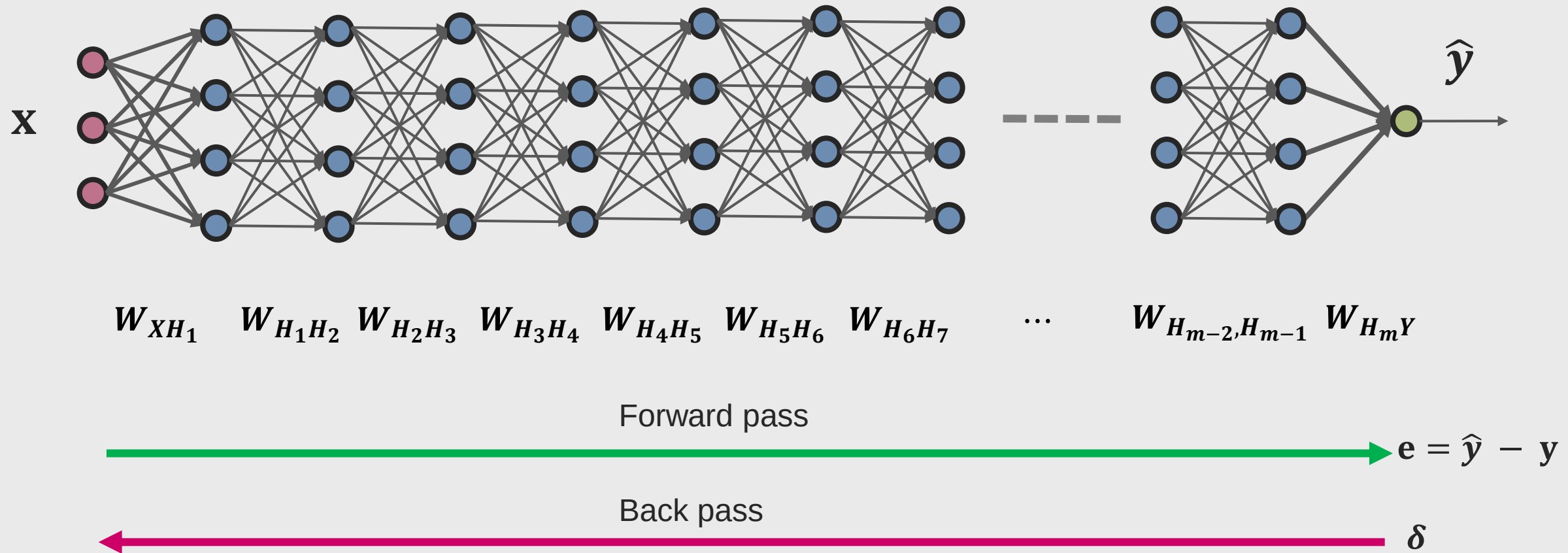
- Vanishing Gradient
- Exploding Gradient
- Activation Function
- Weight Initialisation
- Overfitting
- Loss functions



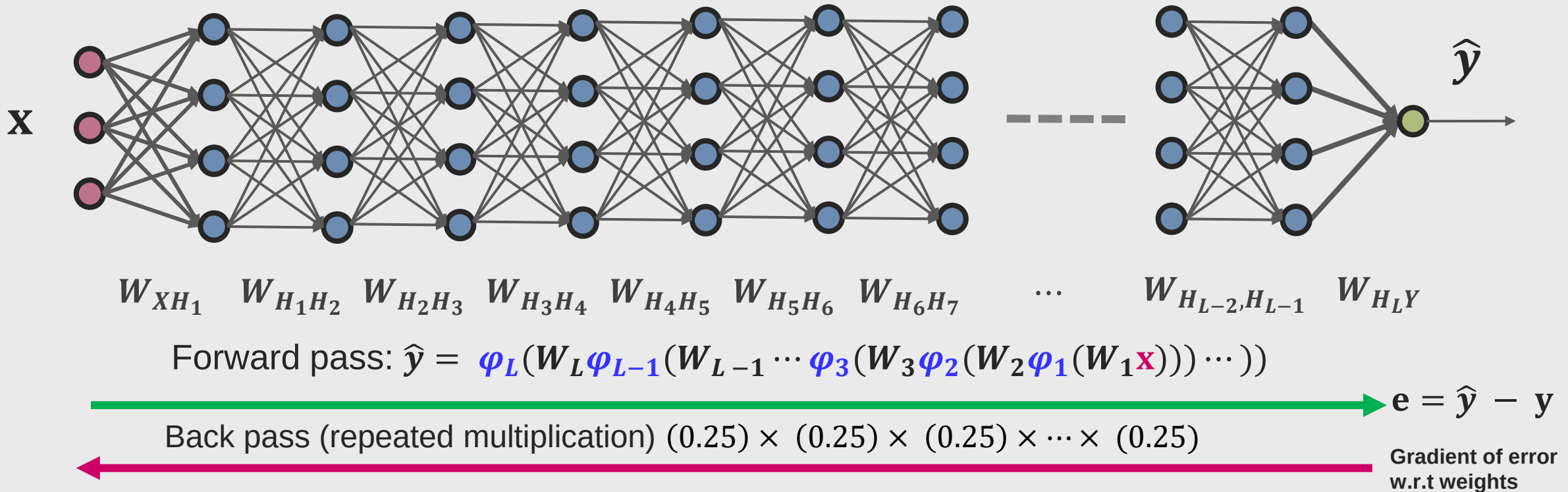
Backpropagation: Deep Learning



Backpropagation: Deep Learning



Vanishing Gradient

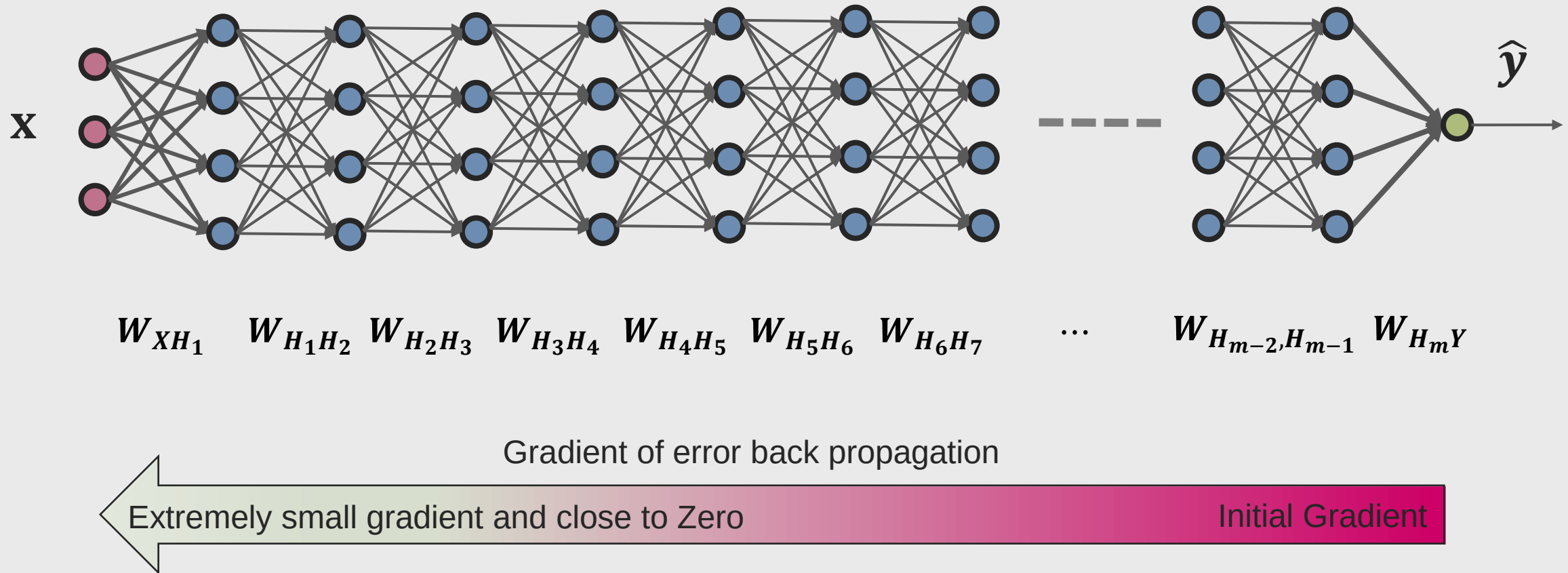


$$\hat{\mathbf{y}} = W_L \begin{bmatrix} 0.5 & \cdots & 0.0 \\ \vdots & \ddots & \vdots \\ 0.0 & \cdots & 0.5 \end{bmatrix}^{L-1}$$

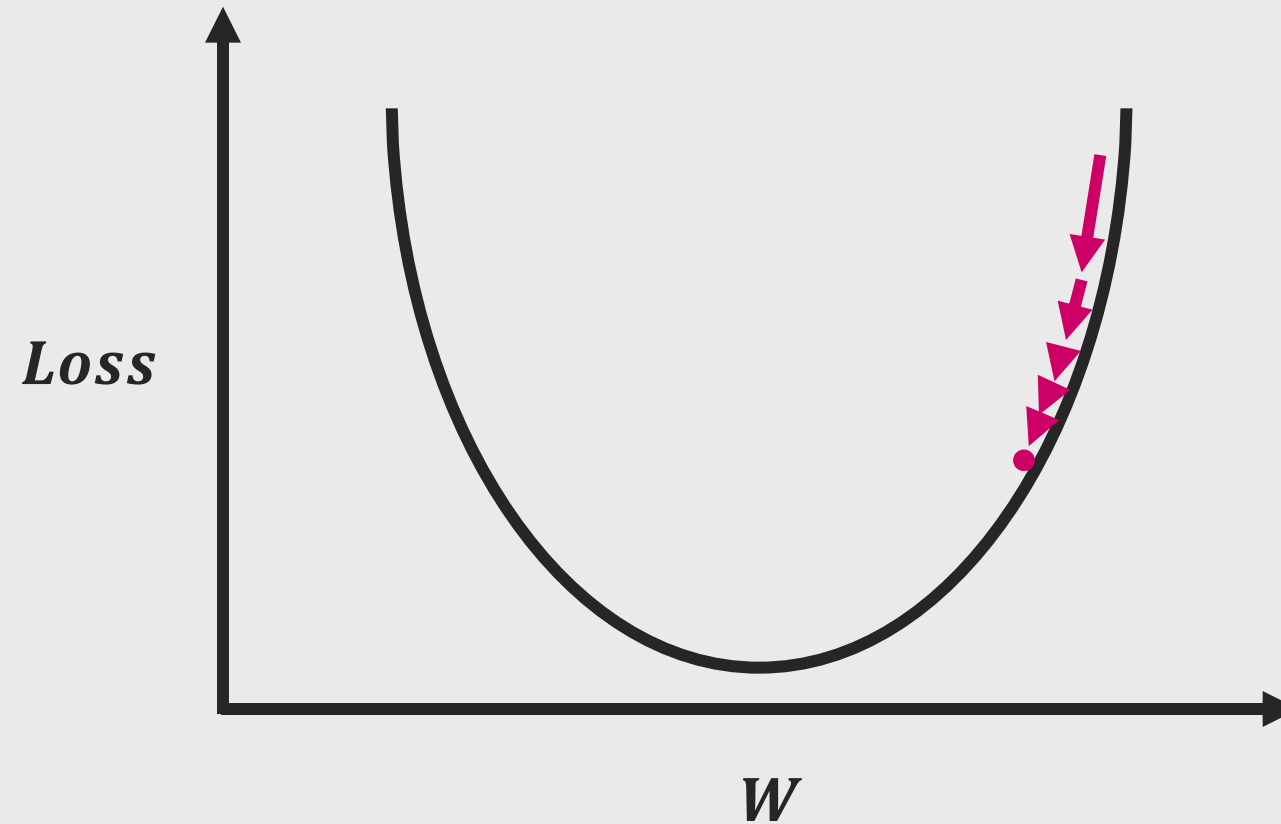
$w = 0.5^{L-1}$ for a large L this will be **extremely small**. That is, weight w is an exponentially **decreasing** function of L

This is caused by **sigmoid function** because its derivative lies between 0.0 and 0.25

Vanishing Gradient



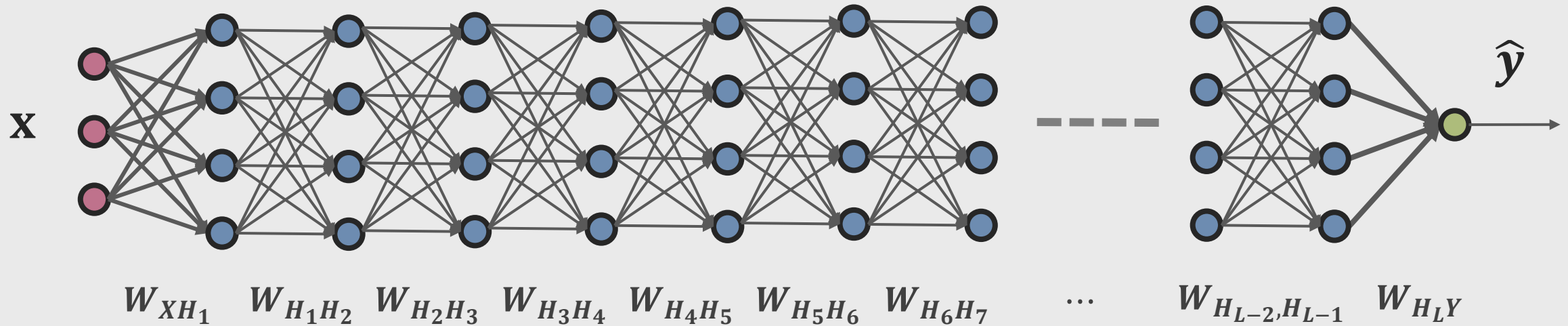
Vanishing Gradient



gradient become
very small

Convergence
virtually stops
because weights do
not change any more

Vanishing Gradient



Forward pass: $\hat{y} = \varphi_L(W_L \varphi_{L-1}(W_{L-1} \cdots \varphi_3(W_3 \varphi_2(W_2 \varphi_1(W_1 \mathbf{x}))) \cdots))$

$\mathbf{e} = \hat{y} - y$

$$\hat{y} = W_L \begin{bmatrix} 0.5 & \cdots & 0.0 \\ \vdots & \ddots & \vdots \\ 0.0 & \cdots & 0.5 \end{bmatrix}^{L-1}$$

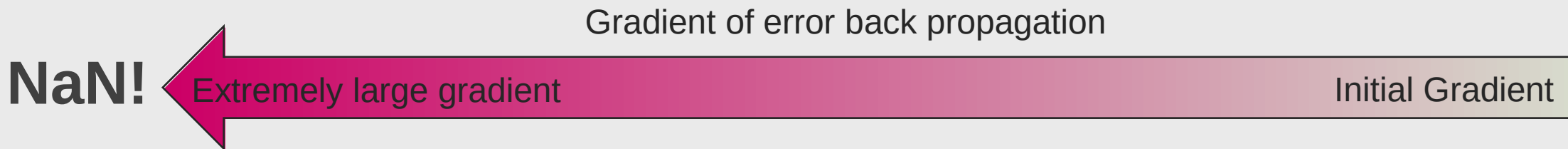
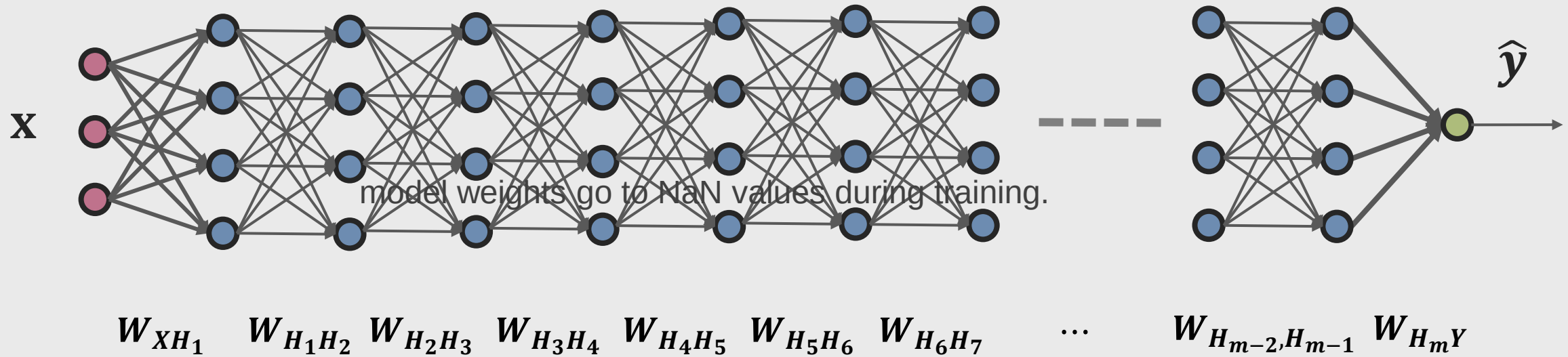
$w = 0.5^{L-1}$ for a large L this will be **extremely small**. That is, weight w is an exponentially **decreasing** function of L

Solution:

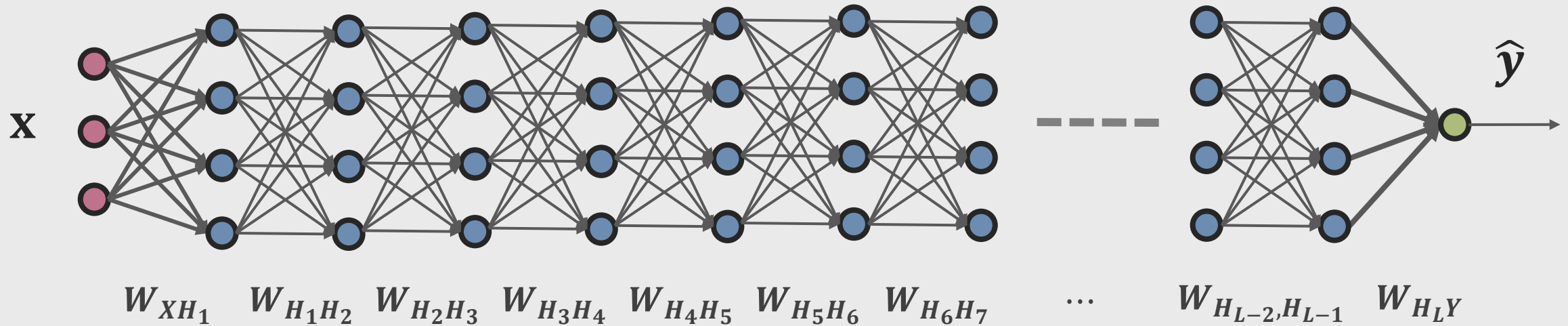
Use of **ReLU** function

$$\varphi_1(x) = \max(0, x)$$

Exploding Gradient



Exploding Gradient



Forward pass: $\hat{y} = \varphi_L(W_L \varphi_{L-1}(W_{L-1} \cdots \varphi_3(W_3 \varphi_2(W_2 \varphi_1(W_1 x))) \cdots))$

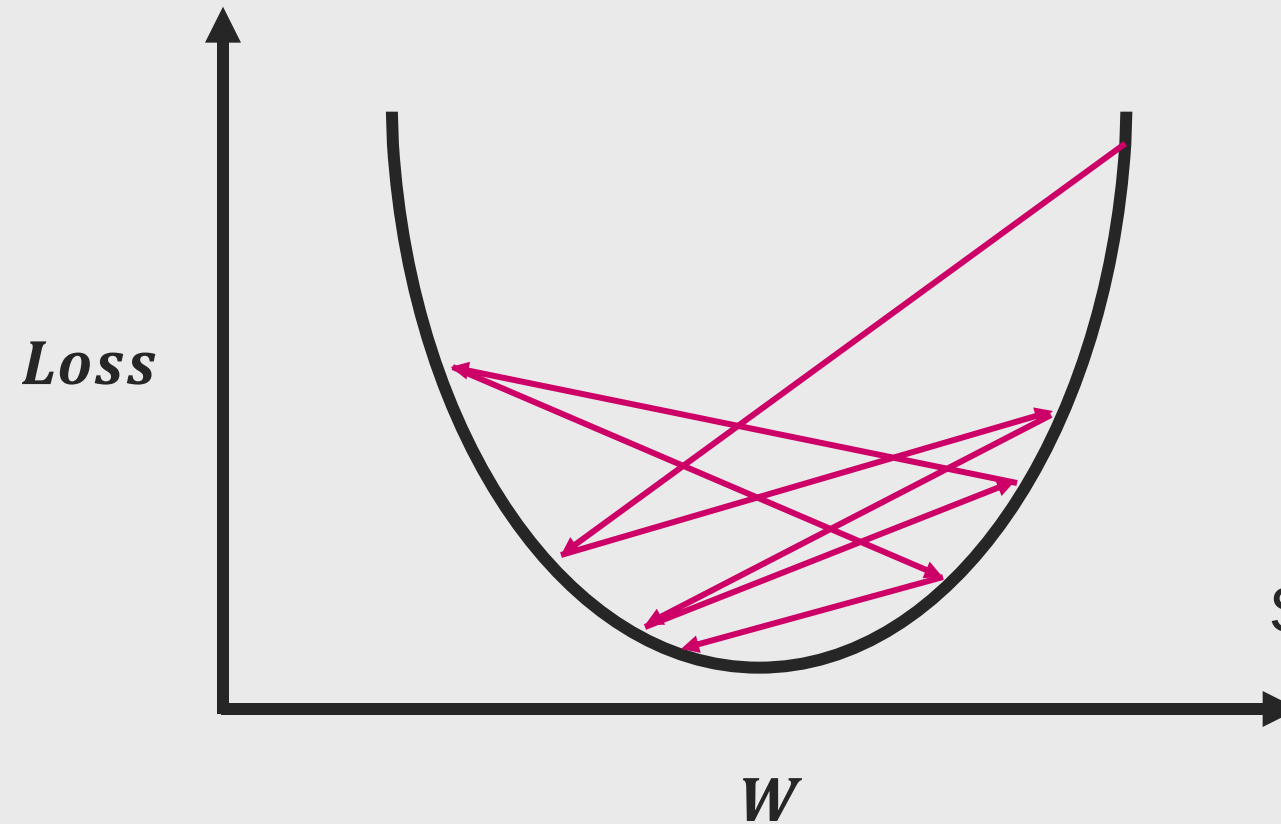
$\xrightarrow{\text{green arrow}} e = \hat{y} - y$

$$\hat{y} = W_L \begin{bmatrix} 1.5 & \cdots & 0.0 \\ \vdots & \ddots & \vdots \\ 0.0 & \cdots & 1.5 \end{bmatrix}^{L-1}$$

$w = 1.5^{L-1}$ for a large L this will be **extremely larger**. That is, weight w is an exponentially **increase** function of L

This is caused by **initialization of weights with large values** activation functions (e.g., ReLU) have a magnitude **greater than 1.0**

Exploding Gradient

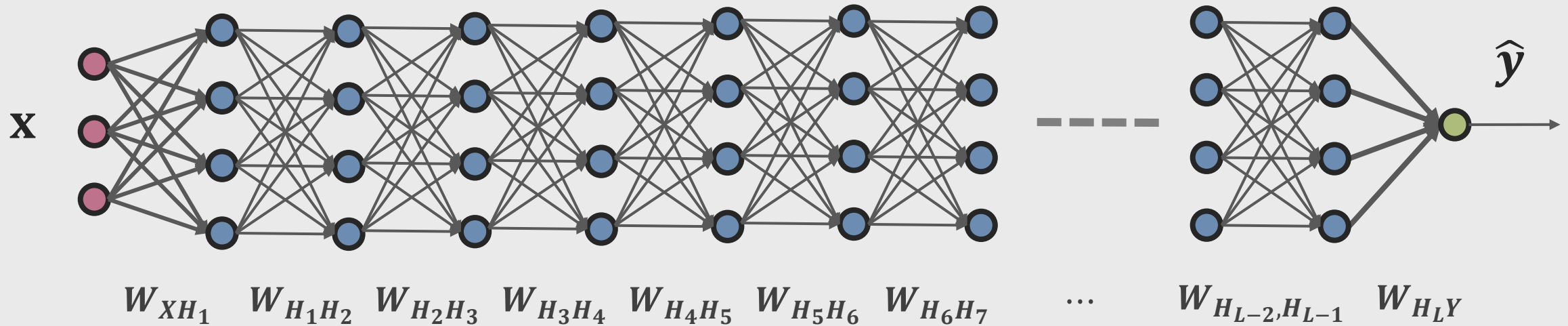


Highly fluctuating
gradient descent

Weight abruptly
changes.

Skips Global minima

Exploding Gradient



Forward pass: $\hat{y} = \varphi_L(W_L \varphi_{L-1}(W_{L-1} \cdots \varphi_3(W_3 \varphi_2(W_2 \varphi_1(W_1 x))) \cdots))$

$\rightarrow e = \hat{y} - y$

$$\hat{y} = W_L \begin{bmatrix} 1.5 & \cdots & 0.0 \\ \vdots & \ddots & \vdots \\ 0.0 & \cdots & 1.5 \end{bmatrix}^{L-1}$$

$w = 1.5^{L-1}$ for a large L this will be extremely larger. That is, weight w is an exponentially **increase** function of L

Solution:

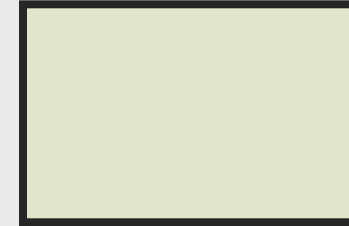
Gradient clipping and/or better weight initialization/Regularization

Training Method

(from previous lecturers)



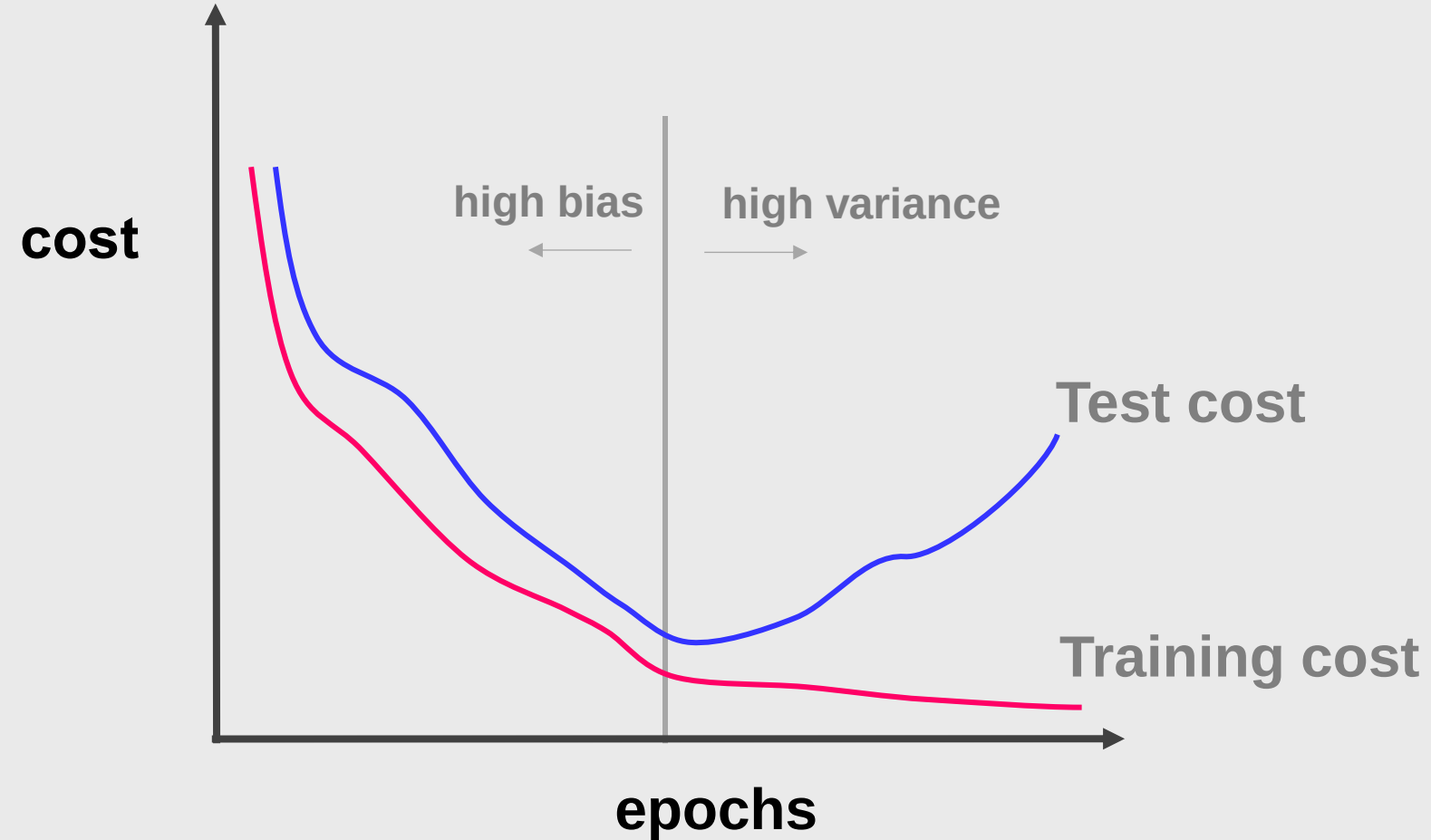
Training Set



Test Set

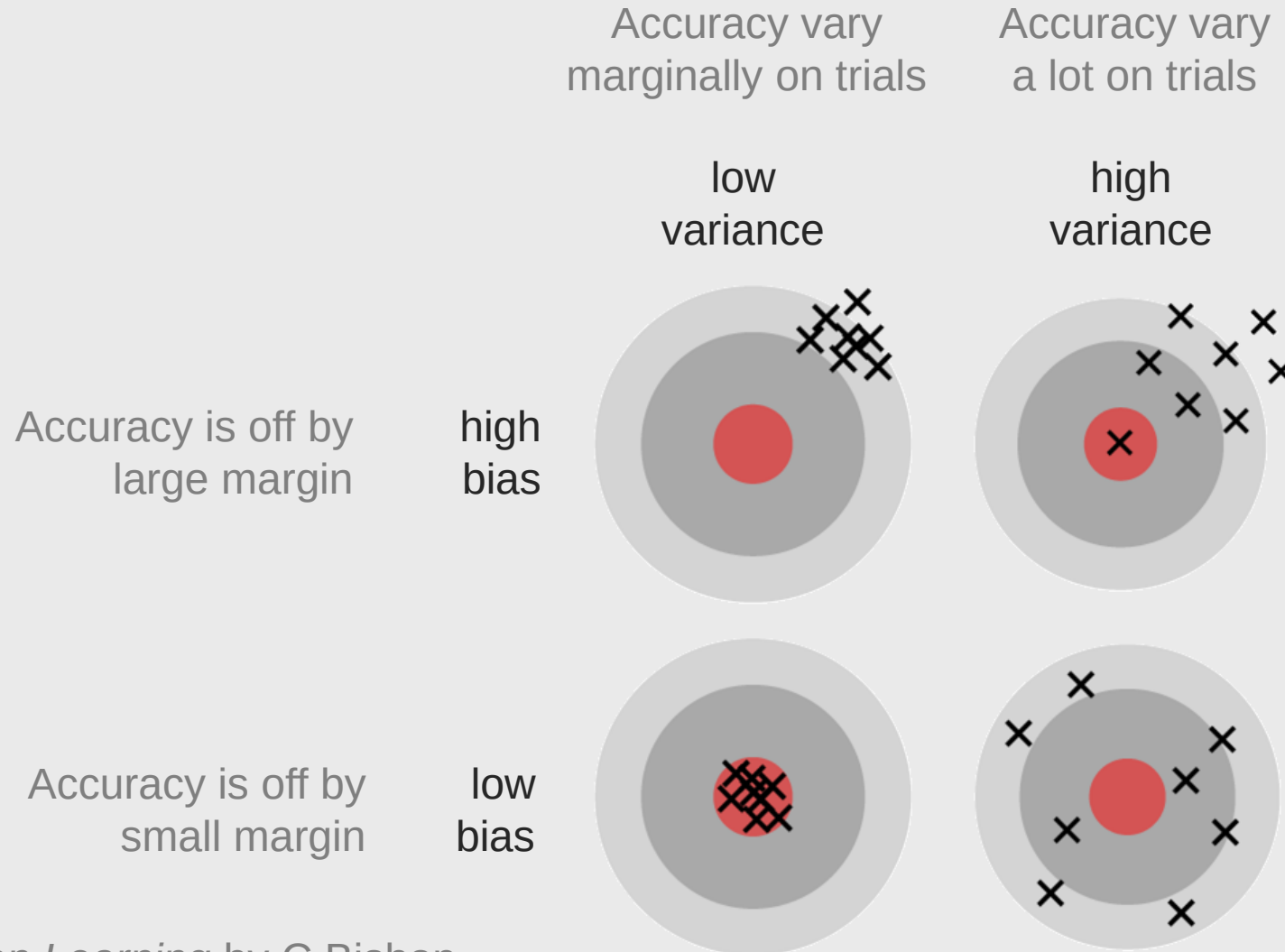
Training Method

(from previous lecturers)



Bias-Variance Issue

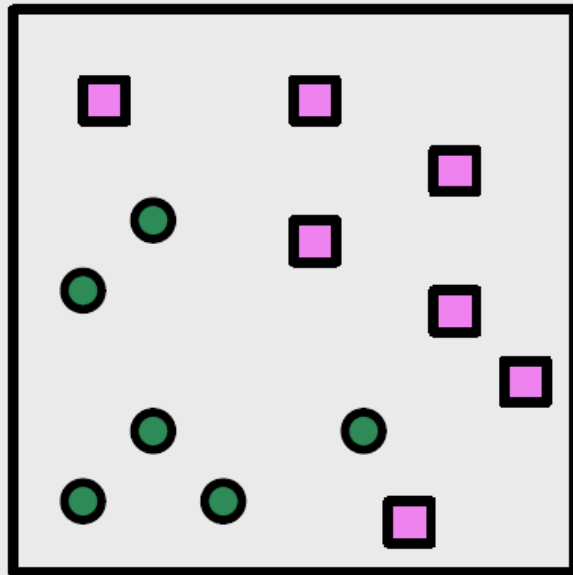
(from previous lecturers)



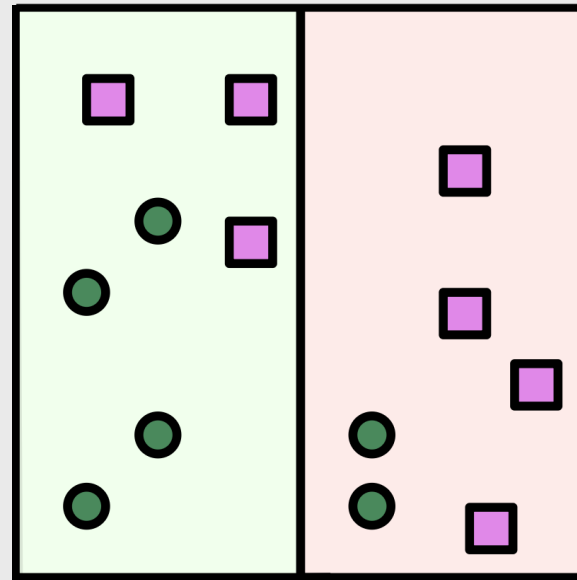
Sec 4.3, *Deep Learning* by C Bishop

Is the chosen model good?

(from previous lecturers)

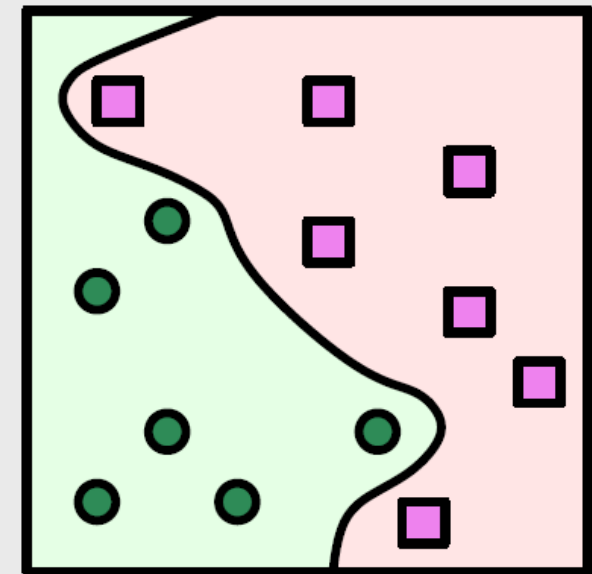


Training data



Underfit

High Bias
*performs poor on
both training and test*



Overfit

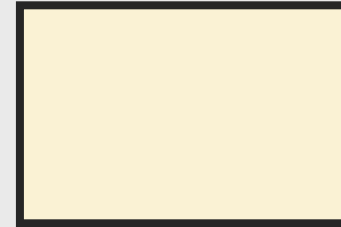
High Variance
*performs too well on
training but poor on test*

Avoid Overfitting

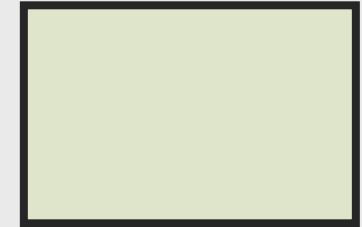
(from previous lectures)



Training Set



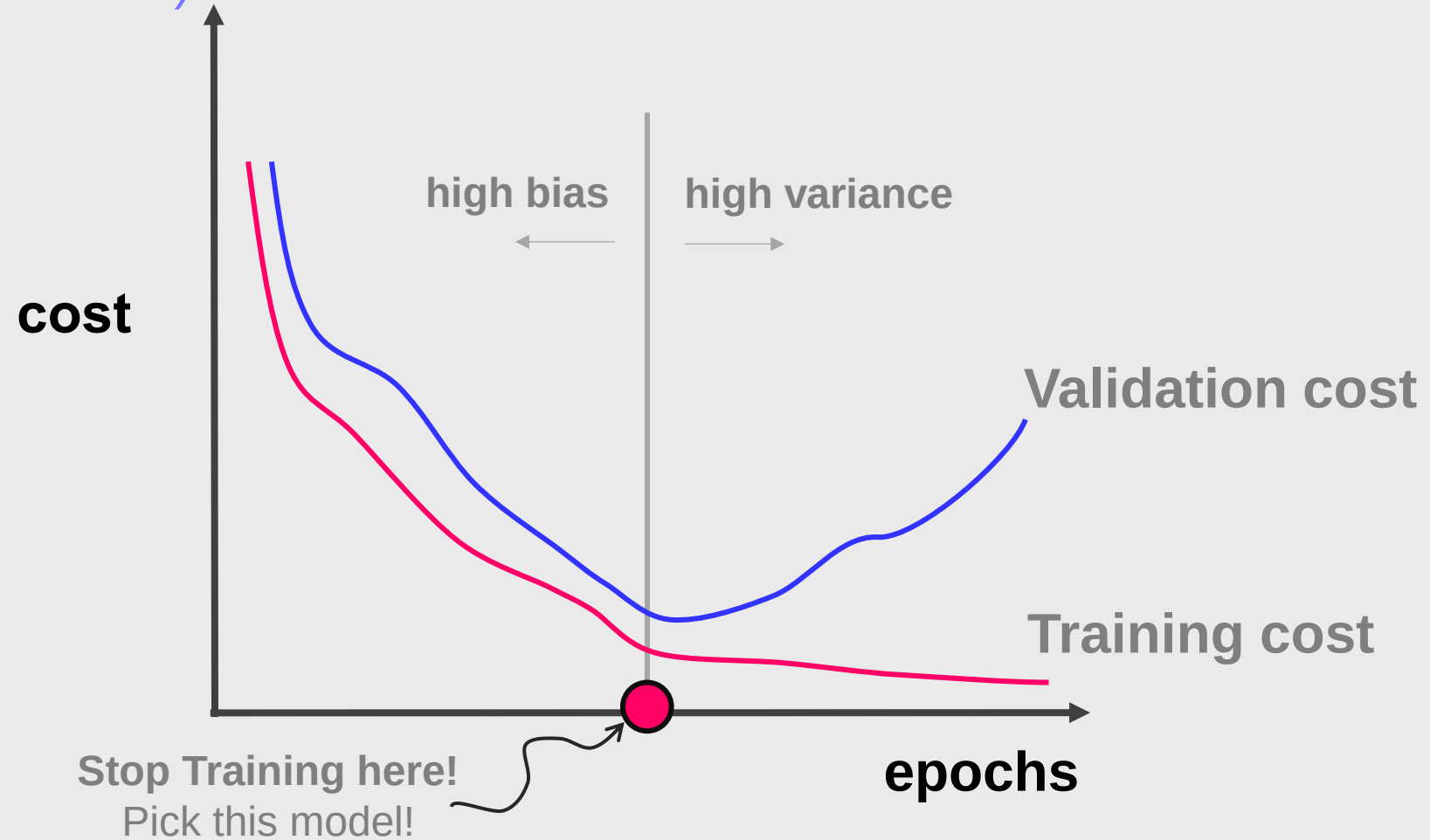
Validation Set



Test Set

Avoid Overfitting: Early Stopping

(from previous lecturers)



Suitable Loss function

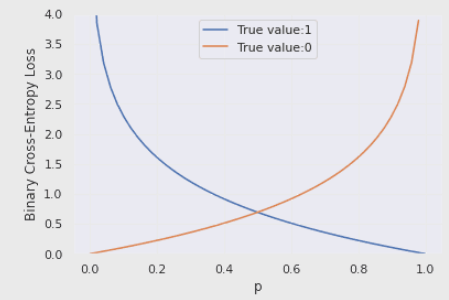
Misclassification rate, E

$$E = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i \neq y_i)$$
An orange and a red apple are positioned above the summation symbol in the equation, representing the two classes in a binary classification problem.

$$\frac{dE}{dw} = \mathbf{0} \text{ (or undefined)}$$

$\hat{y}_i$ - predicted output, y_i - target output, n - number of examples in training/test set

Misclassification Rate is Non-Differentiable for Backpropagation. Hence No learning possible as backpropagation can not calculate gradient



Log loss/Binary Cross Entropy

It effectively works by updating network weights on correct classification and penalizing models for a misclassification, usually used for binary classification problems

penalizing the model based on how close its predicted probability is to 0. This part will be zero if $y_i = 0$, hence this part is only for $y_i = 0$ (positive class)

if $y_i = 1$, penalizing the model based on how close its predicted probability is to 1, This part will be zero if $y_i = 1$, hence this part is only for $y_i = 0$ (negative class)

$$E = -((y_i)\log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i))$$

For N number of data samples

$$E = -\frac{1}{N} \sum_{i=0}^N ((y_i)\log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i))$$

$\hat{y}_i$ - predicted output, y_i - target output, n - number of examples in training/test set

Log loss/Binary Cross Entropy

It effectively works by updating network weights on correct classification and penalizing models for a misclassification, usually used for binary classification problems

Split this into Positive and Negative Classes

$$E = \left[-\frac{1}{N_{pos}} \sum_i^{N_{pos}} (y_i) \log(\hat{y}_i) \right] + \left[-\frac{1}{N_{neg}} \sum_i^{N_{neg}} (1 - y_i) \log(1 - \hat{y}_i) \right]$$

$$E = \left[-\sum_i^{N_{pos}} \frac{1}{N_{neg}} \cdot \mathbf{1} \cdot \log(\hat{y}_i) \right] + \left[-\sum_i^{N_{neg}} \frac{1}{N_{neg}} \cdot \mathbf{1} \cdot \log(1 - \hat{y}_i) \right]$$

$\hat{y}_i$ - predicted output, y_i - target output, N_{pos} - number of positive examples, and N_{neg} - number of negative examples

Log loss/Binary Cross Entropy

It effectively works by updating network weights on correct classification and penalizing models for a misclassification, usually used for binary classification problems

We replace Positive and Negative Classes as their probabilities

$$E = \left[- \sum_i^{N_{pos}} p(y_i = 1) \log(p(\hat{y}_i = 1)) \right] + \left[- \sum_i^{N_{neg}} p(y_i = 0) \log(p(\hat{y}_i = 0)) \right]$$

We replace Positive and Negative Classes as their probabilities

$$E = - \sum_{c \in \{0,1\}} \sum_i^{N_c} p(y_i) \log(p(\hat{y}_i))$$

$\hat{y}_i$ - predicted output, y_i - target output, N_c - number of examples for the value of c

Log loss/Binary Cross Entropy

It effectively works by updating network weights on correct classification and penalizing models for a misclassification, usually used for binary classification problems

We replace Positive and Negative Classes as their probabilities

$$E = \left[- \sum_i^{N_{pos}} p(y_i = 1) \log(p(\hat{y}_i = 1)) \right] + \left[- \sum_i^{N_{neg}} p(y_i = 0) \log(p(\hat{y}_i = 0)) \right]$$

We replace Positive and Negative Classes as their probabilities

$$E = - \sum_{c \in \{0,1\}} \sum_i^{N_c} p(y_i) \log(p(\hat{y}_i))$$

$\hat{y}_i$ - predicted output, y_i - target output, N_c - number of examples for the value of c

Cross Entropy, E

It effectively works by updating network weights on correct classification and penalizing models for a misclassification (larger the more is the penalty), usually used for multiclass classification problems

$$E = - \sum_{i=0}^C y_i \log(z_i)$$

$$\frac{dE}{dx_i} = z_i - y_i$$

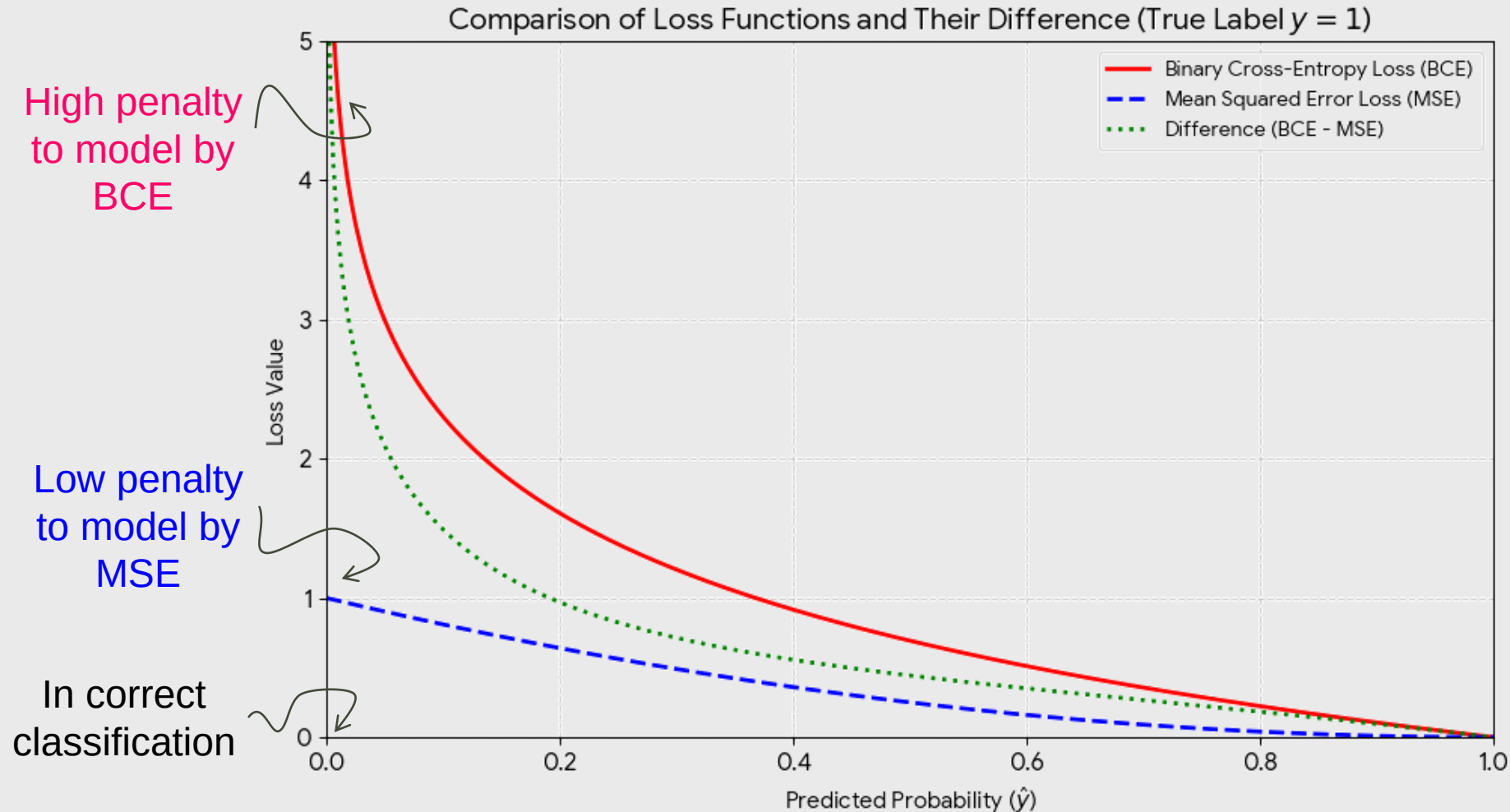
y_i - target output distribution (target out distribution , one-hot encoding)

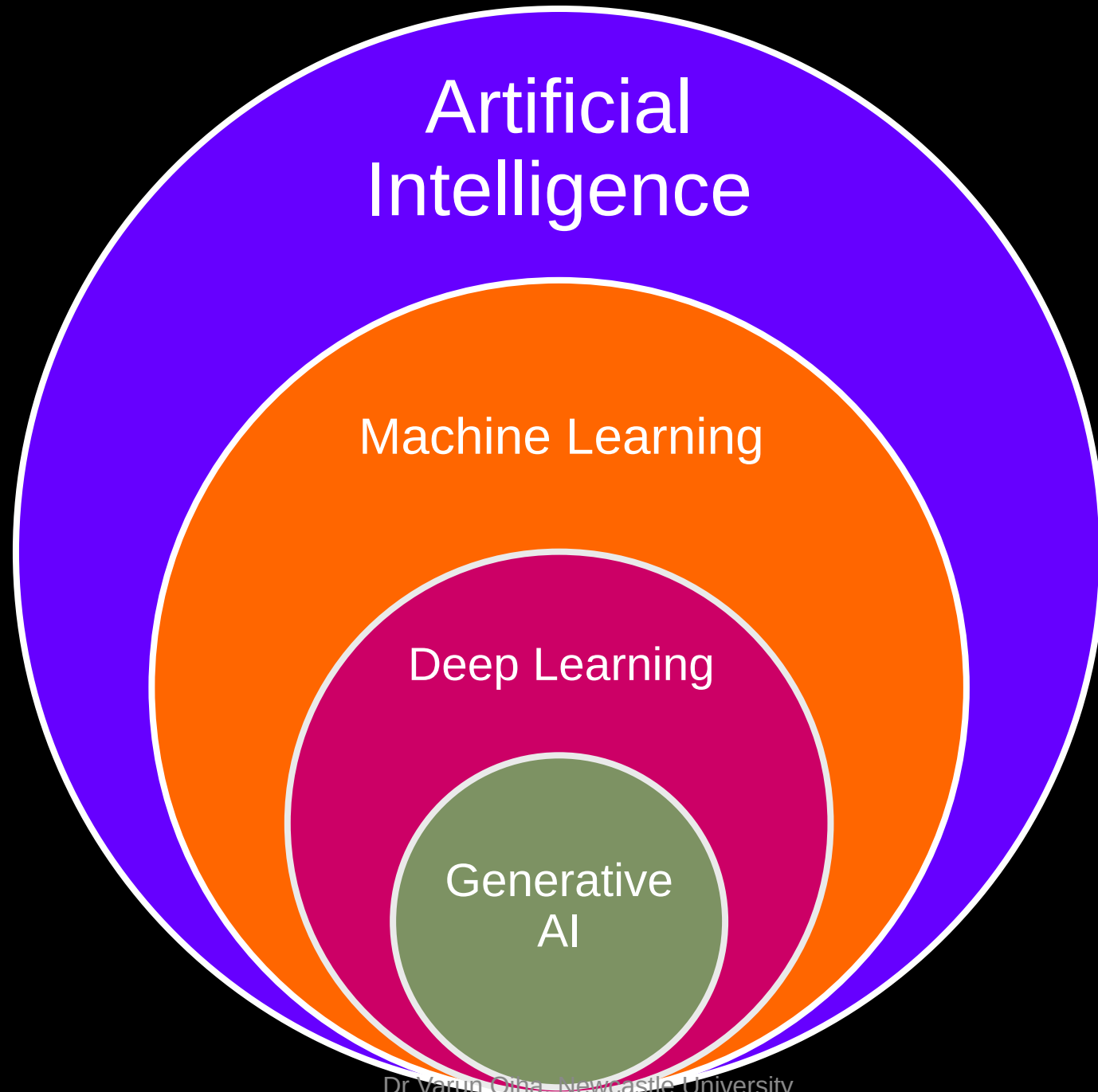
z_i - predicted output distribution (SoftMax output)

C - number of classes, w is the weight of a network

x_i - is the i -th class output

Why Cross-Entropy: When classifying between two categories (a binary classification problem), the **Binary Cross-Entropy (BCE) Loss** and the **Mean Squared Error (MSE) Loss** behave differently, particularly when the prediction is very wrong. The plots below illustrate this difference, assuming the true label is $y=1$. The core distinction is that **BCE Loss penalizes confident and incorrect predictions much more heavily** than MSE Loss does.





Part 4

Practical Session

See attached document in Canvas

